

Beyond Halting

Recurrence, Progress Deadlines and Diophantine Liveness

A progress-modulus hierarchy, quadratic transition laws,
and the limits of finite certificates

Manuscripts dated September 30, 2026; merged in the batch-62 write phase.

Central equivalence (source 10). For a computable binary-controlled system, there is a recurrent run with a computable progress deadline if and only if its accepting schedules contain a nonempty effectively closed subset, if and only if it has an accepting scheduler of hyperimmune-free degree.

Sharp classification (sources 10, 03 and 11). Finite reachability, a fixed recurrence deadline, some bounded-gap guarantee, some computable deadline, and unrestricted existential recurrence have respective worst-case complexities Σ_1^0 , Π_1^0 , Σ_2^0 , Σ_3^0 , Σ_1^1 ; eventual permanence, universal recurrence and universal eventual permanence add Σ_2^0 , Π_2^0 and Π_1^1 (source 11). Computable runs, computable deadlines and recurrence are strictly nested.

Low-degree laws (sources 03 and 11). One auxiliary-free polynomial of degree two gives the labelled transition relation of a counter machine, with coefficient height at most eight (source 03's control-vector encoding) or six (source 11's incoming-edge encoding); a fixed such law already has Σ_1^1 -complete recurrence, and degree one does not suffice.

Status. [write] A merge of three AI-assisted, unrefereed research manuscripts, two delivered in batch 62 and one in batch 63 of ProveIt's incoming reports (section 37). The theorems carry conventional mathematical proofs; the accompanying Python programs perform finite exact checks and are not proofs. No statement of this report is formalized in Lean or Rocq, although the report sits beside ProveIt's formal Hilbert's-tenth-problem project, whose MRDP interface it cites. Classical ingredients (MRDP, the low and hyperimmune-free basis theorems, Σ_1^1 -hardness of recurrence, recursive-tree index sets) are credited; literature-wide priority is claimed for nothing; no finite-fold or single-fold MRDP question is resolved. Every manuscript's own status statement is printed in section 3.

Abstract

[write] The MRDP theorem represents finite successful computations by existential polynomial equations; it does not represent every temporal property of a universal substrate. This report merges three manuscripts that locate that boundary for *recurrence*: the existence of a run that visits an accepting region infinitely often.

A shared *progress deadline* turns recurrence into an effectively closed condition, while checking each finite horizon separately does not (three independent proofs and three explicit counterexample machines). A computable deadline exists exactly when the accepting schedules contain a nonempty effectively closed class, exactly when an accepting schedule of hyperimmune-free degree exists; every c.e. degree is an exact deadline spectrum (source 10). The resulting hierarchy— Σ_1^0 reachability, Π_1^0 fixed deadlines, Σ_2^0 bounded gaps, Σ_3^0 computable deadlines and computable runs, Σ_1^1 recurrence, with strict separations inside one fixed system—is proved three times: for a fixed two-stack machine (source 10), for a fixed degree-two counter graph (source 03), and for a degree-two incoming-edge law (source 11, whose Σ_3^0 rows range over effectively supplied machines); source 11 adds the temporal rows Σ_2^0 , Π_2^0 and Π_1^1 .

On the certificate side, a counter network’s labelled transition relation is the natural zero set of one auxiliary-free polynomial of degree two (coefficient height at most 8 for a control-vector encoding, source 03; at most 6 for an incoming-edge encoding with a sum-of-squares quartic companion, source 11), and a single affine law cannot carry Σ_1^1 -complete recurrence. A two-stack quartic has natural zeros in bijection with rule-labelled runs meeting prescribed deadlines, with exactly $H(3 + R + G) + k$ variables (source 10); for counter machines the deadline certificate can be kept at degree two (source 11). Exact rational piecewise-affine realizations exist on $[0, 1]^3$ (source 10) and in a planar strip (source 11); faithful block simulations preserve every recurrence notion; and no total computable reduction maps any of these liveness properties to finite polynomial solvability.

Contents

1	The research question and the contribution boundary	5
1.1	What follows from universality, and what does not	5
1.2	Repository anchor and what was inspected	5
1.3	Prior work and the novelty ledger	6
1.4	How this report is organized	6
2	Conventions of the merged report	7
2.1	Sources, tags and labels	7
2.2	What is printed once	7
2.3	Notions and symbols across the three sources	8
3	The three manuscripts	9
3.1	Source 10: Beyond Halting	9
3.2	Source 03: Quadratic Diophantine Dynamics	10
3.3	The research question and the resulting contribution [source 03]	11
3.4	Source 11: Quadratic Laws, Infinite Computation	13
3.5	The research question and its scope [source 11]	13
I	Progress deadlines, compactness and the recurrence hierarchy	15
4	Operational semantics and progress clocks	15
4.1	A fixed binary-controlled effective system	15
4.2	Four recurrence predicates and one finite predicate	16
4.3	Conventions and logical prerequisites [source 03]	16

4.4	Semantics before equations [source 11]	18
5	A shared deadline restores compactness	19
5.1	The same trap in sources 03 and 11	21
6	The sharp progress-modulus hierarchy	23
6.1	Statement and conventions	23
6.2	Finite reachability and a fixed clock	24
6.3	An unknown bounded-gap guarantee	24
6.4	An unknown total computable deadline	24
6.5	Unrestricted recurrence and recursive trees	25
6.6	Why one fixed two-stack machine suffices	25
6.7	One hierarchy table for the three sources	26
6.8	Second routes through source 03's quadratic graph	27
6.9	Unrestricted recurrence is analytical-complete [source 03]	27
6.10	Existential computable bounds and computable paths [source 03]	29
6.11	Third routes through source 11's quadratic law	30
6.12	The finite-branching temporal hierarchy [source 11]	30
6.13	The exact cost of supplying a computable deadline [source 11]	31
II	Scheduler degrees, explicit separations and the deterministic case	32
7	Scheduler degrees: an exact characterization	32
7.1	Hyperimmune-free degrees and a basis argument	32
7.2	The main equivalence	33
8	Explicit separations and exact deadline-degree spectra	34
8.1	A computable clock need not compute a scheduler	34
8.2	A recurrent computation that requires a halting oracle	35
8.3	Realizing arbitrary c.e. deadline lower bounds	35
8.4	Computable schedules are a different Σ_3^0 property	37
8.5	Second routes to the separations	37
8.6	Two explicit strict separations [source 03]	37
8.7	Two explicit strict separations [source 11]	39
9	The deterministic specialization	41
III	Low-degree transition laws and finite certificates	42
10	Three compilers compared	42
11	An exact quadratic transition compiler [source 03]	43
11.1	The counter-network language	43
11.2	State coordinates and the polynomial	43
11.3	Coefficient height and circuit size	45
12	Why quadratic degree is minimal here [source 03]	45
12.1	A degree-one obstruction [source 11]	47
13	A fixed universal quadratic substrate [source 03]	47
14	Finite paths remain Diophantine [source 03]	48
15	An incoming-edge polynomial normal form [source 11]	49

15.1	Encoding the current control without a control integer	49
15.2	Exactness and the absence of hidden witnesses	50
15.3	A coefficient audit independent of machine size	51
15.4	A geometric observation	51
16	Finite histories and what their certificates mean [source 11]	52
16.1	The explicit quadratic certificate family	52
16.2	Arithmetical formulas and redundant witnesses	53
17	A fixed quadratic law with analytic-complete recurrence [source 11]	53
17.1	The classical source problem	53
17.2	Replacing infinitely many choices by binary delays	53
17.3	Uniformity and a fixed polynomial	54
18	Exact quartic certificates for two-stack computations	55
18.1	A canonical natural coding	55
18.2	Rule format and variables	55
18.3	The quadratic residuals	56
18.4	The polynomial and the bijection theorem	57
18.5	Deadline compactness as a quartic schema	58
IV	Transport, fairness and exact geometric realizations	58
19	Which simulations preserve a computable clock?	58
20	Transport to other effective computational substrates [source 03]	59
21	One weak-fairness obligation already reaches the analytic boundary [source 03]	60
22	A compact rational piecewise-affine substrate	62
22.1	Finite stacks as rational points	62
22.2	One deterministic map with the scheduler in the initial point	63
23	An unconventional realization: planar rational stack dynamics [source 11]	64
23.1	Finite stacks as rational points	64
23.2	Packing finite control into planar strips	65
23.3	Why this is Turing-complete, and how it reaches counters	65
23.4	Deadlines survive terminating simulations	66
V	Fixed-arity signatures, no-go theorems, implementation, formalization and questions	67
24	Fixed-arity Diophantine normal forms and impossibility results	67
24.1	Parameter tuples and degree reduction	67
24.2	The precise arithmetic signatures	67
24.3	Why a stronger universal polynomial compiler cannot exist	68
25	What finite Diophantine certificates can and cannot certify [source 03]	69
25.1	A useful positive fragment: finite recurrent regions	69
25.2	No hidden conclusion about finite-fold MRDP	70
26	Consequences for Diophantine representation [source 11]	70
26.1	A useful contract for future substrate compilers	71

27 The executable certificate package	71
27.1 Implementation surface	71
27.2 A completely explicit example	71
27.3 Executed tests and their limits	72
28 Executed verification and proof audit [source 03]	73
28.1 The highest-risk proof points	73
29 Executable artifacts and verification results [source 11]	74
29.1 What is implemented	74
29.2 The performed checks	74
29.3 Reproduction	74
30 Formalization and integration with ProveIt	75
30.1 A small trusted interface rather than an all-at-once port	75
30.2 Suggested module boundaries	75
30.3 Proof obligations most likely to conceal an error	76
31 Connection to ProveIt and a formalization plan [source 03]	77
31.1 The inspected source boundary	77
31.2 Suggested module separation	77
32 Relationship to the inspected ProveIt development [source 11]	78
32.1 Snapshot and mathematical dependencies	78
32.2 A feasible formalization boundary	78
32.3 Theorem-status ledger	79
33 Further research questions and concrete next targets	79
34 Conclusion	86
35 Conclusion [source 03]	86
36 Conclusion [source 11]	87
37 Provenance	87
A Reproduction and artifact manifest	89
B Reproduction guide [source 03]	90
C The explicit eight-dimensional budget example [source 11]	90
D A minimal proof checklist for reuse [source 11]	91
E A compact theorem-dependency ledger	92

1 The research question and the contribution boundary

1.1 What follows from universality, and what does not

Suppose a computational model has finitely encoded configurations, an effective finite execution semantics, and a finite event recognized by a semidecision procedure. Its successful finite executions form a computably enumerable relation. MRDP [4] then yields a fixed finite polynomial with natural witnesses for that relation. This reasoning is independent of whether the model is a Turing machine, a combinator system, a graph-rewriting machine, or an exact geometric simulator.

There is an important limit. “Some computation halts” and “some computation returns to an accepting region infinitely often” are different predicates. An infinite execution is not a natural number unless extra restrictions make it effectively describable, and existential quantification over all schedules cannot simply be replaced by existential quantification over program indices. This article asks three precise questions.

- Q1.** Which finite progress requirements allow the compactness argument that turns success at every finite horizon into one successful infinite run?
- Q2.** Exactly what computational strength is needed for a recurrence schedule when a computable clock exists?
- Q3.** Which of these temporal properties admit ordinary Diophantine representations, which need alternating number quantifiers, and which need an infinite witness?

The answers are not merely the observation that every computably enumerable relation is Diophantine. They identify the semantic obstruction, characterize its degree-theoretic boundary, and connect it to a concrete polynomial compiler and an unconventional geometric realization.

1.2 Repository anchor and what was inspected

The ProveIt repository contains a natural-number MRDP interface `Diophantine.mrdp`, its converse `Diophantine.mrdp_iff`, and an adapter to Mathlib’s `Dioph` predicate. The inspected theorem has the form

$$S \text{ is c.e.} \iff \exists r \exists P \in \mathbb{Z}[X, Y_1, \dots, Y_r] \forall n \in \mathbb{N} : n \in S \leftrightarrow \exists \vec{y} \in \mathbb{N}^r P(n, \vec{y}) = 0. \quad (1)$$

Both the polynomial and witness dimension are chosen before the input. The repository’s documentation explicitly separates this existence theorem from numerical degree bounds and from an executable polynomial-extraction algorithm [1, 2].

For reproducibility, the actual public MRDP source was inspected at commit

b6bf6406a4c49017fedc6fba068b36c634987150.

The MRDP guide and the Turing-degree project README were also inspected at revision `b998f70c6886e6a00339a6f4a02ed8625324d357`. These are source inspections, not a fresh build or an independent audit of every repository theorem. The present proofs use ordinary MRDP as a classical backend and do not rely on unrelated repository claims. The Turing-degree README distinguishes its basic Lean theory from deeper Rocq developments and from the separately conditional coarse-degree library [3]. No conditional coarse-degree theorem is used here.

1.3 Prior work and the novelty ledger

Recurrent computations were already used to obtain analytical-hierarchy hardness in the work of Harel, Pnueli, and Stavi; their recurrence problem is also explicitly recalled by Esparza and Krasotin [7, 10]. Thus the Σ_1^1 endpoint below is a classical phenomenon, not a new undecidability breakthrough. Jockusch and Soare’s low and hyperimmune-free basis theorems are likewise classical [5]; domination and its interaction with effectively closed classes are studied further in [6]. We use these results with explicit hypotheses.

Finitary acceptance asks for an unknown finite response bound. Chatterjee and Fijalkow investigate this in an automata-theoretic setting [8]. Their Borel classifications concern sets of infinite words; our hierarchy classifies *sets of finite input indices*. These are not the same hierarchy. Moore’s work supplies early dynamical universality precedent [9]. Our geometric section is an elementary exact construction for controlled guarded affine maps; it asserts neither continuity nor noise-robust physical computation.

Result or component	Status in this article
MRDP; low and HIF basis theorems	Classical ingredients, cited and separated from the new construction.
Analytical recurrence hardness	Classical endpoint, with a self-contained reduction supplied.
Clock/closed-class/HIF equivalence	Derived here in one operational formulation; no priority claim.
The full clock hierarchy	Complete proofs and timing-safe universal realizations supplied here.
C.e.-degree clock spectra	Explicit uniform realization of every c.e. degree; proof supplied here.
Exact two-stack quartic compiler	Explicit construction, exact counts, conventional proof, finite tested implementation.
Geometric and parameter regimes	Constructive consequences with the quantified initial-data domain stated explicitly.
Lean certification of these additions	Not performed; an integration plan is given.

The substantive outcome is a unified theorem package and an implementable arithmetization boundary, rather than an unsupported assertion that a longstanding named conjecture has been settled. Independent mathematical review and a wider priority search remain appropriate.

1.4 How this report is organized

[write] This report merges three manuscripts written independently on the same subject, recurrence beyond halting and its Diophantine boundary. Two were delivered in batch 62 of ProveIt’s incoming reports (manuscripts 10 and 03 of that batch), the third in batch 63 (manuscript 01 of that batch). They are called *source 10*, *source 03* and *source 11* here, after the file prefixes of their shipped programs and data; source 11’s number continues this report’s own sequence and is not a batch number (section 37). Source 10, whose introduction the preceding subsections reproduce, is the base: it treats arbitrary computable finitely branching (binary-controlled) systems and proves the strongest conclusions (the hyperimmune-free characterization, exact c.e. deadline spectra, the

bounded-gap row). Sources 03 and 11 prove the same hierarchy by other routes, through degree-two counter-machine transition laws of their own. The three titles, abstracts, status statements and the introductions of sources 03 and 11 are printed in [section 3](#).

After the conventions ([section 2](#)) the material is arranged by subject in five Parts:

Part	Subject	Sources
I	Progress deadlines, compactness and the recurrence hierarchy (part I)	10, 03, 11
II	Scheduler degrees, explicit separations and the deterministic case (part II)	10, 03, 11
III	Low-degree transition laws and finite certificates (part III)	03, 11, 10
IV	Transport, fairness and exact geometric realizations (part IV)	10, 03, 11
V	Fixed-arity signatures, no-go theorems, implementation, formalization and questions (part V)	10, 03, 11

A result proved by several sources is printed once and credits each source; a genuinely different proof is kept as a marked second (or third) route. The rule for which statements are merged is stated in [section 2.2](#).

2 Conventions of the merged report

2.1 Sources, tags and labels

[write] Text without a source marker is source 10's. Sections and subsections taken from the other sources carry a marker such as [\[source 03\]](#) in their headings, and every theorem-like statement they contribute lies inside such a section or names its source in its title. Text written in the merge is marked [write]. Inside printed source text, "this article", "this paper", "this report", "this manuscript" and "below" refer to that source; theorem, section and equation numbers are this report's. File names inside printed source text are the delivered names; [section 37](#) and the README map them to the shipped names, and displayed command listings keep the delivered layout, because the programs import one another by their delivered names. The README gives rerun commands that work on a copy.

Every label carries the prefix `lbh:`. Source 10's labels are `lbh:`+its own name, source 03's are `lbh:qd:`+its own name, and source 11's are `lbh:q1:`+its own name; labels written in the merge use `lbh:conv:`, `lbh:part:`, `lbh:sec:` and `lbh:q:`. No source label was dropped. Where a statement of source 03 or 11 is merged into a statement of source 10, its label sits on the printed statement.

2.2 What is printed once

[write] The three sources prove one hierarchy, but for different systems. A theorem about *all* systems of a class is the same theorem in each source, and is printed once: deadline compactness ([theorem 5.1](#)), the recursive-tree normal form ([lemma 6.2](#)), the diagonal-tree lemma ([lemma 8.5](#)), the deterministic boundary ([theorem 9.1](#)) and the Δ_2^0 consequence of a computable deadline ([theorem 7.3](#)). The other sources' proofs follow the printed statement as marked second and third proofs, after a sentence recording any difference of hypotheses. A completeness theorem, by contrast, is a statement about one fixed universal system; source 10 proves it for a two-stack machine, source 03 for its fixed quadratic counter graph E_U , and source 11 for its fixed quadratic law P_2 (and, for the Σ_3^0 rows, over effectively supplied machines). Completeness for one fixed system does not

imply it for another, so these statements are each printed once for their own system, in sections marked as second and third routes, and [section 6.7](#) collects all of them in one table. The same applies to the separating examples, the transport theorems and the finite-certificate obstructions. Nothing of any source was omitted: every result, proof, example, remark, limitation and question is printed.

2.3 Notions and symbols across the three sources

[write] Each source keeps its own names inside its own text; no mathematical symbol of any source was renamed. The dictionary below matches them.

Notion	Source 10	Source 03	Source 11
some run is recurrent	$B(e), \mathcal{B}_e \neq \emptyset$	$\text{Rec}(n)$	$\text{Rec}(\mathcal{M}, a), \text{EGFF}$
some run meets some computable deadline	$\text{CD}(e)$	$\text{CB}(n)$	$\text{DRec}(\mathcal{M}, a)$
some computable run is recurrent	$\text{Comp}(e)$	$\text{CP}(n)$	$\text{CRec}(\mathcal{M}, a)$
some run meets a fixed deadline	$\text{D}_d(e)$	$\text{Bound}_h(n)$	$\forall K \, W_g(K)$
some finite accepting visit	$\text{Reach}(e)$	$\text{Hit}(n)$	EFF
some infinite run	—	$\text{Inf}(n)$	EGF with F everything
some run with bounded gaps	$\text{BG}(e)$	—	—
deadline	$d(k)$, strictly increasing	$h(n, k)$, strictly increasing in k , $h \geq k$	$g(k)$, nondecreasing, $g(k) \geq k$
visit count	$N_{e,f}(t)$, times $1, \dots, t$	$V_s(T)$, times $1, \dots, T$	$\tau_k(\rho)$; $R_{g,K}$ sums from $t = 0$
k th visit time	$\tau_{e,f}(k)$	$\tau_s(k)$	$\tau_k(\rho)$
substrate	binary-controlled system; two-stack machine $\mathcal{M}$	counter network M ; universal U	unit-counter machine $\mathcal{M}$
state	(q, x, y) , stack codes v	(p, x, h) : one-hot control, counters, last-edge tag; $D = q + r + m$	(x, b) : counters, incoming-edge tag; $d + m$
transition polynomial	Q , a quartic on a trace	E_M , degree 2, height ≤ 8	P_2 , degree 2, height ≤ 6 ; P_4 , SOS quartic
diagonal (DNR) tree	T_{DNR}	T_D	$\mathcal{T}_{\text{diag}}$
halting-time tree	tree of $g \geq B$ (section 8)	T_H	$\mathcal{T}_{\text{halt}}$

The letters below change meaning across the sources; the tempting false reading is to carry a letter across a section boundary.

- **Deadlines.** Source 10's d and source 03's $h(n, k)$ are strictly increasing; source 11's g is only nondecreasing with $g(k) \geq k$ (so g may repeat values). Source 10's compactness proof applies verbatim to nondecreasing unbounded deadlines, and source 11 proves that case separately ([theorem 5.1](#)). The letter g also names, in source 10, a path of a tree ([section 8](#)), the expansion bound g_e of [theorem 19.2](#) and the bounds g_e of [lemma 7.2](#); none of these is a deadline.

- *h*. Source 10: a computable bound (theorem 7.3). Source 03: both the last-edge tag in states (p, x, h) and the deadline $h(n, k)$. Source 11: the halting time $h(e)$ of $\varphi_e(e)$ (equation (32)), and the homogenizing variable of section 15.
- *H*. Source 10: the horizon of the quartic compiler. Source 03: a halting control label in equation (18). Source 11: the history polynomial H_T and a control label.
- *D*. Source 10's $D = D$ is a sans-serif predicate; source 03's and source 11's italic D is a state dimension; source 03's T_D is its diagonal tree.
- *E, R, Q, A, T*. *E*: source 10's number of empty guards, source 03's energy E_M and source 11's edge set. *R*: source 10's number of rules, a decidable relation in all three sources, and source 11's certificate $R_{g,K}$. *Q*: source 10's trace quartic, and the set of control labels in sources 03 and 11 (the rationals are $\mathbb{Q}$). *A*: the accepting predicate or marker in all three sources (source 11's $A(b)$ is a linear form on tags), a control label in the examples of sources 03 and 11, and a c.e. set in theorem 8.3. *T*: a tree or a horizon in every source.
- **Stack codes**. Source 10 uses the natural code v (equation (48), base four with a sentinel) and the rational code $\rho(w) = \sum_i d_i 4^{-i}$, $d_i \in \{1, 3\}$ (equation (63)). Source 11's rational code $\eta(w) = \sum_j (1 + 2a_j) 4^{-j}$ with $a_j \in \{0, 1\}$ (equation (71)) is source 10's ρ after the digit relabelling $d = 1 + 2a$, and source 11's integer code $N(w)$ is base three. Source 10's η is something else: the real code of a whole schedule (equation (68)). The letter ρ also names a run in source 11.
- **Visit times**. Sources 10 and 03 count accepting visits at times $t \geq 1$. Source 11's $\tau_k(\rho)$ counts positions, and its certificate $R_{g,K}$ sums the marker from $t = 0$. The difference is harmless: in every construction of the three sources the initial configuration is unmarked (source 11's initial state carries the dummy incoming edge e_0 , whose target is the start label, and its start labels are unmarked).
- **Heights 8 and 6**. Source 03's bound $\text{ht}(E_M) \leq 8$ and source 11's $\text{ht}(P_2), \text{ht}(P_4) \leq 6$ concern *different encodings*: source 03 keeps a one-hot control vector and separate source and target residuals, source 11 drops the control vector and merges both into one flow residual. Each bound is sharp for its own compiler (theorem 11.4, section 15.3). The tempting false reading "source 11 improves source 03's sharp height 8 to 6" is wrong; neither contradicts the other's sharpness (section 10).

3 The three manuscripts

[write] Each subsection gives one manuscript's title and date as delivered, its abstract and status statement verbatim, and, for sources 03 and 11, its own introduction; source 10's introduction is section 1. The delivered author lines named the assistant used or the addressee of the manuscript; they are replaced here by the neutral description "AI-assisted research manuscript" (section 37).

3.1 Source 10: Beyond Halting

[write] *Beyond Halting: A Progress-Modulus Hierarchy for Diophantine Liveness. Effective compactness, scheduler degrees, and rational piecewise-affine computation.* Research article and exact certificate implementation; AI-assisted research manuscript, dated September 30, 2026. Batch 62, manuscript 10; base of this report. Its title-page summary box read as follows.

Central equivalence. For a computable binary-controlled system, there is a recurrent run with a computable progress deadline if and only if its accepting schedules contain a nonempty effectively closed subset, if and only if it has an accepting scheduler of hyperimmune-free degree.

Sharp classification. Finite reachability, a fixed recurrence deadline, some bounded-gap guarantee, some computable deadline, and unrestricted existential recurrence have respective worst-case complexities

$$\Sigma_1^0, \quad \Pi_1^0, \quad \Sigma_2^0, \quad \Sigma_3^0, \quad \Sigma_1^1.$$

All five are realized by one fixed two-stack system and its exact rational piecewise-affine realization.

Its status statement:

Status. This is a research manuscript with detailed mathematical proofs and a tested finite-horizon compiler, not a claim of a new proof of MRDP, a solution of the finite-fold conjecture, or established publication priority. The basis theorems and classical recurrence hardness are explicitly attributed. The integrated progress-clock formulation, exact compiler, and applications are developed here. No new Lean formalization is included or claimed.

Its abstract:

The Matiyasevich–Robinson–Davis–Putnam theorem represents finite successful computations by existential polynomial equations. It does not represent every property of a universal computational substrate: the temporal predicate and its quantifiers matter. We analyze existential recurrent reachability in computable, finitely branching systems through *progress deadlines*. A shared deadline converts recurrence into an effectively closed condition; checking each finite horizon independently without retaining earlier deadlines does not. We prove a three-way equivalence between a computable deadline, a nonempty effectively closed family of accepting schedules, and an accepting schedule of hyperimmune-free Turing degree. We give full upper-bound and hardness constructions for the resulting arithmetical/analytical hierarchy, separate computable clocks from computable schedulers, and construct a system with recurrent runs but no computable deadline. More generally, every c.e. Turing degree is realized as the exact minimum oracle degree needed to compute a successful deadline.

An explicit two-stack compiler produces a nonnegative quartic polynomial whose natural zeros correspond bijectively to rule-labelled finite runs satisfying all prescribed deadlines. With horizon H , R rules, G top guards, E empty guards, and k deadlines, it uses exactly $H(3+R+G)+k$ variables and $H(4R+E+2G+2)+k$ quadratic residuals. The same semantics are realized by a fixed guarded rational piecewise-affine relation on a compact three-dimensional cube. A fourth coordinate internalizes the scheduler as an initial real parameter, yielding a deterministic map. We explain the different classifications for rational, computable, and unrestricted initial parameters, derive fixed-arity polynomial quantifier normal forms using ProveIt’s MRDP interface, and specify concrete subsequent formalization and research tasks. The accompanying dependency-free Python implementation includes exact arithmetic tests and an exported sample certificate.

3.2 Source 03: Quadratic Diophantine Dynamics

[write] *Quadratic Diophantine Dynamics: Recurrence, computable schedules, and the limits of finite certificates*. Computability / Diophantine representation; AI-assisted research manuscript, dated 30 September 2026. Batch 62, manuscript 03. Its abstract:

A finite counter network can be compiled into a single integer polynomial $E(s, s')$ of degree two whose natural zero set is exactly its labelled transition relation. The compiler uses no existential

auxiliary coordinates: its state dimension is $q + r + m$ for q controls, r counters and m edges, and every expanded coefficient has absolute value at most eight. A mixed nonnegative zero-test term, rather than a squared zero-test equation, is the key to retaining quadratic degree. Binary branching is preserved globally.

This normal form supports a sharp study of infinite computation. For one fixed universal polynomial, existence of a recurrent path is Σ_1^1 -complete; existence of a recurrent path with some total computable return-time bound is Σ_3^0 -complete; and existence of a computable recurrent path is also Σ_3^0 -complete. Nevertheless, the last two properties are distinct, and both are strictly weaker than unrestricted recurrence. Explicit recursive-tree constructions separate them. A nine-coordinate example refutes the tempting inference from arbitrarily many finite visits to one recurrent path. A matching affine obstruction proves that degree two is minimal for this uniformly bounded-branching recurrence universality. We also give an explicit weak-fairness translation, finite-certificate boundaries, executable compiler checks, and a proposed ProveIt integration plan.

Its status statement:

Status. Conventional proofs and executed finite tests; not Lean- or Rocq-verified, independently refereed, or claimed to solve a named published open problem. Classical recurrence and fairness undecidability are credited as background. The exact normal form, quantitative bounds, and combined separation results are the contributions developed here; literature-wide priority remains unverified.

Its introduction follows.

3.3 The research question and the resulting contribution [\[source 03\]](#)

The finite computational behavior of an effective machine has a familiar arithmetic description: a successful execution is a finite object, and a checker can verify it. The more delicate question is whether the same representation remains possible for a machine that must keep succeeding forever. The issue is not the exotic appearance of the substrate. It is the quantification over an entire infinite trajectory.

ProveIt’s inspected Diophantine library already provides exact finite iteration and finite reachability interfaces, as well as MRDP in both directions [\[12, 11\]](#). We therefore investigate the next semantic boundary instead of merely applying MRDP again to another effective halting predicate. The central questions formulated in this manuscript are: can one obtain a particularly small-degree local arithmetic dynamics; what infinite properties survive as ordinary Diophantine sets; and what changes when the times of recurrent events admit a computable bound?

3.3.1 A quadratic, not quartic, local representation

The construction uses an energy of the form

$$E = \text{sum of squares of affine residuals} + \text{sum of nonnegative selector-counter products.} \quad (2)$$

The second part enforces zero tests. Squaring those products would raise the degree to four, but is unnecessary on the nonnegative orthant. Integrality of a selector vector whose coordinates sum to one chooses exactly one edge. The selected edge is written into the successor state, so two labelled edges remain distinct without existential choices hidden behind a transition.

The resulting substrate is a discrete zero-energy relation, not iteration of a polynomial map and not a continuous differential equation. This distinction is essential. The relation can branch, its coordinates are natural numbers, and its zero tests are complementarity constraints. Integrality supplies the discrete choice.

3.3.2 The main theorem package

The following overview refers to precise statements proved below. All completeness assertions use computable many-one reductions.

Result	Quantitative or logical conclusion
Local counter-network compiler	Degree 2; height ≤ 8 ; dimension $q + r + m$; no existential auxiliaries
Affine obstruction	Degree 1 cannot have the same recurrent universality with a uniform finite branching bound
Finite hit in a fixed universal system	Σ_1^0 -complete; ordinary Diophantine
Existence of an infinite path	Π_1^0 -complete
Unrestricted recurrent path	Σ_1^1 -complete
A prescribed computable return-time bound	Π_1^0 upper bound; sharp for suitable uniformly specified families
Some computable return-time bound	Σ_3^0 -complete
Some computable recurrent path	Σ_3^0 -complete, but strictly stronger than the preceding existence property
One always-enabled weak-fairness obligation	Σ_1^1 -complete; branching at most 3 after translation

Here “some computable return-time bound” means that a bound exists for one path, not that an algorithm must compute a bound from the input. It also does not mean that the bounded path itself is computable. These two distinctions are responsible for the middle part of the classification.

3.3.3 What is and is not new

Counter-machine universality is classical [13]. Highly undecidable recurrence and its relation to infinite trees and fairness are also classical: Harel’s work is directly relevant [14, 15]. Finkel gives related analytical-hierarchy classifications for infinite tiling systems [16]. We do not present the general phenomenon of Σ_1^1 recurrence as a discovery.

The research contribution here is the explicit conjunction of a quadratic, height-eight, auxiliary-free transition normal form; a degree-one lower obstruction under the same global branching requirement; the exact computable-deadline classification; and concrete strict-separation constructions that survive this arithmetic compilation. The proofs are given in full except for explicitly identified standard computability background. No claim of literature-wide priority is made. The construction for a fixed universal system is effective but its full low-level universal interpreter is not numerically instantiated in the delivered code. The code does instantiate and test the local compiler and the small counterexamples.

3.4 Source 11: Quadratic Laws, Infinite Computation

[write] *Quadratic Laws, Infinite Computation: Exact Diophantine Dynamics, Planar Stack Machines, and the Boundary Created by Recurrence Deadlines*. AI-assisted research report, presented by its author line as an extension of the Diophantine and computability program in Provelt; dated September 30, 2026. Batch 63, manuscript 01; its file prefix is 11-. Its abstract (source 11 has no separate status statement; its status is stated in its abstract, in section 32 and in its theorem-status ledger):

Finite computation and infinite behavior have sharply different arithmetic content, even when they obey the same low-degree local law. We construct, from any binary-branching unit-counter machine with d counters and m edges including one initialization edge, a polynomial $P_2(X, Y) \in \mathbb{Z}[X, Y]$ in $2(d + m)$ variables. It has degree at most two, coefficient height at most six, and is nonnegative on the entire nonnegative real orthant. Its natural-number zeros are exactly the machine steps, without existential transition witnesses. A companion polynomial P_4 has the same natural zeros, the same coefficient bound and dimension, and is globally a sum of squares of degree at most four. Both preserve edge-labelled finite and infinite computations exactly.

A fixed instance of this construction has a Σ_1^1 -complete recurrent reachability problem, although its one-step graph is semilinear and has out-degree at most two. We then isolate a finer boundary: existence of a recurrent run with a computable bound on its k th marked visit is Σ_3^0 -complete. Existence of a computable recurrent run is also Σ_3^0 -complete, but is a strictly stronger property. Two explicit diagonal constructions separate these notions from unrestricted recurrence. A prescribed total computable deadline restores an effective compactness criterion, expressed by compatible finite quadratic certificates.

The same distinctions transfer to exact rational piecewise-affine dynamics in a bounded planar region through an explicit two-stack coding. The article includes proofs, a coefficient audit, an executable compiler, exact finite tests, a repository integration plan, and further research questions. The classical high-undecidability results are credited; the coefficient-controlled normal form and deadline refinements are presented as proved results of this report, not as an established claim of literature priority.

Its keywords: Diophantine representation; counter machines; Büchi recurrence; effective compactness; computable deadlines; piecewise-affine computation; analytic hierarchy. Its introduction follows.

3.5 The research question and its scope [\[source 11\]](#)

3.5.1 Why another compilation theorem is not enough

The MRDP theorem identifies the computably enumerable subsets of $\mathbb{N}$ with sets of the form

$$n \in S \iff \exists w \in \mathbb{N}^r H(n, w) = 0, \quad H \in \mathbb{Z}[n, w_1, \dots, w_r]. \quad (3)$$

The polynomial and its finite number of witnesses are fixed before n is chosen. The repository exposes precisely this interface, including input zero, and proves its converse as well [19]. Thus finite acceptance in any effectively presented Turing-complete substrate is already Diophantine at the level of existence. Merely changing the substrate does not by itself improve that theorem.

A more demanding question is what the same arithmetic representation says about a computation that must continue forever, repeatedly make progress, or obey a schedule. These properties occur in interactive computation, reactive systems, nondeterministic verification, infinite tilings, and exact

dynamical simulations. A finite accepting history is not a certificate that an infinite history exists with infinitely many marked events.

This report addresses three concrete questions.

- Q1.** How simple can a single polynomial local transition law be while its infinite behavior has genuinely non-arithmetical complexity?
- Q2.** Which additional hypothesis makes finite-prefix reasoning valid for recurrence, and what is the exact complexity of existentially supplying that hypothesis as an algorithm?
- Q3.** Do these distinctions survive translation to an unconventional, low-dimensional computational substrate without relying on arbitrary real oracles or hidden infinite-precision input advice?

The answers are a nonnegative quadratic local law, a computable recurrence deadline, and an exact rational planar stack realization, respectively.

3.5.2 A statement of the principal conclusions

Theorem 3.1 (Summary of the proved results). *The following assertions hold.*

- (i) *Every finite binary-branching unit-counter machine has the explicit witness-free quadratic and quartic presentations of theorem 15.1, with coefficient height at most six and state dimension $d + m$.*
- (ii) *For one fixed quadratic presentation, the set of effectively encoded initial states admitting a run with infinitely many marked visits is Σ_1^1 -complete. Its one-step natural relation is nevertheless semilinear and has out-degree at most two.*
- (iii) *Over effectively presented finite counter machines, both existence of a computable recurrent run and existence of a recurrent run with some total computable deadline are Σ_3^0 -complete. The former implies the latter, but not conversely. Unrestricted recurrence is strictly more general than both.*
- (iv) *For a prescribed total computable deadline, recurrence is equivalent to the satisfiability of every member of a compatible family of finite quadratic systems. Each system has unique slack variables once the finite path is fixed.*
- (v) *The qualitative distinctions in (ii)–(iv) are realizable by exact rational piecewise-affine two-stack dynamics in the plane.*

These are statements about ordinary standard natural numbers and actual infinite sequences. They are not claims about nonstandard finite runs or formal satisfiability inside an arbitrary model of arithmetic.

3.5.3 What is classical, what is developed here

The analytic completeness of recurrent nondeterministic counter computation is classical, associated in particular with Harel's work on infinite trees, dominoes, and fairness [15]. A recent primary source explicitly records the two-counter formulation [20, Appendix A]. Generalized shifts and computational dynamics likewise have a substantial literature [22]. We do not rename these established results as new discoveries.

The work developed here is the exact incoming-edge quadratic normal form, its uniform coefficient audit, the compatible deadline certificates, the explicit strict witness separations, and their integration into one substrate-preserving framework. The Σ_3^0 arguments are proved directly rather than asserted by analogy. Related recursive-tree index-set techniques are also classical. In particular, the Σ_3^0 classification of trees having a computable path appears in [21, Theorem 2.10(e)]; our proof specializes this machinery to recurrent runs and deadlines. A comprehensive priority check for the precise combination of refinements in this report remains necessary. No named long-standing open conjecture is claimed resolved, and no new Lean or Rocq verification is claimed.

Part I

Progress deadlines, compactness and the recurrence hierarchy

[write] *Part I prints source 10's semantics, compactness theorem and hierarchy, the conventions of sources 03 and 11, the three counterexamples to naive compactness, and the second and third routes to the hierarchy through the fixed quadratic systems of sources 03 and 11. The degree-theoretic characterization of computable clocks is in Part II.*

4 Operational semantics and progress clocks

4.1 A fixed binary-controlled effective system

Definition 4.1. A *computable binary-controlled system* consists of natural-number-coded configurations c , a total computable transition function $\delta(c, b)$ for $b \in \{0, 1\}$, a computable initial configuration $i(e)$ for finite input $e \in \mathbb{N}$, and a decidable accepting predicate $A(c)$. A schedule $f \in \{0, 1\}^{\mathbb{N}}$ determines

$$c_0 = i(e), \quad c_{t+1} = \delta(c_t, f(t)).$$

Halting, disabled actions, or invalid configurations can be completed by an absorbing rejecting sink. A schedule is an arbitrary infinite bit sequence, not necessarily computable.

The transition computation is total; therefore the finite run determined by a finite bit string can be computed completely. Deterministic steps may ignore their action bit. More general effective finite branching can be encoded by finite action alphabets, but the binary formulation makes the compactness and degree arguments transparent. For a rule-based compiler, we retain the rule label as part of the finite trace.

We count accepting visits at times $1, 2, \dots$, excluding the initial time:

$$N_{e,f}(t) = \sum_{u=1}^t \mathbf{1}_{A(c_u)}, \quad \tau_{e,f}(k) = \min\{t : N_{e,f}(t) \geq k\}. \quad (4)$$

The second expression is defined only when the k th visit occurs. We often suppress e in the notation. The set of accepting schedules is

$$\mathcal{B}_e = \{f : \forall k \geq 1 \exists t N_{e,f}(t) \geq k\}. \quad (5)$$

Thus acceptance is the Büchi condition of infinitely many visits.

Definition 4.2. A *deadline* is a strictly increasing function $d : \mathbb{N}_{>0} \rightarrow \mathbb{N}_{>0}$. In particular $d(k) \geq k$. A schedule meets d when $N_{e,f}(d(k)) \geq k$ for every $k \geq 1$. Write

$$\mathcal{K}_{e,d} = \{f : \forall k \geq 1 \ N_{e,f}(d(k)) \geq k\}.$$

For a finite deadline vector $\vec{d} = (d(1), \dots, d(k))$, let

$$\text{Fin}(e, \vec{d}) \iff \exists \sigma \in \{0, 1\}^{d(k)} \forall j \in \{1, \dots, k\} : N_{e,\sigma}(d(j)) \geq j. \quad (6)$$

The empty vector is feasible. Every earlier deadline is retained.

For a finite explicit vector, Fin is decidable: enumerate its finitely many binary schedules and compute their finite runs. The procedure can be very expensive, but the hierarchy requires decidability, not an efficient bound. When d is computable, $(e, k) \mapsto \text{Fin}(e, d \upharpoonright k)$ is decidable.

4.2 Four recurrence predicates and one finite predicate

For a fixed computable deadline d , define

$$D_d(e) \iff \mathcal{K}_{e,d} \neq \emptyset, \quad (7)$$

$$\text{BG}(e) \iff \exists f \exists b \geq 1 \forall t \geq 0 : N_{e,f}(t+b) - N_{e,f}(t) \geq 1, \quad (8)$$

$$\text{CD}(e) \iff \exists \text{ a total computable deadline } d : \mathcal{K}_{e,d} \neq \emptyset, \quad (9)$$

$$\text{B}(e) \iff \mathcal{B}_e \neq \emptyset, \quad (10)$$

$$\text{Reach}(e) \iff \exists f \exists t \geq 1 : A(c_t). \quad (11)$$

The gap condition includes the initial delay: the first visit occurs by b , and successive visits are at most b steps apart. Requiring a bound only after some finite prefix gives the same *existence* predicate, because a finite prefix and finitely many initial gaps can be absorbed into a larger bound. However, a linear bound $\tau_f(k) \leq bk$ is not the same as a bound on every individual gap. We never use those two notions interchangeably.

There are immediate inclusions

$$D_{\text{id}} \subseteq \text{BG} \subseteq \text{CD} \subseteq \text{B}, \quad \text{where } \text{id}(k) = k. \quad (12)$$

The second inclusion follows by taking $d(k) = bk$.

[write] *The next subsection prints source 03's conventions (its Section 2). Its predicates Hit, Rec, CB, CP are source 10's Reach, B, CD, Comp (section 2.3); its systems are effectively finitely branching rather than binary-controlled.*

4.3 Conventions and logical prerequisites [\[source 03\]](#)

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. All polynomial coefficients belong to $\mathbb{Z}$; all state and witness coordinates belong to $\mathbb{N}$ unless another domain is explicitly specified. For a polynomial P , its coefficient height $\text{ht}(P)$ is the maximum absolute value of an expanded coefficient. This is unrelated to the size of a state or the duration of a computation.

Definition 4.3 (Effective transition system). An effectively finitely branching system consists of a decidable state space with an algorithm that, on every state, returns its complete finite list of successors. In our applications the state space is $\mathbb{N}^D$, the list has length at most two, the initial state $c(n)$ is computable from an input n , and a marked-state predicate $A(s)$ is decidable.

The complete-list requirement is stronger than saying that the edge relation is decidable and every vertex has at most two successors. Merely enumerating possible successors does not in general tell us when that enumeration is finished. Our compiler supplies the complete-list algorithm directly from the finite edge table.

A finite path of length T has states $s_0, \dots, s_T$. An infinite path has states s_t for every $t \in \mathbb{N}$. A path starts at $c(n)$ when $s_0 = c(n)$. We count marked visits at positive times; thus

$$V_s(T) = \#\{t : 1 \leq t \leq T, A(s_t)\}. \quad (13)$$

Omitting a possible visit at time zero does not change recurrence. A recurrent path is an infinite path with unbounded $V_s(T)$. Its k th marked time, for $k \geq 1$, is $\tau_s(k) = \min\{T : V_s(T) \geq k\}$.

Definition 4.4 (Four existence predicates). For a fixed system and initial-state map, let

- Hit(n) : some finite path reaches a marked state,
- Rec(n) : some infinite path is recurrent,
- CB(n) : some recurrent path has a total computable bound on $\tau_s(k)$,
- CP(n) : some recurrent path $t \mapsto s_t$ is computable.

The definition of CB permits an arbitrary, possibly noncomputable path. Only its numerical upper bound must be computable.

A bound can always be replaced by a strictly increasing bound h with $h(k) \geq k$: replace $g(k)$ by $k + \max_{1 \leq j \leq k} g(j)$. Consequently we impose these two conditions on deadline functions without loss of generality when existentially quantifying over them.

4.3.1 Arithmetic and analytic complexity

A predicate is Σ_1^0 when it has the form $\exists u R(n, u)$ with computable R ; it is Π_1^0 when it has the form $\forall u R(n, u)$. The pattern $\exists a \forall b \exists c R(n, a, b, c)$ defines Σ_3^0 . Finite blocks of consecutive quantifiers can be coded by single natural numbers. A Σ_1^1 predicate permits an existential function quantifier, followed by an arithmetic predicate:

$$n \in S \iff \exists f \in \mathbb{N}^{\mathbb{N}} \theta(n, f).$$

A complete set for a class is one to which every set in that class reduces computably. We use the standard strictness of the arithmetic and analytical hierarchies, and in particular that Σ_1^1 -complete sets are not arithmetic [16] ([write] a textbook reference is [17]).

Definition 4.5 (Ordinary Diophantine representation). A set $S \subseteq \mathbb{N}$ has an ordinary Diophantine representation when there exist a fixed finite v and $P \in \mathbb{Z}[X, Y_1, \dots, Y_v]$ such that

$$n \in S \iff \exists y_1, \dots, y_v \in \mathbb{N} P(n, y_1, \dots, y_v) = 0.$$

Every such set is computably enumerable, by exhaustive enumeration of finite witness tuples. The converse is MRDP, which is available in the inspected ProVeIt library [11]. An infinite list of polynomial equations with an existentially quantified infinite sequence is *not* an ordinary Diophantine representation in this sense.

[write] *The next subsection prints source 11's semantics (its Section 2). Its Rec, CRec, DRec are source 10's B, Comp, CD; its deadlines are nondecreasing (section 2.3).*

4.4 Semantics before equations [\[source 11\]](#)

4.4.1 Counter machines with at most two choices

A *unit-counter machine* consists of a finite set Q of control labels, a start label q_0 , a marked subset $F \subseteq Q$, and $d \geq 1$ natural counters. Its edges are of four types:

$$\text{inc}(i), \quad \text{dec}(i), \quad \text{zero}(i), \quad \text{nop}.$$

An increment adds one to counter i . A decrement is enabled only when that counter is positive and subtracts one. A zero edge is enabled only when the counter is zero and changes no counter. A no-op is always enabled and changes no counter. Every ordinary control label has at most two outgoing edges. Different edges may have identical endpoints and effects; they remain distinct choices.

The usual zero-test/decrement instruction has one zero edge and one decrement edge. A nondeterministic binary choice has two no-op edges. Standard counter-machine programs can therefore be put into this form by inserting finitely many control labels. Marker visits are assigned only to the intended instruction boundaries, not to every internal simulation state.

A configuration is (q, x) with $x \in \mathbb{N}^d$. A finite path of length T contains T edges and $T + 1$ configurations. An infinite run has one configuration at each $t \in \mathbb{N}$. Deadlocks have no successors. All temporal properties below quantify over genuine infinite runs unless explicitly stated otherwise.

4.4.2 Three different recurrent witnesses

For a marked run ρ , let $\tau_k(\rho)$ be the time of its k th marked configuration, where $k = 1, 2, \dots$. Visits count time positions, not distinct marked configurations. Define:

$$\begin{aligned} \text{Rec}(\mathcal{M}, a) &\iff \exists \rho \text{ from } a \ \forall k \ \tau_k(\rho) \text{ exists,} \\ \text{CRec}(\mathcal{M}, a) &\iff \exists \rho \text{ total computable from } a \ \forall k \ \tau_k(\rho) \text{ exists,} \\ \text{DRec}(\mathcal{M}, a) &\iff \exists g \text{ total computable } \exists \rho \text{ from } a \ \forall k \ \tau_k(\rho) \leq g(k). \end{aligned}$$

A computable run means that the complete encoded state at time t can be computed from t . The deadline g is allowed to depend on the instance. There is no requirement that an algorithm find g from the instance. We normalize deadlines to be nondecreasing with $g(k) \geq k$; replacing a bound by $\max\{k, g(1), \dots, g(k)\}$ preserves its meaning.

The quantifier order in DRec is important. The same computable function must bound all marked visits of one run. The run itself need not be computable. An arbitrary recurrent run always has a deadline as a set-theoretic function, namely its visit-time function. That function need not be computable or computably dominated.

4.4.3 Effective finite branching

In the machines above, the complete finite successor list of any state is computable. This is stronger than merely knowing that a decidable relation has finite fibers. Our finite-tree searches always use the complete successor list. The constructions below preserve this property and preserve the bound of two successors.

5 A shared deadline restores compactness

Theorem 5.1 (Coherent deadline compactification). *For every system, input e , and deadline d , computable or not,*

$$\mathcal{K}_{e,d} \neq \emptyset \iff \forall k \geq 1 \text{ Fin}(e, d \upharpoonright k). \quad (14)$$

When d is computable, $\mathcal{K}_{e,d}$ is an effectively closed subset of Cantor space, given uniformly by a computable binary tree.

Proof. For each j , the condition $N_{e,f}(d(j)) \geq j$ depends only on the first $d(j)$ bits of f . Its solution set C_j is therefore clopen. Feasibility of the first k deadlines says that $C_1 \cap \dots \cap C_k$ is nonempty. These intersections are nested compact sets. Compactness of Cantor space gives a point in their intersection, which is exactly $\mathcal{K}_{e,d}$. The reverse implication follows by truncation.

For effectiveness, put a finite string σ into $T_{e,d}$ precisely when it satisfies every deadline $d(j) \leq |\sigma|$ that is already due. Because d is increasing, compute its values until the first one exceeds $|\sigma|$, then check the finite list of requirements. This is a total decision procedure, and the tree is prefix closed. Its infinite paths are exactly $\mathcal{K}_{e,d}$. The assertion also follows directly from König's lemma applied to this finitely branching tree. $\square$

Second route (source 03). [write] Source 03 states [theorem 5.1](#) for effectively finitely branching systems with an input-dependent deadline prescription (its Section “Prescribed deadlines restore effective compactness”). It fixes a total computable function $h(n, k)$, $k \geq 1$, strictly increasing in k and satisfying $h(n, k) \geq k$, and, for a fixed input n , defines

$$\text{Bound}_h(n) : \exists \text{ infinite path } s \text{ from } c(n) \forall k \geq 1 V_s(h(n, k)) \geq k.$$

This asks for a path meeting every prescribed deadline, not merely a path that eventually reaches each finite visit count. Its theorem (“Deadline compactness”) states that in an effectively finitely branching system $\text{Bound}_h(n)$ is equivalent to

$$\forall k \geq 1 \exists \text{ a path of length } h(n, k) \text{ meeting every deadline } h(n, j), 1 \leq j \leq k. \quad (15)$$

For each n, k the inner finite-path condition is decidable. In particular, Bound_h is Π_1^0 .

Second proof of theorem 5.1 (source 03). An infinite witnessing path supplies each finite prefix in (15). Conversely, form the tree of finite paths that respect every deadline that has already elapsed. More precisely, a prefix of length T belongs to this tree if

$$\forall j \text{ with } h(n, j) \leq T : V_s(h(n, j)) \geq j.$$

This tree is prefix closed. It is finitely branching because it is a subtree of the original finite-path tree. A path of length $h(n, k)$ meeting the first k deadlines belongs to it: strict increase ensures that these are exactly the deadlines that have elapsed. By (15) and the unboundedness of $h(n, k)$, this tree has arbitrarily long paths. König's lemma gives an infinite branch. Each fixed deadline eventually lies inside a prefix, so the branch meets all deadlines.

Compute $h(n, 1), \dots, h(n, k)$, enumerate the complete finite successor tree through depth $h(n, k)$, and check the visits at each required time. This is a terminating decision procedure for the inner predicate, regardless of how large the tree is. Universal quantification over k gives a Π_1^0 definition. $\square$

Remark 5.2 (All earlier deadlines must be checked). Replacing the inner condition by “some prefix has k visits by time $h(n, k)$ ” can lose constraints on the first visits. The proof uses the closed tree of prefixes satisfying *every elapsed deadline*. It does not permit an unrelated choice of early deadlines for each k .

Proposition 5.3 (Sharpness of the deadline upper bound). *The Π_1^0 upper bound is sharp for uniformly coded effective systems with a fixed computable deadline prescription. It is also attained by a suitable input-dependent total computable deadline function on the fixed interpreter U .*

Proof. For the first assertion, take a deterministic effective simulation of a machine, mark every live state, and let halting have no continuation. With $h(k) = k$, meeting every deadline is exactly nonhalting. The next-state algorithm can compute a simulated tape step as one transition, so this already proves the upper bound cannot generally be improved.

For the fixed counter interpreter, use a program that simulates one source step and then emits, repeating until the source machine halts. Put these wrapper programs in a decidable tagged input subfamily. For such an input n and integer k , simulate the actual counter run until it either halts or completes its k th emit. This always terminates: at most k source steps are required, and each implemented source step has finite duration. Measure the elapsed counter time, and monotonize the resulting function in k . Call it $h_0(n, k)$; outside the tagged subfamily, set $h_0(n, k) = k$. It is total computable. On a nonhalting wrapped input, the actual first k emits occur within the bound. On a halting input, the actual program has only finitely many emits and therefore fails some deadline. Thus Bound_{h_0} is Π_1^0 -hard on this coded subfamily. Theorem 5.1 gives membership. $\square$

No fixed-deadline computable selection of a path has been proved. Compactness establishes the existence of a compatible infinite branch; it need not supply a computable branch. Section 8.6 makes this failure explicit.

Third route (source 11). [write] Source 11 proves the same theorem (its “Deadline compactness”) for effectively finitely branching machines and one fixed total computable deadline g normalized only to be nondecreasing with $g(k) \geq k$. It lets $W_g(K)$ mean that there is a path of length $g(K)$ from a whose first j marked visits occur by $g(j)$, simultaneously for every $1 \leq j \leq K$, and proves

$$\exists \rho \forall k \tau_k(\rho) \leq g(k) \iff \forall K W_g(K). \quad (16)$$

For fixed total computable g , the right-hand predicate is Π_1^0 in the machine and its initial state.

Third proof of theorem 5.1 (source 11). An infinite witness gives all finite witnesses by restriction. Conversely, consider the tree of finite paths that have met every deadline already due within that path. It is prefix-closed. Fix any length n and take a successful witness for $W_g(n+1)$. Its length is at least $n+1$, since $g(k) \geq k$. Restrict it to length n . Any deadline $g(j) \leq n$ has $j \leq n$, so that constraint was already enforced in the witness and remains satisfied after restriction. Thus the restricted path is in the compatible tree. This works for every n , including when g has repeated values, so the tree has nodes at every depth.

The tree is finitely branching, so König’s lemma yields an infinite branch. At time $g(k)$, its prefix meets the k th deadline, proving the forward condition. Each $W_g(K)$ is decidable by finite exploration after computing $g(1), \dots, g(K)$. Searching for a failed $W_g(K)$ semidecides the complement, giving the Π_1^0 upper bound. $\square$

The qualification “fixed total computable” is not cosmetic. Given an arbitrary program index for g , totality itself is a Π_2^0 requirement. One cannot silently treat a partial proposed clock as a decidable deadline.

Example 5.4 (The incoherent-prefix trap). Consider a machine that first chooses a natural number n in unary: n continue steps followed by one stop step. It then emits exactly n accepting pulses, one per step, and enters a rejecting sink. The path that continues forever emits no pulses. Every run has only finitely many accepting visits.

Nevertheless, for each $k \geq 1$, choosing $n = k$ gives k visits by time $2k + 1$. Hence

$$\forall k \exists \sigma \in \{0, 1\}^{2k+1} : N_{e,\sigma}(2k+1) \geq k \quad (17)$$

is true, while $B(e)$ is false. With $d(j) = 2j + 1$, meeting the first deadline forces $n = 1$: the first pulse is at $n + 2$, and $n = 0$ has no pulse. That choice cannot meet the second deadline. In particular, $\text{Fin}(e, (3, 5))$ is false.

This example locates the exact error in an attractive but invalid proof: one cannot interchange the finite witnesses for different requirements without checking that they belong to a nested family. Büchi acceptance is generally an effective G_δ condition,

$$\mathcal{B}_e = \bigcap_{k \geq 1} \bigcup_{t \geq 1} \{f : N_{e,f}(t) \geq k\},$$

not a closed condition. A chosen clock replaces each open union by one finite-prefix requirement.

Proposition 5.5 (Closed subsets yield computable clocks). *If P is a nonempty effectively closed subset of $\mathcal{B}_e$, then there is a computable deadline d with $P \subseteq \mathcal{K}_{e,d}$.*

Proof. Write $P = [T]$ for a computable binary tree. For a fixed k , search for a level n such that every node $\sigma \in T \cap \{0, 1\}^n$ has $N_{e,\sigma}(n) \geq k$. Each level test is decidable. If the search never succeeded, the nodes of T with fewer than k accepting visits would form an infinite finitely branching subtree. König’s lemma would provide a path in P with fewer than k visits, contradicting $P \subseteq \mathcal{B}_e$. Thus the search terminates. Starting the search above $d(k - 1)$ makes the result strictly increasing, with $d(0) = 0$. The finite program implementing this search computes a deadline valid for every path in P . $\square$

The algorithm in this proof uses the promise $\emptyset \neq P \subseteq \mathcal{B}_e$. It is not a decision procedure for that promise. This distinction matters when a proposed compiler tries to synthesize a clock from an unverified invariant.

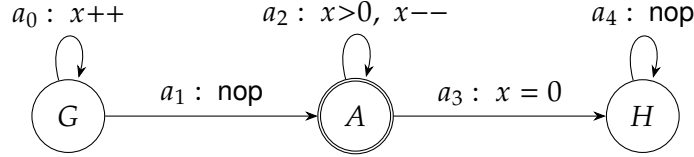
5.1 The same trap in sources 03 and 11

[write] Sources 03 and 11 refute the same tempting inference with machines of their own: source 03 with a nine-coordinate quadratic graph, source 11 with a one-counter “budget” machine whose compiled polynomial has dimension eight. The three examples (with example 5.4) share one mechanism: an unmarked initial guessing loop that can be prolonged forever. The polynomials of both examples use the compilers of Part III (equation (35) and equation (43)); source 11’s explicit residual form is in section C, and source 11’s illustration of the deadline $g(k) = 4k$ on its machine follows corollary 16.2.

5.1.1 A nine-coordinate failure of naive compactness [\[source 03\]](#)

The following completely explicit example is independent of universality or hierarchy theory. It demonstrates the missing compatibility in a common finite-witness argument.

Take three controls G, A, H , one counter x , and five edges:



Only A is marked. Initially the control is G , the counter is zero, and the old tag is zero. The state dimension is $D = 3 + 1 + 5 = 9$, and $e = (e_0, \dots, e_4)$ denotes the next tag.

$$\begin{aligned}
 E = & (e_0 + e_1 + e_2 + e_3 + e_4 - 1)^2 \\
 & + (p_G - e_0 - e_1)^2 + (p_A - e_2 - e_3)^2 + (p_H - e_4)^2 \\
 & + (p'_G - e_0)^2 + (p'_A - e_1 - e_2)^2 + (p'_H - e_3 - e_4)^2 \\
 & + (x' - x - e_0 + e_2)^2 + e_3 x.
 \end{aligned} \tag{18}$$

There are eighteen source-and-target coordinate slots, five of which are unused old tags. Its expanded form has forty-four nonzero monomials and coefficient height four, as independently computed by the delivered compiler.

Proposition 5.6 (Unbounded finite visits without recurrence). *For every $k \geq 1$, this system has a finite path with at least k marked visits, and every finite path can be continued to an infinite path. Nevertheless, no infinite path is recurrent.*

Proof. Choose n increment loops at G , then move to A . Arrival at A gives one visit. Exactly n decrement loops are then possible before the counter becomes zero; each gives another visit. The zero edge moves to H , which loops forever. This path has exactly $n + 1$ marked visits. Taking $n = k - 1$ gives the required finite prefix and infinite extension.

Every infinite path either stays at G forever, in which case it has no marked visits, or leaves G after a finite number n of loops. In the second case it has precisely the finite behavior just described and then remains at H . No path has infinitely many marked visits. There are no dead ends, so every finite path has an infinite continuation. $\square$

Thus the implication

$$[\forall k \exists \text{ finite path with } k \text{ marked visits}] \implies [\exists \text{ path with infinitely many marked visits}] \tag{19}$$

is false even for a globally binary quadratic graph. The witnesses on the left choose different lengths of the initial guessing segment. Their limit may stay in that segment forever. The topology of a finite-branching path space is compact, but a requirement of infinitely many visits is not a closed finite-prefix condition. Deadlines repair precisely this defect.

5.1.2 Why arbitrarily many finite visits do not imply recurrence [\[source 11\]](#)

One tempting assertion is

$$\forall k \exists \text{ finite path with at least } k \text{ marked visits} \implies \text{Rec.} \quad (20)$$

It is false even for a one-counter binary-branching machine.

Example 5.7 (Choose a budget, then spend it). Use labels G, I, D, A, H , one counter x , and mark only A . The edges are

$$\begin{aligned} G &\xrightarrow{\text{nop}} I, & G &\xrightarrow{\text{nop}} D, & I &\xrightarrow{\text{inc}} G, \\ D &\xrightarrow{\text{dec}} A, & D &\xrightarrow{\text{zero}} H, & A &\xrightarrow{\text{nop}} D. \end{aligned}$$

Start at $(G, 0)$, and give H no outgoing edges. Repeating G, I, G exactly n times and then entering D produces exactly n marked visits before halting. The only infinite behavior is to keep increasing the budget in the G, I loop forever. That behavior never reaches A .

Consequently the left side of (20) holds and its right side fails. This is not a violation of König's lemma. The full tree of finite runs has an infinite branch, namely the unmarked choosing branch. Having many marks somewhere in the tree is not a closed, prefix-compatible constraint forcing marks onto that branch.

The compiled example has state dimension eight and 88 expanded monomials in each of P_2 and P_4 . Both have height four. The package exports both polynomials exactly, not just their numerical evaluations.

6 The sharp progress-modulus hierarchy

6.1 Statement and conventions

Completeness below means computable many-one completeness for sets of natural input codes. For a fixed system, the unknown is e ; neither the transition function nor the accepting region is changed with e . We use the standard arithmetical normal forms $\exists m \forall k R(e, m, k)$ and $\exists m \forall k \exists s R(e, m, k, s)$, with total decidable R .

Theorem 6.1 (Progress hierarchy). *Every computable binary-controlled system has the upper bounds in the middle column of the following table. There is a single fixed universal system for which every bound is complete, and which can be implemented as a finite binary-controlled two-stack machine.*

Finite-input property	Complexity	Defining pattern
Some finite accepting visit, Reach	Σ_1^0	$\exists n$
Some run meets a fixed computable d , D_d	Π_1^0	$\forall k$
Some run has bounded gaps, BG	Σ_2^0	$\exists b \forall n$
Some run has a computable deadline, CD	Σ_3^0	$\exists m \forall k \exists s$
Some run is recurrent, B	Σ_1^1	$\exists f \forall k \exists t$

For the fixed-deadline row, completeness holds for every fixed computable strictly increasing d , including $d(k) = k$.

6.2 Finite reachability and a fixed clock

For Reach , enumerate finite schedules and simulate them. This semidecides an accepting visit. To reduce halting, simulate an ordinary program without accepting and enter an accepting state exactly when it halts. The finite input encodes the simulated program, so one interpreter handles the entire reduction.

For D_d , theorem 5.1 gives the Π_1^0 definition $\forall k \text{Fin}(e, d \upharpoonright k)$. For hardness, simulate a program while marking every nonhalting interpreter microstep accepting. If it halts, enter a rejecting sink. Nonhalting produces one accepting visit at every time and therefore satisfies every $d(k) \geq k$. Halting produces only finitely many accepting visits and satisfies no deadline. This reduces the complement of the halting problem to D_d .

6.3 An unknown bounded-gap guarantee

Fix $b \geq 1$. A finite schedule of length n is b -admissible when every window $(t, t + b]$ contained in that finite prefix contains an accepting visit. Checking that a b -admissible schedule of length n exists is decidable. These conditions define a prefix-closed binary tree; König's lemma gives

$$\exists f \forall t \geq 0 : N_f(t + b) - N_f(t) \geq 1 \iff \forall n \exists \text{ a } b\text{-admissible prefix of length } n.$$

The finite existential search is part of the decidable matrix. Quantifying b gives a Σ_2^0 upper bound.

For hardness, let $S(e) \leftrightarrow \exists m \forall k R(e, m, k)$, with R decidable. The system first guesses m in unary without accepting. After the guess ends, it deterministically checks $R(e, m, 0), R(e, m, 1), \dots$. All computation steps during these checks are accepting. On the first false test it enters the rejecting sink.

If some m works, the finite guessing prefix is followed by an accepting visit at every step, so some gap bound holds. If no m works, every completed guess eventually reaches the sink. A schedule that never finishes guessing has no accepting visits. Thus BG holds exactly when $S(e)$ holds. The acceptance flag during verification is essential: the computation times of the individual decidable tests need not have a uniform bound.

6.4 An unknown total computable deadline

Fix an effective enumeration (φ_m) of partial computable functions. There is a decidable predicate $V(e, m, k, s)$ saying:

By computation stage s , all values $\varphi_m(1), \dots, \varphi_m(k)$ have halted, are positive and strictly increasing, and their resulting finite deadline vector is feasible.

One may either include the finite feasibility computation in the stage bound or run its total decision procedure after the bounded simulations; both define decidable predicates with the same existential closure in s . The condition for $k = 0$ can be declared true. Then

$$\text{CD}(e) \iff \exists m \forall k \exists s V(e, m, k, s). \quad (21)$$

The same index m must serve every k . If the right side holds, it defines a single total computable deadline, and coherent compactification supplies a run meeting it. This proves the Σ_3^0 upper bound.

For hardness, start with $S(e) \leftrightarrow \exists m \forall k \exists s R(e, m, k, s)$. Guess m in unary. For $k = 0, 1, 2, \dots$, search $s = 0, 1, 2, \dots$ until $R(e, m, k, s)$ is true; then emit one accepting pulse and advance to $k + 1$. No search step is accepting. If m satisfies the formula, the run obtained by using the least successful s at each stage is computable and recurrent. Its k th accepting time is therefore a total computable function; strictify it if necessary to obtain a deadline. If no m satisfies the formula, every completed guess eventually stalls on a stage with no witness. Infinite unary guessing produces no pulses. There is then no recurrent run at all, hence no computable-deadline run. This proves Σ_3^0 hardness.

6.5 Unrestricted recurrence and recursive trees

The definition $\exists f \forall k \exists t N_f(t) \geq k$ is analytic, so B belongs to Σ_1^1 . For completeness, we spell out the standard recursive-tree reduction behind this endpoint; the recurrent-machine hardness itself is classical [7, 10].

Lemma 6.2 (Analytic tree normal form). *For every Σ_1^1 set $S \subseteq \mathbb{N}$, there is a uniformly primitive-recursive family of prefix-closed trees $T_e \subseteq \mathbb{N}^{<\mathbb{N}}$ such that*

$$e \in S \iff \exists g \in \mathbb{N}^{\mathbb{N}} \forall n : g \upharpoonright n \in T_e.$$

Normal-form argument. Write an analytic definition as an existential function quantifier followed by an arithmetic formula. Put the finite number-quantifier part in prenex form and absorb all existential number choices into additional Skolem functions, dependent on the preceding universal number variables. Encode these functions and the original function into one g . This gives $\exists g \forall n R(e, g, n)$, where R is a total oracle-computable Boolean test.

For a finite sequence σ of length l , run the first l tests for l steps, answering oracle questions within σ and suspending a test that asks outside it. Reject σ exactly when a fully evaluated test is false. This is a primitive-recursive bounded computation and is prefix closed. A g satisfying every test has all its prefixes in the tree. Conversely, along an infinite tree path every particular test obtains all its finitely many oracle answers and finishes at a sufficiently long stage, so it cannot be false. This proves the equivalence. Standard coding of the finitely many function components is effective. $\square$

Given e , the recurrent machine maintains a finite sequence σ . It guesses the next natural n by unary binary choices, appends n , checks $\sigma \frown n \in T_e$, and emits one pulse exactly when the check succeeds. A failed check leads to the rejecting sink. The tree check is always a terminating finite computation. An infinite branch gives an accepting schedule. Conversely, an accepting schedule must complete infinitely many guesses and successful checks, so its guessed values form an infinite branch. Remaining forever inside one unfinished unary guess cannot create recurrence. This is the reduction.

6.6 Why one fixed two-stack machine suffices

The reductions above describe a finite menu of interpreters with input parameters: a halting simulator, an always-accepting safety simulator, a unary-witness verifier, a pulse-on-success searcher, and a recursive-tree checker. A single machine can include these modes in its finite control and read a mode-tagged program from its initial finite storage.

A two-stack machine simulates an ordinary tape by storing the finite tape on the two sides of the head, keeping the scanned symbol in finite control. A fixed binary block code represents the finitely many tape symbols using stack symbols 1 and 3. Reading or modifying a symbol requires finitely many top tests, pops, and pushes; the fixed universal interpreter has only finitely many such instruction types. External binary choices are read only at the intended guessing steps and ignored elsewhere. Rejecting halts are completed by a sink. This gives one finite rule table with effective finite input encoding. No minimum rule count or explicit small universal table is claimed.

The accepting flag is part of the finite control. In the Π_1^0 and Σ_2^0 safety modes it is set during every verification microstep. In the search and tree modes it is set for exactly one microstep per successful stage. These choices eliminate hidden assumptions about interpreter overhead. The natural-number arguments of all normal forms are parameters to the fixed interpreter, not infinitely many new control states.

The strictness of the arithmetical hierarchy and the existence of nonarithmetical analytic sets imply that the inclusions in equation (12) are strict for such a universal system. We next give concrete separations that explain their computational meaning.

6.7 One hierarchy table for the three sources

[write] The table lists every temporal predicate classified in the three sources, with the theorem that proves each classification. Completeness is for a fixed universal system unless marked “across machines”, which means over effectively supplied finite machines and initial states (source 11’s Σ_3^0 rows; source 11 notes that a fixed universal interpreter can absorb the program into the initial encoding where appropriate).

Property	Class	Proved in
some finite accepting visit	Σ_1^0	10: Reach, theorem 6.1; 03: Hit for E_U , theorem 6.3; 11: EFF, theorem 6.7
some infinite run	Π_1^0	03: Inf for E_U , theorem 6.3; 11: EGF, theorem 6.7
some run marked at every step	Π_1^0	10: D _{id} , theorem 6.1 (times $t \geq 1$); 11: EGF, theorem 6.7
some run meets a fixed computable deadline	Π_1^0	10: D _d , complete for <i>every</i> fixed d , theorem 6.1; 03: Bound _h , sharp for suitable families, proposition 5.3; 11: upper bound, theorem 5.1
some run is eventually always marked	Σ_2^0	11: EFGF, theorem 6.7
some run has bounded gaps	Σ_2^0	10: BG, theorem 6.1
every infinite run is marked infinitely often	Π_2^0	11: AGFF, theorem 6.7
deterministic recurrence	Π_2^0	10: theorem 9.1; 03: second proof there; 11: remark after theorem 6.7
some run meets some computable deadline	Σ_3^0	10: CD, theorem 6.1; 03: CB for E_U , theorem 6.5; 11: DRec across machines, theorem 6.8
some computable run is recurrent	Σ_3^0	10: Comp, section 8.4; 03: CP for E_U , theorem 6.6; 11: CRec across machines, theorem 6.8
some run is recurrent	Σ_1^1	10: B, theorem 6.1; 03: Rec for E_U , theorem 6.4; 11: Rec for a fixed P_2 , theorem 17.2, and EGFF, theorem 6.7
one always-enabled weak-fairness obligation	Σ_1^1	03: theorem 21.1

Property	Class	Proved in
every infinite run is eventually always marked	Π_1^1	11: AFGF, theorem 6.7
strict chain $\text{Comp} \subsetneq \text{CD} \subsetneq \text{B}$		10: proposition 8.1 and corollary 8.4; 03: corollary 8.9; 11: theorems 8.10 and 8.11

[write] Source 03’s summary “ Π_1^0 upper bound; sharp for suitable uniformly specified families” and source 10’s “ Π_1^0 -complete for every fixed computable strictly increasing d ” are compatible: source 10’s reduction marks every live interpreter microstep accepting, so every deadline $d(k) \geq k$ is met exactly on nonhalting inputs, while source 03’s second construction (proposition 5.3) marks only emits and therefore needs an input-dependent deadline. Source 11’s temporal rows EFG, AGF and AFG are new relative to the other two sources; its EF, EG and EGF rows repeat classifications printed above for other fixed systems. Report B of this category (probabilistic-quantum-and-continuous-computation) has the probabilistic counterpart of the deterministic Π_2^0 row: almost-sure termination is Π_2^0 (its `pqc:eq:te` and theorem `pqc:thm:unconditionaltable`), by the same totality-type argument. Report A (canonical-diophantine-certificates) supplies the lowest levels for other substrates: perpetual safety of heap programs is not r.e. (Π_1^0 ; its batch-62 dynamic-heaps Part) and eventual periodicity of priority programs is c.e.-complete (Σ_1^0 ; its batch-62 priority-pumping Part).

6.8 Second routes through source 03’s quadratic graph

[write] Source 03 proves the rows Hit, Inf, Rec, CB and CP for one fixed degree-two counter graph E_U (corollary 13.2, Part III). Its tree normal form is lemma 6.2, whose second proof is source 03’s. Its statements and proofs follow; they reuse only the event-reflecting interpreter of lemma 13.1.

Theorem 6.3 (Finite hit and plain infinite continuation). *For the fixed universal system, Hit is Σ_1^0 -complete and has an ordinary Diophantine representation. The predicate Inf that some infinite path exists is Π_1^0 -complete.*

Proof. Finite hit is computably enumerable: enumerate the finite successor tree level by level until a marked vertex occurs. To reduce halting, interpret a program that simulates the given machine and emits once precisely when it halts. MRDP supplies the ordinary Diophantine representation.

For infinite continuation, existence of a path of any particular length T is decidable by complete finite branching. By König’s lemma, an infinite path exists if and only if paths of every length exist. This is Π_1^0 . For hardness, interpret a deterministic simulation that halts exactly when the given machine halts. Positive finite simulation of each source step ensures an infinite counter run exactly in the nonhalting case. $\square$

6.9 Unrestricted recurrence is analytical-complete [source 03]

A tree $T \subseteq \mathbb{N}^{<\mathbb{N}}$ is prefix closed. It is recursive when membership of a finite sequence is decidable. An infinite branch is a function $f \in \mathbb{N}^{\mathbb{N}}$ with $f \upharpoonright k \in T$ for every k . Notice that such a tree is not required to be finitely branching.

[write] Source 03's recursive-tree normal form (its Lemma "Recursive-tree normal form") is lemma 6.2, stated for primitive recursive trees $T_n \subseteq \mathbb{N}^{<\mathbb{N}}$, with the added remark that infinite-branch existence for such trees is consequently Σ_1^1 -complete.

Second proof of lemma 6.2 (source 03). The upper bound existentially quantifies over a branch and arithmetically checks every finite prefix. For completeness, start from $\exists f \theta(n, f)$, where θ is arithmetic. Skolemize the existential number quantifiers inside θ by functions of the preceding universal number variables, and combine the finitely many function variables into one. This gives an equivalent form

$$\exists u \in \mathbb{N}^{\mathbb{N}} \forall z R(n, u, z),$$

where R can be checked by an oracle computation that uses finitely many values of u on each terminating computation. One can choose the matrix checker total; alternatively, accepting computation transcripts provide the needed finite evidence directly.

Let a branch code alternate entries $u(i)$ and finite accepting computation transcripts C_z for the checker of $R(n, u, z)$. For a finite prefix, verify that each transcript already present is a syntactically valid accepting computation, treating the recorded oracle answers as part of the transcript. Whenever a queried value $u(i)$ is also present in the prefix, require that the recorded answer equals it. Any coding inconsistencies cause rejection. Finite computation checking and decoding finite natural-number codes are primitive recursive. Only finitely many present entries and present transcripts are checked, so membership in the resulting tree is primitive recursive, uniformly in n .

The conditions are prefix closed: extending a prefix adds checks but never removes an existing one. A witness u together with its actual accepting transcripts gives an infinite branch. Conversely, on an infinite branch, every transcript is eventually present and every one of its finitely many oracle answers is eventually compared with the corresponding $u(i)$. Thus all transcripts are genuine accepting computations relative to the single function u , and $R(n, u, z)$ holds for every z . This proves the normal form. It is the standard tree mechanism underlying analytical recurrence reductions; compare [15]. $\square$

Theorem 6.4 (Fixed quadratic recurrent universality). *For the single polynomial E_U and marker $p_{q_A} = 1$,*

$$\text{Rec}(n) \iff \exists s \in (\mathbb{N}^{Du})^{\mathbb{N}} [s_0 = c(n) \wedge \forall t E_U(s_t, s_{t+1}) = 0 \wedge \forall k \exists t \geq k A(s_t)]$$

is Σ_1^1 -complete.

Proof. The displayed formula gives membership in Σ_1^1 , since a sequence of finite-dimensional natural states is a single function after pairing. For hardness, use the family T_n from Lemma 6.2. Interpret the following program, where each choice is binary:

```
sigma := empty sequence
repeat:
  a := 0
  while choose(CONTINUE, STOP) == CONTINUE:
    a := a + 1
  sigma := sigma concatenated with [a]
  if sigma is not in T_n: halt
EMIT
```

Membership tests terminate because the tree is primitive recursive. If f is an infinite branch of T_n , choose $f(k)$ continuations at stage k before stopping. Every test succeeds, so the resulting run emits infinitely often.

Conversely, infinitely many emits require infinitely many completed stages. A run that chooses CONTINUE forever in a single stage has no further emits; it cannot be recurrent. Read the successive completed guesses from a recurrent run. They form $f \in \mathbb{N}^{\mathbb{N}}$, and every finite prefix passes its tree test. Hence f is an infinite branch. The interpreter preserves and reflects emits, and every completed source operation takes positive finite time. Therefore exactly the same equivalence holds after compilation into the fixed quadratic graph. The program code depends computably on n . $\square$

The apparent paradox with finite branching is resolved by the infinite unary guess. A countably branching tree is represented by a binary choice process that may postpone completion forever. Recurrence discards those postponed branches. König's lemma guarantees an infinite binary run but does not guarantee that the run completes infinitely many guesses.

6.10 Existential computable bounds and computable paths [\[source 03\]](#)

The next theorem concerns one fixed system E_U , not an unbounded collection of different-dimensional polynomials. Input codes select source programs inside the fixed interpreter.

Theorem 6.5 (Exact complexity of computable deadline existence). *For the fixed universal quadratic system, CB is Σ_3^0 -complete.*

Proof. Enumerate partial computable functions φ_e on positive integers. For a candidate index e and k , require that $\varphi_e(1), \dots, \varphi_e(k)$ all halt, produce an increasing sequence $h(1), \dots, h(k)$ with $h(j) \geq j$, and that a finite path of length $h(k)$ meets all these k deadlines. The last condition is decidable after the finite values of h have been computed, by the complete successor-list algorithm.

Let $R(n, e, k, s)$ say that all these calculations and the finite-path search finish successfully within a total computation budget s . This is a computable predicate; it can also be implemented by bounded simulation. Then

$$\text{CB}(n) \iff \exists e \forall k \geq 1 \exists s R(n, e, k, s). \quad (22)$$

For the right-to-left direction, the universal condition forces φ_e to be total and yields every finite deadline condition. Theorem 5.1 gives a single infinite path meeting all of them. The reverse direction follows by taking an index of the computable bound. This proves the Σ_3^0 upper bound.

For hardness, let S be any Σ_3^0 set and fix a decidable relation R_0 with

$$n \in S \iff \exists a \forall b \exists c R_0(n, a, b, c).$$

Interpret the following program:

```
a := a finite natural chosen by a unary CONTINUE/STOP loop
for b = 0, 1, 2, ...:
  search c = 0, 1, 2, ... until R_0(n,a,b,c) holds
  EMIT
```

An infinite unfinished initial guess produces no emits. After a completed guess a , the rest of the run is deterministic. If a witnesses $n \in S$, every search terminates. The resulting recurrent

path is computable: the finite choice of a can be built into its program, and each later step is computed by simulation. In particular, its k th emit time is a total computable function of k . Thus the corresponding input belongs to CB. If no a is a witness, each completed choice eventually reaches a search that never succeeds, and hence has only finitely many emits. There is no recurrent path at all. The input code is computable from n , proving Σ_3^0 -hardness. $\square$

Theorem 6.6 (Exact complexity of computable recurrent paths). *For the same fixed system, CP is Σ_3^0 -complete, and*

$$\text{CP} \subseteq \text{CB} \subseteq \text{Rec}.$$

Proof. For the upper bound, existentially choose an index e of a proposed state sequence. For every k , require a finite computation budget s within which e produces a prefix $s_0, \dots, s_T$ with $T \geq k$, the prescribed initial state, valid adjacent transitions, and at least k marked visits. This condition is decidable by bounded simulation and finite checking. The formula has shape $\exists e \forall k \exists s$. Because every prefix uses the same deterministic program e , the verified outputs are compatible. Since $T \geq k$ for unbounded k , that program is defined on every time index. It therefore describes one computable infinite recurrent path. Conversely, every such path supplies the required finite computations.

The reduction used in Theorem 6.5 already has a computable recurrent path whenever it has any recurrent path, proving the same hardness for CP. Finally, from a computable recurrent path and a decidable marker, search for successive marked times. Each search terminates, so $k \mapsto \tau_s(k)$ is total computable and is a valid deadline function. This proves $\text{CP} \subseteq \text{CB}$; the other inclusion is immediate. $\square$

Equality of complexity classes does not imply equality of sets. In this case CP and CB are both complete for the third arithmetic level, yet there are explicit inputs belonging to one and not the other.

6.11 Third routes through source 11's quadratic law

[write] Source 11 classifies the temporal properties of its incoming-edge law (Part III) and proves the two Σ_3^0 rows over effectively supplied machines. Its fixed-law Σ_1^1 theorem is theorem 17.2 in Part III. Its sections follow.

6.12 The finite-branching temporal hierarchy [source 11]

The local equations remain unchanged while temporal specifications cross several levels of definability. The following table gives sharp completeness bounds over effectively supplied finite binary counter machines and initial states. A fixed universal interpreter can also absorb the program into the initial encoding where appropriate.

Property	Meaning	Completeness
EFF	Some finite path reaches a marked state	Σ_1^0
EGF	Some infinite run stays marked throughout	Π_1^0
EFGF	Some infinite run is eventually always marked	Σ_2^0
AGFF	Every infinite run is marked infinitely often	Π_2^0
EGFF	Some infinite run is marked infinitely often	Σ_1^1
AFGF	Every infinite run is eventually always marked	Π_1^1

Universal claims about infinite runs are vacuously true when there are no infinite runs. This convention is part of the table.

Theorem 6.7 (Temporal classifications). *The completeness claims in the table hold under total computable many-one reductions. They continue to hold after the witness-free polynomial compilation.*

Proof. Finite reachability is computably enumerable by exploring finite paths; halting gives hardness.

For EGF, keep only marked states in the run tree. There is an infinite run exactly when this effectively finitely branching tree has a node at every depth. Each finite-level test is decidable, so the property is Π_1^0 . For hardness, simulate a deterministic computation with every nonhalting state marked and deadlock on halting.

For EFGF, existentially choose a finite prefix and then require an infinite all-marked continuation. This is a Σ_2^0 condition. For hardness, put a Σ_2^0 predicate in the form $\exists n \forall k R(x, n, k)$ with recursive R . Guess n in an unmarked unary loop. After committing to n , enter marked states and check $R(x, n, 0), R(x, n, 1), \dots$ sequentially, halting upon failure. Every individual check terminates. The resulting run is eventually always marked exactly when some chosen n passes all checks. An infinite uncommitted guessing run is unmarked, so it cannot produce a false positive.

The identity $\text{AGFF} = \neg \text{EFG}(Q \setminus F)$ gives the Π_2^0 line, including hardness. The Σ_1^1 recurrence line follows from theorem 17.2. Finally $\text{AFGF} = \neg \text{EGF}(Q \setminus F)$ gives the Π_1^1 line. Compilation preserves all labelled paths and marker visits, so it preserves these properties. $\square$

For a deterministic machine made total by an unmarked sink, recurrence is Π_2^0 rather than Σ_1^1 . Hardness follows by successively searching for witnesses in a predicate $\forall k \exists s R(x, k, s)$ and marking completion of each successful search. If one search never succeeds, only finitely many marks occur. Every recurrent deterministic computable run also has a computable deadline, obtained by simulating until its k th visit. Nondeterminism is what permits the witness distinctions below.

6.13 The exact cost of supplying a computable deadline [\[source 11\]](#)

Theorem 6.8 (Two Σ_3^0 -complete witness notions). *The index sets DRec and CRec, for effectively supplied finite binary-branching counter machines and encoded initial states, are both Σ_3^0 -complete.*

Proof. *Upper bound for DRec.* Existentially choose a program index e for a proposed deadline. A finite verification at stage K can wait for the program to return values on $1, \dots, K$, check normalization, and exhaustively test $W_g(K)$. The assertion that for every K this finite verification successfully finishes has the form

$$\exists e \forall K \exists s \text{ Verified}(\mathcal{M}, a, e, K, s),$$

where the matrix is a decidable bounded-computation predicate. These conditions also enforce totality of the proposed deadline. By theorem 5.1, they are equivalent to DRec.

Upper bound for CRec. Existentially choose an index e for the run. For each K , require a finite computation witnessing a legal initial run segment of length at least K with at least K marked positions, all of whose states are returned by program e . As K varies, this forces the program to define a total legal run and forces infinitely many marks. It is again an $\exists e \forall K \exists s$ condition.

Hardness for both. Let S be any Σ_3^0 set, and choose a recursive matrix R such that

$$x \in S \iff \exists n \forall k \exists s R(x, n, k, s).$$

Construct a program which guesses n by an unmarked unary loop, commits to it, and then for $k = 0, 1, \dots$ searches deterministically for an s satisfying $R(x, n, k, s)$. Each successful search emits exactly one marked checkpoint. If $x \in S$, a witnessing finite n gives a computable recurrent run. Its visit-time function is total computable, so it also witnesses DRec. If $x \notin S$, every committed n eventually reaches an unsuccessful search that continues without further marks, while an uncommitted run has no marks. There is no recurrent run at all. The construction is effective and converts to the counter normal form, proving hardness of both properties. $\square$

The quantifier $\exists n$ in the hardness proof is not uniformly computable from a positive input x . That does not matter: a witnessing natural number can be hard-coded into the program for that one run. Confusing a computable witness with a uniform algorithm selecting witnesses would change the problem.

Proposition 6.9 (Computable runs supply computable deadlines). $\text{CRec} \subseteq \text{DRec} \subseteq \text{Rec}$.

Proof. Given a total computable recurrent run, compute successive states until the k th mark appears. Recurrence ensures termination for each k , giving a total computable visit-time function. Normalize it if necessary. The second inclusion is immediate from the deadline condition. $\square$

Completeness at the same arithmetical level does not imply equality of these properties on individual machines. The next two constructions prove both inclusions strict without appealing merely to different complexity classes.

Part II

Scheduler degrees, explicit separations and the deterministic case

[write] Part II prints source 10's degree-theoretic characterization of computable clocks, its separations and c.e. spectra, then source 03's and source 11's separations as second and third routes, and the deterministic boundary with source 03's second proof.

7 Scheduler degrees: an exact characterization

7.1 Hyperimmune-free degrees and a basis argument

Definition 7.1. A schedule f has *hyperimmune-free degree* when every total function $g : \mathbb{N} \rightarrow \mathbb{N}$ with $g \leq_T f$ is bounded by some total computable function. Using eventual domination instead of pointwise domination gives the same notion: finitely many exceptional values can be incorporated as constants in a computable bound. A schedule is *low* when $f' \leq_T \emptyset'$.

The hyperimmune-free basis theorem says that every nonempty effectively closed subset of Cantor space contains a member of hyperimmune-free degree [5, 6]. We record its compactness mechanism because it is precisely what is needed for clocks.

Lemma 7.2 (Classical HIF basis theorem). *Every nonempty effectively closed $P \subseteq \{0, 1\}^{\mathbb{N}}$ contains an HIF member.*

Proof. Construct a nested sequence of nonempty effectively closed classes $P = P_0 \supseteq P_1 \supseteq \dots$, handling the e th oracle functional Φ_e at stage e . If for some n the class

$$P_e \cap \{f : \Phi_e^f(n) \uparrow\}$$

is nonempty, choose such an n and use this class as P_{e+1} . It is effectively closed: nonconvergence is a closed finite-computation condition. If no such n exists, then $\Phi_e^f(n)$ converges for every $f \in P_e$ and every n .

In the latter case fix a computable tree for P_e . For each input n , search for a level l at which every tree node of length l gives a convergent computation of $\Phi_e(n)$ within l steps, using only the supplied l oracle bits. Such a level exists; otherwise the nonconvergent finite computations form an infinite subtree and yield a path on which the functional diverges. The maximum of the finitely many outputs at the successful level is a computable bound $g_e(n)$. Set $P_{e+1} = P_e$.

Compactness gives $f \in \bigcap_e P_e$. If Φ_e^f is total, the divergent restriction could not have been chosen at stage e , so g_e bounds it. Therefore f has HIF degree. The stage choices need not be effective; each chosen finite tree index is a fixed constant in the corresponding program for g_e . This is an existence proof, not an algorithm selecting an HIF path from a tree index. $\square$

7.2 The main equivalence

Theorem 7.3 (Clock–closed–class–degree equivalence). *For every input e to a computable binary-controlled system, the following are equivalent:*

- (i) $\text{CD}(e)$: *some accepting schedule meets a computable deadline;*
- (ii) $\mathcal{B}_e$ *contains a nonempty effectively closed subset of Cantor space;*
- (iii) $\mathcal{B}_e$ *contains a schedule of hyperimmune-free Turing degree.*

Moreover, whenever these conditions hold, $\mathcal{B}_e$ contains a low schedule. The low schedule and the HIF schedule are not asserted to be the same.

Proof. For (i) $\Rightarrow$ (ii), use the nonempty effectively closed set $\mathcal{K}_{e,d}$ from [theorem 5.1](#). The converse is [proposition 5.5](#). For (ii) $\Rightarrow$ (iii), apply the HIF basis theorem to the closed subclass.

For (iii) $\Rightarrow$ (i), let $f \in \mathcal{B}_e$ be HIF. Given oracle f , simulate the run until the k th accepting visit. Because f is accepting, this algorithm computes the total function $k \mapsto \tau_{e,f}(k)$. There is a computable bound h for it. The function

$$d(k) = k + \max_{1 \leq j \leq k} h(j) \tag{23}$$

is computable and strictly increasing, and bounds $\tau_{e,f}(k)$. Finally, the classical low basis theorem of Jockusch and Soare applied to $\mathcal{K}_{e,d}$ supplies a low accepting schedule [\[5\]](#). $\square$

[write] Source 11 proves the weaker Δ_2^0 form of the last assertion by a direct path selection (its proposition in a subsection “What a computable deadline does give”): *every recurrent instance with a computable deadline has a recurrent run computable in the halting oracle $0'$* . It follows from [theorem 7.3](#), since a low schedule is Δ_2^0 and computes its run.

Second proof of the Δ_2^0 form (source 11). The deadline-compatible tree in theorem 5.1 is decidable and effectively finitely branching. Whether a node has arbitrarily long extensions is Π_1^0 , hence decidable by $0'$. Starting at the root, use that oracle to choose a successor with arbitrarily long extensions, taking the first one in the computable successor ordering. This constructs an infinite compatible path. The deadline ensures recurrence. $\square$

This theorem distinguishes two kinds of information. A computable clock bounds how late progress may occur; it need not tell us which compatible branch to choose. Conversely, an HIF schedule may contain noncomputable information, but cannot use that information to produce a progress-time function escaping every computable upper bound.

Corollary 7.4 (Oracle-relative form). *Let Z be any oracle, and allow the transition function, initial coding, and acceptance predicate to be computable relative to Z . There is a Z -computable deadline met by some accepting schedule if and only if there is an accepting f such that every total $g \leq_T f \oplus Z$ is dominated by a Z -computable function.*

Proof. Relativize the effective closed-tree construction, the basis argument, and the computation of $\tau_{e,f}$ in the preceding proof. $\square$

For an ordinary computable system, the set of oracle degrees that can compute some adequate deadline is upward closed. The relative theorem characterizes it without confusing a schedule with a program that prints that schedule. Determining finer properties of this deadline-degree spectrum is one of the research questions in section 33.

8 Explicit separations and exact deadline-degree spectra

8.1 A computable clock need not compute a scheduler

Fix an effective enumeration (φ_e) of partial computable natural-number functions. Define a binary tree T_{DNR} by

$$\sigma \in T_{\text{DNR}}, \quad |\sigma| = n \quad \iff \quad \forall e < n : \quad [\varphi_e(e) \text{ halts within } n \text{ steps with } b \in \{0, 1\}] \Rightarrow \sigma(e) \neq b. \quad (24)$$

Membership is decidable by bounded simulation, and the tree is prefix closed. An infinite path exists: at each coordinate choose the opposite bit if the diagonal computation eventually outputs a bit, and choose arbitrarily if it does not. But no computable path exists. A total computable binary path would equal φ_j for some j , so at coordinate j it would eventually violate its own diagonal requirement.

A verifier requests one new bit at each stage, checks the resulting prefix against equation (24), and emits a pulse after success. Every finite verification terminates. For each fixed stage k , its worst-case computation time over the finitely many possible bit prefixes is computably bounded: enumerate those prefixes and run the terminating bounded checks. Adding these bounds gives a computable deadline shared by all genuine infinite paths. A computable accepting schedule would compute a path by recording the bits requested at successive stages, which is impossible.

Proposition 8.1 (The clock/controller distinction is strict). *There is a computable binary-controlled system with $\text{CD}(e)$ true and with no computable accepting schedule. In fact, there is such a system with $\text{D}_{\text{id}}(e)$ true.*

Proof. The pulse verifier proves the first assertion. For the stronger assertion, mark every working microstep of the same verifier accepting, instead of marking only successful stage completions. A failed check enters the rejecting sink. Because every finite check terminates and exactly one bit is requested per stage, an infinite nonsink computation determines an infinite path of the tree. Therefore the schedules meeting $d(k) = k$ are exactly the schedules that never fail a check. Such schedules exist, but a computable one would compute an impossible computable DNR path. $\square$

This example also explains why the compactness and basis arguments cannot be silently turned into effective selection algorithms. It does not conflict with the existence of low and HIF paths: those paths need not be computable, and the two basis applications select potentially different paths.

8.2 A recurrent computation that requires a halting oracle

List ordinary programs $M_0, M_1, \dots$ and let

$$B(i) = \max(\{0\} \cup \{\text{halting time of } M_j \text{ on blank input} : j \leq i\}).$$

For a sequence σ of length n , require that whenever a program M_j with $j \leq i < n$ has halted within n steps, its halting time is at most $\sigma(i)$. This is a primitive-recursive, prefix-closed tree, and its infinite paths are exactly the functions g satisfying $g(i) \geq B(i)$ for all i .

Run the unary-guess tree verifier of section 6. Choosing $g = B$ gives an accepting run. Every accepting schedule computes a path g , and such a path decides halting: to test M_i , simulate it for $g(i)$ steps. Also, guessing $g(i)$ in unary costs at least $g(i)$ microsteps before the $(i + 1)$ st accepting pulse. Hence any successful deadline satisfies

$$B(i) \leq g(i) \leq \tau_f(i + 1) \leq d(i + 1).$$

A computable deadline would decide the halting problem. No such deadline exists, although an accepting run does. The next theorem gives the exact generalization to every c.e. degree.

8.3 Realizing arbitrary c.e. deadline lower bounds

Let $(A_s)_{s \in \mathbb{N}}$ be a computable increasing sequence of finite sets with union A , so A is computably enumerable. Fix a stage convention and write $t_A(x)$ for the first stage at which x enters A , when it does. Define its finite-initial-segment enumeration modulus

$$b_A(i) = \max(\{0\} \cup \{t_A(x) : x \leq i, x \in A\}). \quad (25)$$

The function b_A is computable from A : ask which of $0, \dots, i$ enter, then enumerate until all those positive answers have appeared. Conversely, any pointwise upper bound h for b_A computes A , since

$$x \in A \iff x \in A_{h(x)}. \quad (26)$$

Definition 8.2. For a system input e , its *deadline oracle spectrum* is

$$\text{Spec}_{\text{clk}}(e) = \{\deg_T(Z) : \text{some } Z\text{-computable deadline is met by a run on } e\}.$$

This records oracle capabilities, not the degrees of every individual schedule. In particular, the accepting schedule need not be computable from the oracle that computes the deadline.

Theorem 8.3 (Exact realization of every c.e. degree). *Uniformly from an index for a c.e. set A , one can construct an input e_A to a fixed binary-controlled two-stack system such that*

$$\text{Spec}_{\text{clk}}(e_A) = \{\mathbf{z} : \deg_T(A) \leq \mathbf{z}\}. \quad (27)$$

Every accepting scheduler computes A , and an A -computable accepting scheduler exists. Consequently its least accepting-scheduler degree and its least deadline-oracle degree are both $\deg_T(A)$. The same result holds for the fixed rational guarded affine realization constructed in section 22.

Proof. Define a tree T_A on natural sequences. A sequence σ of length n is in T_A when

$$\forall i < n \ \forall x \leq i : \quad x \in A_n \Rightarrow t_A(x) \leq \sigma(i). \quad (28)$$

All entry stages needed in this formula can be computed by looking only at the finite enumeration through stage n . Thus the tree is uniformly computable, and under a bounded universal-enumeration convention it is primitive recursive. It is prefix closed because shortening a sequence only removes coordinates and reduces the enumeration stage inspected. Its infinite paths are exactly the g with $g(i) \geq b_A(i)$ for all i . Indeed, an eventual enumeration at stage s is enforced by all sufficiently long prefixes $n \geq \max(s, i + 1)$, and the reverse implication is immediate.

The fixed tree-verification interpreter takes the enumeration index as part of its finite input. It guesses each $g(i)$ in unary, checks the next prefix, and pulses on success. From an accepting schedule f , simulate the run and recover each completed unary value; this computes g from f . Equation (26) then computes A , so $A \leq_T f$. Taking $g = b_A$, which is A -computable, and using a default bit at every ignored action step gives an A -computable accepting schedule.

Now suppose a Z -computable deadline d is met by some accepting schedule, not assumed Z -computable. Unary guessing gives

$$b_A(i) \leq g(i) \leq \tau_f(i + 1) \leq d(i + 1).$$

The Z -computable function $i \mapsto d(i + 1)$ therefore decides A by equation (26). Hence $A \leq_T Z$. Conversely, if $A \leq_T Z$, use the A -computable schedule just constructed and simulate its successive accepting times with oracle Z . They form a Z -computable deadline. This proves equation (27), as well as the least-degree assertions. The geometric realization is step-exact, so the same inequalities and clocks are preserved there. $\square$

This is a useful concrete form of the clock obstruction: it can have any c.e. degree, not just the full halting degree. It also strengthens a purely negative statement. The construction identifies exactly which oracles are sufficient, and it supplies a matching least-degree accepting schedule. The ingredients are enumeration moduli and recursive trees; the theorem packages them as a uniform liveness and exact-substrate realization.

Corollary 8.4. *There is an input with recurrent runs but no computable deadline, whose deadline oracle spectrum is exactly the upper cone of $\mathbf{0}'$. There is a $\mathbf{0}'$ -computable accepting schedule and a $\mathbf{0}'$ -computable deadline, but every accepting schedule computes $\mathbf{0}'$.*

Proof. Take A to be the halting set in theorem 8.3. $\square$

8.4 Computable schedules are a different Σ_3^0 property

Let $\text{Comp}(e)$ mean that an accepting schedule is computable. There is a Σ_3^0 upper bound: existentially choose a program index, then require that for every k some finite simulation produces enough schedule bits for k accepting visits, while verifying totality on the first k inputs. A single larger finite stage combines these requirements. The Σ_3^0 hardness construction in [section 6](#) already produces a computable accepting schedule in every positive instance and no accepting schedule in any negative instance. Thus Comp is Σ_3^0 -complete for the fixed universal system.

Every computable accepting schedule has computable accepting times, so $\text{Comp} \subseteq \text{CD}$. The DNR example makes this inclusion strict. Equal completeness classifications do not identify the underlying semantics. In particular, existential quantification over program indices represents Comp , not arbitrary B acceptance.

8.5 Second routes to the separations

[write] *Source 03 proves the two separations for its fixed quadratic graph E_U , with the diagonal tree T_D (source 10's T_{DNR} , equation (24)) and the halting-time tree T_H . Its section “Two explicit strict separations” follows. Source 11's diagonal lemma is lemma 8.5, whose third proof is source 11's (section 8.7).*

8.6 Two explicit strict separations [\[source 03\]](#)

We fix an effective enumeration $(\varphi_e)_{e \in \mathbb{N}}$ of partial computable unary functions. A halting time means the number of steps in this fixed enumeration's simulation, counted from one. All tests bounded by a numerical stage are decidable. The infinite paths constructed below need not be computable; their existence is a classical mathematical statement.

8.6.1 Computably bounded recurrence without a computable run

For a binary string σ of length n , define

$$\sigma \in T_D \iff \forall e < n : \left[\varphi_e(e) \text{ halts within } n \text{ steps with output } b \in \{0, 1\} \right] \implies \sigma(e) \neq b. \quad (29)$$

The implication is understood separately for each e .

Lemma 8.5. T_D is a recursive binary tree with an infinite branch but no computable infinite branch.

Proof. Only n computations, each for n steps, are needed to decide membership. If a string satisfies these conditions at length n , a prefix of length $m < n$ satisfies them as well: every computation checked at stage m is among those checked at stage n . Thus T_D is a tree.

Define, without asserting computability,

$$f(e) = \begin{cases} 1 - b, & \varphi_e(e) \text{ eventually halts with } b \in \{0, 1\}, \\ 0, & \text{otherwise.} \end{cases}$$

Every finite prefix of f belongs to T_D , so a branch exists. If a branch f were computable, there would be an index j with $\varphi_j = f$. Then $\varphi_j(j)$ halts with bit $f(j)$. For every sufficiently large $n > j$, membership of $f \upharpoonright n$ would require $f(j) \neq \varphi_j(j) = f(j)$, a contradiction. $\square$

Theorem 8.6 (First strict inclusion). *There is an input to the fixed quadratic system in $\text{CB} \setminus \text{CP}$. Moreover, its successful paths share one computable deadline bound.*

Proof. Interpret a program that repeatedly chooses one bit, appends it to its current string, tests membership in T_D , halts on failure, and emits on success. Its recurrent paths correspond exactly to infinite branches of T_D . Such branches exist by Lemma 8.5.

For any k , there are only 2^k bit strings of length k . Simulate the counter implementation separately with each of these finite choice strings until it rejects or completes its k th emit. All these computations terminate: there are only k bounded membership tests, and each implemented operation terminates. Compute the maximum of their finite runtimes and monotonize these maxima. This gives a total computable $h(k)$ bounding the k th emit time on every surviving path. Hence CB holds.

If a recurrent counter path were computable, search its state sequence for successive emit boundaries and decode the chosen bits there. Event reflection and effective boundary decoding make this a total computable infinite branch of T_D . Lemma 8.5 rules this out. Thus CP does not hold. $\square$

This is a constructive specification of the system and its common deadline algorithm, not a constructive specification of an infinite branch. It is precisely the distinction between effective finite constraints and effective choice from their nonempty inverse limit.

8.6.2 Recurrence with no computable bound on any successful run

For $\sigma \in \mathbb{N}^n$, define another tree:

$$\sigma \in T_H \iff \forall e < n : [\varphi_e(e) \text{ halts within } n \text{ steps at time } t \implies \sigma(e) \geq t]. \quad (30)$$

Again membership is decidable by bounded simulation, and earlier checks remain in force under extension.

Lemma 8.7. *T_H is a recursive tree with an infinite branch. No infinite branch of T_H is pointwise bounded by a total computable function.*

Proof. Prefix closure and decidability follow as in Lemma 8.5. Define $f(e)$ to be the halting time of $\varphi_e(e)$ if it halts, and zero otherwise. Every prefix of f passes the test, so a branch exists.

Let g be any infinite branch. For every halting computation $\varphi_e(e)$ with time t , take a prefix length at least both $e + 1$ and t . The condition then forces $g(e) \geq t$. If a total computable function $b(e)$ bounded $g(e)$ for every e , one could decide the diagonal halting problem: simulate $\varphi_e(e)$ for $b(e)$ steps, answer yes if it halts and no otherwise. A halting computation must have halted within that bound. This contradicts undecidability of the halting problem. $\square$

Theorem 8.8 (Second strict inclusion). *There is an input to the fixed quadratic system in $\text{Rec} \setminus \text{CB}$. Every one of its recurrent paths has return times not pointwise bounded by any total computable function.*

Proof. Use the unary-guess program from Theorem 6.4 with tree T_H . A branch exists, so a recurrent run exists. On any recurrent run, let $g(e)$ be its e th completed natural-number guess. The corresponding guesses form a branch of T_H .

Producing the guess $g(e)$ requires $g(e)$ iterations of the explicit CONTINUE loop before the $(e + 1)$ st emit. Each implemented iteration takes at least one counter step. Therefore

$$g(e) \leq \tau_s(e + 1).$$

If a total computable deadline h bounded the marked times of this path, then $e \mapsto h(e + 1)$ would pointwise bound the branch g . This contradicts Lemma 8.7. The argument applies to every recurrent path, not just the particular branch used to prove existence. $\square$

Corollary 8.9 (Strict hierarchy inside one quadratic graph). *For suitable inputs of the same fixed E_U ,*

$$\boxed{\text{CP} \subsetneq \text{CB} \subsetneq \text{Rec.}}$$

The first two sets are both Σ_3^0 -complete; the third is Σ_1^1 -complete.

The pointwise form of the bound is intentional. If an eventually dominating computable function existed for a particular branch, finitely many constant corrections could turn it into a pointwise computable bound. Thus the second separation also excludes eventual computable domination for a successful run's return times.

[write] Source 11 proves the two separations for effectively presented binary-branching programs and their polynomial presentations, with the trees $\mathcal{T}_{\text{diag}}$ and $\mathcal{T}_{\text{halt}}$. Its section “Two explicit strict separations” follows; its proposition on $0'$ -computable runs is printed after theorem 7.3.

8.7 Two explicit strict separations [source 11]

Fix a standard effective enumeration (φ_e) of partial computable natural functions, with a decidable bounded-step simulation relation. A computation that is said to halt “within n steps” refers to this fixed simulation convention.

8.7.1 A computable deadline, but no computable recurrent run

Define a binary tree $\mathcal{T}_{\text{diag}} \subseteq \{0, 1\}^{<\mathbb{N}}$ by

$$\sigma \in \mathcal{T}_{\text{diag}} \iff \forall e < |\sigma| \left[\begin{array}{l} \varphi_e(e) \text{ halts within } |\sigma| \text{ steps} \\ \text{with output } b \in \{0, 1\} \end{array} \implies \sigma(e) \neq b \right]. \quad (31)$$

Membership is decidable by bounded simulation. The set is prefix-closed: every constraint seen at an earlier length is still seen at a later length.

[write] Source 11's lemma (“ $\mathcal{T}_{\text{diag}}$ has an infinite path but no computable infinite path”) is lemma 8.5 for the tree (31), which is (29).

Third proof of lemma 8.5 (source 11). For each e , choose the bit opposite to the eventual binary output of $\varphi_e(e)$ if there is one, and choose zero otherwise. This defines a path through the tree. If a computable binary path f existed, it would have an index e with $\varphi_e(e) = f(e)$. At a sufficiently long prefix, that halting computation would be detected and (31) would require $f(e) \neq f(e)$, a contradiction. $\square$

Now build a binary-branching program that chooses one bit at each stage, appends it to σ , tests (31), halts on failure, and marks success once.

Theorem 8.10 (Strictness of $\text{CRec} \subseteq \text{DRec}$). *This program has a recurrent run with a computable deadline but has no computable recurrent run. The same is true of its polynomial presentation.*

Proof. An infinite tree path gives a recurrent run. The test at each fixed length is a total bounded computation. There are finitely many binary words at that length. Exhaustively simulate the finitely many stage computations and take a maximum of their running times; add the preceding maxima and normalize. This gives a total computable deadline for every run that successfully reaches those stages, in particular for the recurrent one.

From a computable recurrent run, search for successive marked checkpoints and read the chosen bits. This would compute an infinite path in $\mathcal{T}_{\text{diag}}$, contradicting lemma 8.5. The incoming-edge compiler preserves complete paths and marker times exactly, so it preserves both conclusions. $\square$

The deadline need not be efficient. Its computability is enough. This example also shows why finite branching plus an effective clock does not automatically produce a computable infinite choice sequence.

8.7.2 Recurrence with no computable deadline at all

Let $h(e)$ be the halting time of $\varphi_e(e)$ when it halts. Define a tree $\mathcal{T}_{\text{halt}} \subseteq \mathbb{N}^{<\mathbb{N}}$ by

$$\sigma \in \mathcal{T}_{\text{halt}} \iff \forall e < |\sigma| \left[\varphi_e(e) \text{ halts within } |\sigma| \text{ steps} \implies h(e) \leq \sigma(e) \right]. \quad (32)$$

Again membership is decidable and the tree is prefix-closed. There is an infinite path f , obtained by taking $f(e) = h(e)$ for halting computations and $f(e) = 0$ otherwise.

Use the unary-guessing program of lemma 17.1 for this tree. Implement the choice of $f(e)$ with at least one machine transition per increment, and give exactly one mark for each successfully completed tree level.

Theorem 8.11 (Strictness of $\text{DRec} \subseteq \text{Rec}$). *The resulting binary-branching program is recurrent, but no recurrent run of it admits any total computable deadline. Its polynomial presentation has the same property.*

Proof. The path described above gives recurrence. Let ρ be any recurrent run and $f(e)$ its e th chosen natural number. For every halting e , sufficiently late tests in (32) force $h(e) \leq f(e)$. The unary choice mechanism ensures

$$f(e) \leq \tau_{e+1}(\rho).$$

If a total computable g bounded all visit times, then for every halting e ,

$$h(e) \leq f(e) \leq g(e+1).$$

The diagonal halting problem could therefore be decided by computing $g(e+1)$ and simulating $\varphi_e(e)$ for that many steps. This contradicts its undecidability. The argument applies to every recurrent run, not just to the particular path used to prove existence. $\square$

This example has a recurrent witness computable in the halting oracle: that oracle can determine each $h(e)$ or its divergence. Thus even existence of a $0'$ -computable recurrent run does not imply existence of a computable deadline.

Combining the results gives the strict witness hierarchy

$$\boxed{\text{CRec} \subsetneq \text{DRec} \subsetneq \text{Rec}.}$$

The first two index sets are both Σ_3^0 -complete; the third is Σ_1^1 -complete. The proof supplies concrete separating machines in addition to these index-set distinctions.

9 The deterministic specialization

If the transition ignores the action bit, an input has a unique computable run. This removes the source of the analytical jump.

Theorem 9.1 (Deterministic boundary). *For a computable deterministic system,*

$$\text{CD}(e) \iff \text{B}(e).$$

The common predicate is Π_2^0 in general and Π_2^0 -complete for a suitable fixed deterministic two-stack system. Finite reachability, fixed-deadline recurrence, and bounded-gap recurrence have sharp bounds Σ_1^0 , Π_1^0 , and Σ_2^0 , respectively.

Proof. A recurrent computable run yields a program computing its k th accepting time by simulation, so it yields a computable deadline. Conversely every deadline entails recurrence. For the unique run, the condition $\forall k \exists t N_e(t) \geq k$ has a decidable matrix. To obtain Π_2^0 hardness, on program index p sequentially simulate $\varphi_p(0), \varphi_p(1), \dots$, emitting one pulse after each halt. The run is recurrent exactly when φ_p is total. More generally, the same construction implements any $\forall k \exists s$ decidable normal form.

The reachability and fixed-deadline lower bounds in section 6 are already deterministic. For bounded gaps, the unique run yields the upper bound $\exists b \forall t$ directly. For hardness, use the Σ_2^0 -complete finiteness problem for a c.e. set W_p . Run a standard staged enumerator, marking all ordinary work steps accepting. At the j th distinct enumeration event, insert a nonaccepting block of at least j microsteps and then resume ordinary work. Each stage and each inserted block is finite. If W_p is finite, only finitely many nonaccepting blocks occur, so a gap bound exists. If W_p is infinite, nonaccepting blocks have unbounded lengths, so no gap bound exists.

For completeness, finiteness of c.e. sets has the required hardness by a standard moving-candidate construction. Given $\exists m \forall k R(e, m, k)$, at stage s let m_s be the least $m \leq s$ passing all tests $k \leq s$, or let $m_s = s + 1$ if there is none. Enumerate a fresh number whenever m_s changes. If there is a least true witness m , all smaller false candidates are eventually excluded and m_s stabilizes at m . If no witness exists, every finite candidate is eventually excluded and m_s changes infinitely often. This reduces the normal form to finiteness. These procedures are modes of one fixed deterministic interpreter, with accepting flags as above. $\square$

[write] Source 03 proves the same boundary (its Proposition “Deterministic recurrence”): *for effective deterministic systems, recurrence is Π_2^0 , and this bound is attained in an effectively coded family of counter programs, hence in the deterministic inputs of U (lemma 13.1).* Its proof refers to the last paragraph of the proof of theorem 12.1, which treats a one-successor relation.

Second proof (source 03). The upper bound is the formula in the last paragraph of Theorem 12.1. For a predicate $\forall b \exists c R(n, b, c)$ with decidable R , run a deterministic search for a witness c for $b = 0$, emit when found, then search for $b = 1$, and so on. There are infinitely many emits exactly when the predicate holds. A failed search makes no further emits. This is a reduction from any Π_2^0 predicate. $\square$

Property	Controlled universal system	Deterministic universal system
Finite reachability	Σ_1^0	Σ_1^0
Fixed computable deadline	Π_1^0	Π_1^0
Some finite gap bound	Σ_2^0	Σ_2^0
Some computable deadline	Σ_3^0	Π_2^0
Unrestricted recurrence	Σ_1^1	Π_2^0

Every entry denotes completeness for the corresponding fixed universal system. Infinite behavior by itself does not force analytic complexity; existential quantification over arbitrary compatible schedules does.

Part III

Low-degree transition laws and finite certificates

[write] *Part III prints the three polynomial compilers: source 03's degree-two control-vector law with its affine lower bound and fixed universal instance, source 11's degree-two incoming-edge law with its finite-history and deadline certificates and fixed analytic-complete instance, and source 10's two-stack trace quartic. They are compared first.*

10 Three compilers compared

[write] The three sources compile three different machine models, and their numbers are not interchangeable.

	Source 03, theorem 11.1	Source 11, theorem 15.1	Source 10, theorem 18.1
system	counter network	binary-branching unit-counter machine	two-stack machine, one trace
coordinates	$q + r + m$: one-hot control, counters, last-edge tag	$d + m$: counters, incoming-edge tag	horizon-indexed trace, $H(3 + R + G) + k$
degree	2	2 (P_2); sum of squares of degree 4 (P_4)	4
coefficient height	≤ 8 , sharp for this encoding (theorem 11.4)	≤ 6 , sharp for this encoding (section 15.3)	not stated
old tag	free (the source tag is unrestricted)	constrained by $U(b)^2$	—
auxiliaries	none	none	rule indicators and slacks, all unique
deadlines	—	degree 2 (corollary 16.2)	inside the quartic (equation (59))

The two height bounds concern different encodings; neither contradicts the other's sharpness. Source 03 keeps a one-hot control vector and separate source and target residuals, so two parallel increments share four residual terms and reach 8; source 11 reads the control from the target of the incoming edge and uses one flow residual per control, so the same pair reaches 6 in its expansion. Source 11 thereby answers in part two of source 03's questions for its own encoding: height drops

from 8 to 6 while q control coordinates disappear (“Height reduction versus dimension”, research question 33.14), and control coordinates are dispensable when tags are kept, the dual of “When are edge tags dispensable?” (research question 33.15).

For counter machines, source 11’s degree-two deadline certificate (corollary 16.2) uses the same surplus device as source 10’s deadline residual (equation (59)) but keeps the whole certificate at degree two, where source 10’s two-stack trace needs degree four.

Report A of this category (canonical-diophantine-certificates) encodes a zero guard of a counter program by the squared product $e_{i,a}x_{i,p} = 0$ (its `cdc:eq:zero-guard`) or by an affine big- M guard for fixed numerical registers, and observes that with free initial registers “the uniform-parameter presentation is then quartic unless another representation is supplied”. Sources 03 and 11 supply that other representation: the unsquared nonnegative product $e_ax_{j(a)}$ (source 03) or $c_ex_{i(e)}$ (source 11) keeps the transition relation at degree two uniformly in the registers, at the price of convexity and of a pure sum-of-squares form; source 11’s P_4 squares the guard, which is how Report A reaches its quartics.

11 An exact quadratic transition compiler [\[source 03\]](#)

11.1 The counter-network language

Let $Q = \{0, \dots, q-1\}$ be a nonempty set of controls, let there be r natural counters, and let $\mathcal{E} = \{0, \dots, m-1\}$ be a finite list of labelled edges. An edge a has source $u(a)$ and target $v(a)$, and one of the following operations:

Operation	Enabling condition	Counter update
<code>inc(j)</code>	Always	$x'_j = x_j + 1$
<code>dec(j)</code>	$x_j > 0$	$x'_j = x_j - 1$
<code>zero(j)</code>	$x_j = 0$	All counters unchanged
<code>nop</code>	Always	All counters unchanged

All counters not named in an update remain unchanged. Distinct edges remain distinct even if their endpoints and operations coincide. A usual conditional decrement instruction has a decrement edge and a zero edge from the same control. A binary nondeterministic choice uses two no-operation edges.

Write $\Delta_a \in \{-1, 0, 1\}^r$ for the update vector. It has at most one nonzero coordinate. Let $Z \subseteq \mathcal{E}$ consist of the zero-test edges, and let $j(a)$ be the tested counter for $a \in Z$. In particular, $\Delta_a = 0$ for every $a \in Z$.

11.2 State coordinates and the polynomial

Use the state space

$$s = (p, x, h) \in \mathbb{N}^q \times \mathbb{N}^r \times \mathbb{N}^m = \mathbb{N}^D, \quad D = q + r + m. \quad (33)$$

Here p is a one-hot control vector, x is the counter vector, and h records the most recently selected edge. The old tag h is deliberately ignored. Initial states use $h = 0$. If $s' = (p', x', h')$, write $e = h'$

and set

$$\begin{aligned}
L_0 &= \sum_{a \in \mathcal{E}} e_a - 1, \\
O_i &= p_i - \sum_{u(a)=i} e_a, \quad I_i = p'_i - \sum_{v(a)=i} e_a, \\
C_j &= x'_j - x_j - \sum_{a \in \mathcal{E}} \Delta_{a,j} e_a.
\end{aligned} \tag{34}$$

The compiler outputs

$$E_M(s, s') = L_0^2 + \sum_{i < q} O_i^2 + \sum_{i < q} I_i^2 + \sum_{j < r} C_j^2 + \sum_{a \in \mathcal{Z}} e_a x_{j(a)}. \tag{35}$$

This is one polynomial in $2D$ coordinates. There are no implicit witness variables. Some of its variables, namely the source tag coordinates, do not occur with nonzero coefficients.

Theorem 11.1 (Exact local semantics). *For every source state $s \in \mathbb{N}^D$ and target state $s' \in \mathbb{N}^D$, $E_M(s, s') = 0$ if and only if s has a one-hot control vector and s' is obtained by one enabled labelled edge of M , with h' the one-hot tag of that edge. The old source tag is unrestricted. Every state has at most*

$$b = \max_{i < q} \#\{a : u(a) = i\}$$

successors. The polynomial has degree at most two and is nonnegative on $\mathbb{R}_{\geq 0}^{2D}$.

Proof. All terms in (35) are nonnegative on the stated orthant. If the sum is zero, every square vanishes and every displayed product vanishes. The equation $L_0 = 0$ says that natural numbers e_a sum to one. Therefore there is exactly one a_* with $e_{a_*} = 1$, and all other selectors are zero. The equations $O_i = 0$ and $I_i = 0$ now give the source and target control vectors of precisely this edge. In particular, malformed source controls cannot acquire a successor.

The equations $C_j = 0$ give $x' = x + \Delta_{a_*}$. If the operation is a decrement, $x'_{j(a_*)} \in \mathbb{N}$ implies $x_{j(a_*)} > 0$. If it is a zero test, the term $e_{a_*} x_{j(a_*)}$ forces $x_{j(a_*)} = 0$. No additional guard is needed for an increment or a no-operation edge. Finally, $h' = e$ records the edge uniquely.

Conversely, an enabled edge with its prescribed target and tag makes every residual and zero-test term vanish. Once the edge is chosen, its target control, all counters, and tag are fixed. Distinct labelled edges have different target tags, so the number of successors is exactly the number of enabled edges at a well-formed source, and is at most b . Every term has degree at most two. Nonnegativity follows directly from the displayed factorization, without relying on the signs of expanded coefficients. $\square$

Remark 11.2 (Why there is no Boolean equation). Over $\mathbb{N}$, the equation $\sum e_a = 1$ already forces one-hotness. Equations $e_a(e_a - 1) = 0$ would be redundant. Over $\mathbb{R}_{\geq 0}$ this inference fails: for two parallel increment edges, selectors $(1/2, 1/2)$ satisfy the relaxed control and update equations. The continuous zero set is not the intended finite-branching graph.

Remark 11.3 (Why the zero-test term is not squared). The unsquared product $e_a x_j$ is nonnegative because both factors are natural numbers. Thus $E = 0$ forces this product to vanish. Replacing it by $(e_a x_j)^2$ would preserve natural semantics but unnecessarily raise the degree to four. Over unrestricted integers, the nonnegativity argument would no longer justify the unsquared term.

11.3 Coefficient height and circuit size

Theorem 11.4 (Uniform quantitative bounds). *The compiler satisfies $\text{ht}(E_M) \leq 8$, independently of q, r, m . The bound is attained by two parallel increment edges. If m_u is the number of increment or decrement edges and $z = |Z|$, the displayed factored evaluation uses at most*

$$1 + 2q + r + z \quad \text{multiplications,} \quad (36)$$

$$3m + m_u + 2q + 2r + z \quad \text{additions or subtractions,} \quad (37)$$

when input access, constants, and copying are free. Thus its factored size is linear in the edge table and state dimension.

Proof. For a selector e_a , the coefficient of e_a^2 is

$$1 + 1 + 1 + \langle \Delta_a, \Delta_a \rangle \leq 4.$$

For distinct selectors e_a, e_b , the coefficient is

$$2(1 + \mathbf{1}_{u(a)=u(b)} + \mathbf{1}_{v(a)=v(b)} + \langle \Delta_a, \Delta_b \rangle). \quad (38)$$

The inner product lies between -1 and 1 , so the absolute value is at most eight. Source-control-selector and target-control-selector coefficients are -2 . Counter-selector coefficients contributed by a square are 0 or ± 2 . A zero-test term contributes coefficient 1 to $e_a x_{j(a)}$; this cannot add to an update contribution for the same selector because $\Delta_a = 0$ on a zero edge. Counter squares have coefficient 1 , $x_j x'_j$ has coefficient -2 , and the remaining linear and constant terms have absolute value at most two. Old tag coordinates never occur. This exhausts the monomial types.

If two distinct edges have identical source, target, and increment update, every term inside the parentheses in (38) equals one, so their cross coefficient equals eight. This proves sharpness for this compiler, not for all possible encodings.

To count the factored computation, initialize L_0 to -1 and add each selector once, costing m additions. Initialize each O_i to p_i and subtract its outgoing selectors, for m more operations; do the same for I_i , for another m . Counter residuals cost $r + m_u$ operations. There are $1 + 2q + r$ squares and z zero-test products. Summing these scalar terms costs $2q + r + z$ additions. This yields (36)–(37). $\square$

The expanded polynomial may have quadratically many selector cross terms. The linear-size assertion concerns the factored circuit, not a sparse monomial list. Likewise, the operation count is for *one local step*; it is not a competing operation count for a complete universal halting certificate.

12 Why quadratic degree is minimal here [\[source 03\]](#)

The next obstruction is useful because it states exactly which constraints make the degree-two result optimal. There must be one equality, the entire state space must be $\mathbb{N}^D$, no existential variables or side conditions may hide a second relation, and outdegree must have a global finite bound.

Theorem 12.1 (Affine bounded-branching obstruction). *Let*

$$L(x, y) = c + \sum_{i=1}^D a_i x_i + \sum_{j=1}^D b_j y_j \quad (c, a_i, b_j \in \mathbb{Z})$$

define a graph on $\mathbb{N}^D$ by $L(x, y) = 0$. Suppose every vertex has at most $B < \infty$ successors.

If $D \geq 2$, either the graph has no edges, or all its targets lie in a fixed finite set. Consequently recurrence from a computable initial-state map, with any fixed decidable marker, is decidable. If $D = 1$, recurrence is at most Π_2^0 . In no dimension can such an affine graph have Σ_1^1 -complete recurrence.

Proof. Assume first that some edge exists and $D \geq 2$. If one b_j were zero, we could vary y_j arbitrarily along that edge, producing infinitely many successors. If two target coefficients had opposite signs, there would be a nonzero $v \in \mathbb{N}^D$ with $b \cdot v = 0$, and $y + tv$ would again give infinitely many successors. Hence the b_j are all nonzero and of the same sign. Multiply L by -1 if necessary, so all $b_j > 0$.

We claim all $a_i \geq 0$. Suppose instead that $a_i = -u < 0$, and fix a solution $(x^{(0)}, y^{(0)})$. Choose two target coordinates with coefficients b_1, b_2 . For $t \geq 0$ increase $x_i^{(0)}$ by $b_1 b_2 t$. Then, for each $k \in \{0, \dots, t\}$, the target obtained by adding

$$b_2 u k \text{ to } y_1^{(0)}, \quad b_1 u(t - k) \text{ to } y_2^{(0)}$$

and leaving other coordinates unchanged is a solution for the same new source. Indeed, the target contribution rises by $u b_1 b_2 t$, exactly cancelling the source contribution. These are $t + 1$ distinct successors, contradicting the fixed bound B for large t .

It follows that every edge satisfies

$$\sum_j b_j y_j = -c - \sum_i a_i x_i \leq -c.$$

Thus each target is in the finite box $0 \leq y_j \leq \lfloor (-c)/b_j \rfloor$. Every path, after its first step, lies in this fixed finite set. Enumerate the possible first successors and then inspect the finite graph. An infinite recurrent path exists exactly when a reachable directed cycle can visit a marked state. This is decidable. The empty-edge cases are also trivial. Under the promised global branching bound, the coefficient cases ruled out above may simply be treated as empty.

For $D = 1$, if $b_1 = 0$ an edge would have infinitely many targets, so the graph is empty. Otherwise each source has at most one successor, $y = (-c - a_1 x)/b_1$, when this is a natural integer. Its partial successor function is computable. Recurrence is therefore expressible by

$$\forall k \exists t \geq k : \text{the unique run is defined through time } t \text{ and is marked at } t,$$

a Π_2^0 statement. The strictness of the analytical hierarchy excludes Σ_1^1 -completeness in all cases. $\square$

Corollary 12.2 (Degree optimality). *Among single-polynomial natural transition relations on an unrestricted fixed finite-dimensional state space, with a uniform finite outdegree bound and no existential transition auxiliaries, degree two is the least degree that can realize Σ_1^1 -complete recurrent-path existence.*

Proof. The lower bound is Theorem 12.1; constant polynomials are included. The upper bound is the universal construction and recurrence theorem proved in Sections 6.9 and 13. $\square$

This lower bound does not apply to a conjunction of affine equations together with external nonlinear guards, to a restricted state domain that already encodes computation, or to unbounded branching. Those are different models. It also does not assert minimal state dimension or minimal coefficient height.

[write] Source 11 proves a degree-one obstruction under complementary hypotheses: theorem 12.1 allows a signed affine relation but needs a uniform finite branching bound; source 11's proposition 12.3 needs the affine polynomial to be nonnegative on the orthant but allows unbounded branching. Its subsection follows (from its section on the fixed analytic-complete law, section 17).

12.1 A degree-one obstruction [\[source 11\]](#)

Proposition 12.3 (Degree two is minimal in the stated nonnegative model). *Let $L(X, Y)$ be a polynomial of degree at most one with integer coefficients, nonnegative on $\mathbb{R}_{\geq 0}^{2D}$. For a computable marked set, recurrence in its natural zero graph is at most computably enumerable as a predicate of the initial state. In particular it cannot be Σ_1^1 -complete.*

Proof. Write $L = c + \alpha \cdot X + \beta \cdot Y$. Nonnegativity at zero and along each coordinate ray forces $c, \alpha_i, \beta_i \geq 0$. If $c > 0$, the graph is empty. Otherwise it is a Cartesian product $U \times V$, where U and V are coordinate-zero faces of $\mathbb{N}^D$.

A recurrent run from a exists precisely when $a \in U$ and there is a marked $z \in U \cap V$. Sufficiency follows from the run $a, z, z, \dots$. Necessity follows because every positive-time state of an infinite run lies in $U \cap V$, and some such state is marked. The existential search for z is computably enumerable. For the linear one-hot-style marker used in this report it is even an elementary integer linear feasibility problem. $\square$

This is a lower bound for a *single nonnegative affine polynomial*. It is not a classification of arbitrary signed affine relations, arbitrary systems, or different ambient-domain conventions.

13 A fixed universal quadratic substrate [\[source 03\]](#)

To transport reductions without introducing false recurrent runs, ordinary forward simulation is not enough. We require event reflection and terminating implementations of individual source steps.

Lemma 13.1 (Positive-step, event-reflecting compilation). *There is a fixed finite counter network U with at most two syntactic outgoing edges per control, a marked control q_A , and a computable input map $n \mapsto c(n)$ such that the following programs can be interpreted from n : deterministic effective algorithms, binary choices, and a distinguished emit operation. On well-formed programs, every implemented source step takes a positive finite number of counter steps; nondeterministic choices are exactly the source choices; and visits to q_A correspond exactly to completed emit operations. Decoding source configurations at instruction boundaries is effective. Halting is not replaced by an accepting self-loop.*

Proof. We spell out the operational ingredients; counter universality itself is classical [\[13\]](#). Represent a finite stack over a fixed alphabet by a natural number in base B , assigning symbols digits $1, \dots, B-1$ and representing the empty stack by zero. If the top is the least significant digit, push has the form $x \mapsto Bx + d$, and pop returns the remainder and quotient on division by B . An empty pop returns a specified blank symbol and leaves the stack empty.

Both operations can be implemented with increment, decrement, zero, and no-operation edges. To multiply x by B , clear a scratch counter, decrease x once per loop iteration while increasing the scratch counter B times, and then transfer the scratch value back. A finite string of d increments

completes the push. To divide, repeatedly decrease x , cycling a remainder in the finite control through $0, \dots, B - 1$ and increasing a quotient counter whenever the remainder wraps. When x reaches zero, transfer the quotient back. The final control records the remainder. Scratch counters can be cleared or restored by terminating transfer loops. Every loop counts down a natural number, so every individual stack operation terminates.

Two stacks, together with a current scanned symbol in the finite control, simulate a Turing tape. A head move pushes the old symbol onto one stack and pops from the other. Every source tape step therefore has a finite positive implementation. Conditional loops have two syntactic edges, a decrement edge and a zero edge, of which exactly one is enabled. A binary choice uses two no-operation edges; splitting a larger finite choice into a binary sequence is effective.

Take a fixed sequential interpreter for coded tape programs, augmented with binary choice and emit. A program description, its work tape, and its current simulated instruction are finite strings, so the interpreter stores and updates them by the same tape operations. It scans the finite code table to implement the next source instruction. The interpreter does not optimize away source steps. Its sole nondeterministic operation is the explicit binary choice. Route emit through one reserved marked control, with a single exit returning to the interpreter. No other operation enters that control. A halt instruction has no continuation.

Compile this fixed interpreter into the counter network just described. The initial natural n codes the program and its finite input. Parsing and loading terminate on valid codes; invalid codes may be rejected. The stack representation and interpreter bookkeeping give effective decoding at instruction boundaries. Every transition used to execute a source step is positive in duration, and no unfinished step passes through q_A . These facts give both event preservation and reflection. $\square$

Apply [Theorem 11.1](#) to U , and write its polynomial E_U . We will use the fixed marker $A(s) \iff p_{q_A} = 1$. The dimensions q_U, r_U, m_U and $D_U = q_U + r_U + m_U$ are finite constants, independent of the input.

Corollary 13.2 (One fixed low-degree graph). *A fixed integer polynomial E_U of degree two and height at most eight, on $\mathbb{N}^{D_U} \times \mathbb{N}^{D_U}$, has globally at most two successors per state and can interpret all the programs used in this manuscript. The complete successor list is computable from each state.*

Explicitness boundary. Formula (35) is an explicit compiler for every supplied finite edge table. The universal interpreter in [Lemma 13.1](#) is specified constructively at the simulation level, not delivered as a fully enumerated edge table. Accordingly, this paper claims no numerical value of D_U , no smallest universal machine, and no numerical expanded universal polynomial. The existence and degree/height bounds do not depend on such an optimization.

14 Finite paths remain Diophantine [\[source 03\]](#)

Proposition 14.1 (Quadratic fixed-horizon certificate). *For a fixed counter network and $T \in \mathbb{N}$, set*

$$Q_{M,T}(s_0, \dots, s_T) = \sum_{t=0}^{T-1} E_M(s_t, s_{t+1}).$$

On natural tuples, $Q_{M,T} = 0$ if and only if $s_0, \dots, s_T$ is a legal path. This certificate has degree at most two, $(T + 1)D$ state coordinates, and factored size linear in $T(q + r + m)$. Fixed endpoints can be imposed by adding squares of coordinate differences.

Proof. Every summand is nonnegative, so the sum vanishes exactly when every local transition is legal. Adding endpoint squares preserves this equivalence. The variable and size assertions follow by counting the displayed copies. $\square$

[write] *Source 03's source tag h is unrestricted (theorem 11.1), so $Q_{M,T}$ has infinitely many natural zeros unless the initial tag is pinned. For a single-fold certificate, pin the initial tag $h_0 = 0$ (source 03's initial states use $h = 0$) by adding the squares of its coordinates, as the last sentence of the proposition allows. Source 11's history certificate (corollary 16.1) substitutes its initial state $X_0 = a$ and so has its zeros in bijection with paths.*

This does not give a single fixed finite tuple of variables for arbitrary T . That compression is a different operation. It can be obtained through finite-trace arithmetic or MRDP, but cannot be inferred merely by writing a sum whose number of variables depends on T .

15 An incoming-edge polynomial normal form [\[source 11\]](#)

15.1 Encoding the current control without a control integer

Adjoin a fresh label $*$ and a dummy edge

$$e_0 : * \longrightarrow q_0$$

with no counter change. No edge targets $*$. Write E for the resulting edge set, $m = |E|$, and $\bar{Q} = Q \cup \{*\}$. Let

$$\delta_e \in \{0, \pm \mathbf{e}_1, \dots, \pm \mathbf{e}_d\}$$

be the effect of edge e . A zero edge has $\delta_e = 0$. Write $Z \subseteq E$ for the zero edges and $i(e)$ for their tested counter.

A state is

$$X = (x, b) \in \mathbb{N}^d \times \mathbb{N}^m, \quad \sum_{e \in E} b_e = 1.$$

Because the coordinates are natural, this last equation already says that b is one-hot. No Boolean equations are needed. If $b_e = 1$, the current control is $\text{tgt}(e)$. The initial state uses $b_{e_0} = 1$ and the chosen input counters. For a successor $Y = (y, c)$, the selected coordinate $c_f = 1$ records *the edge just taken*. Incoming-edge memory distinguishes parallel choices without creating an existential auxiliary witness.

Define the linear residuals

$$U(b) = \sum_{e \in E} b_e - 1, \tag{39}$$

$$V_q(b, c) = \sum_{\text{tgt}(e)=q} b_e - \sum_{\text{src}(e)=q} c_e \quad (q \in \bar{Q}), \tag{40}$$

$$W_i(x, y, c) = y_i - x_i - \sum_{e \in E} \delta_{e,i} c_e \quad (1 \leq i \leq d). \tag{41}$$

Set

$$C(X, Y) = U(b)^2 + U(c)^2 + \sum_{q \in \bar{Q}} V_q(b, c)^2 + \sum_{i=1}^d W_i(x, y, c)^2. \tag{42}$$

The two promised polynomials are

$$P_2(X, Y) = C(X, Y) + \sum_{e \in Z} c_e x_{i(e)}, \quad (43)$$

$$P_4(X, Y) = C(X, Y) + \sum_{e \in Z} (c_e x_{i(e)})^2. \quad (44)$$

The subscripts are degree bounds, not assertions that every instance attains the bound. For example, a machine without zero edges gives a quadratic P_4 .

15.2 Exactness and the absence of hidden witnesses

Theorem 15.1 (Witness-free polynomial compiler). *For every unit-counter machine as above, (43) and (44) have integer coefficients and satisfy:*

- (a) $\deg P_2 \leq 2$, and $P_2 \geq 0$ on $\mathbb{R}_{\geq 0}^{2(d+m)}$.
- (b) $\deg P_4 \leq 4$, and P_4 is a sum of $2 + |\bar{Q}| + d + |Z|$ squares, hence is nonnegative on all real inputs.
- (c) For $X, Y \in \mathbb{N}^{d+m}$, $P_2(X, Y) = 0$ if and only if $P_4(X, Y) = 0$, if and only if Y is the next incoming-edge state obtained by one enabled machine edge from X .
- (d) Every natural state has at most two successors. From an encoded initial state there is a bijection between edge-labelled machine paths and polynomial paths, at each finite length and for infinite runs.
- (e) Every expanded coefficient of either polynomial has absolute value at most six.

Proof. Parts (a) and (b) follow from the displayed formulas. In (a), each product $c_e x_{i(e)}$ is nonnegative on the nonnegative orthant; the other terms are squares.

Suppose $P_2(X, Y) = 0$ at natural coordinates. Nonnegativity forces every summand to vanish. In particular, $U(b) = U(c) = 0$, so b and c select unique edges, say e and f . The vector $(\sum_{\text{tgt}(e')=q} b_{e'})_{q \in \bar{Q}}$ is the unit vector at $\text{tgt}(e)$. Similarly the outgoing-source vector defined by c is the unit vector at $\text{src}(f)$. Thus all $V_q = 0$ mean precisely

$$\text{tgt}(e) = \text{src}(f).$$

The dummy edge cannot be selected as f : no incoming edge targets its source $*$. The equations $W_i = 0$ say $y = x + \delta_f$. If f decrements counter i , then $y_i \in \mathbb{N}$ forces $x_i \geq 1$. If f is a zero edge, its product term becomes $x_{i(f)}$, forcing that counter to be zero. These are exactly the edge's enabling conditions.

Conversely, an enabled edge gives one-hot b, c , matching source and target, the correct update, and all required zero tests. Every summand vanishes. Squaring a product does not alter its zero set, so the same equivalence holds for P_4 .

For a fixed valid X , each outgoing edge specifies all coordinates of Y : the updated counters and the incoming-edge unit vector. There are at most two such edges. If X is invalid, $U(b)^2$ prevents any successor. The bijection of paths follows inductively, and the coordinatewise correspondence extends to streams. The coefficient claim is established in the next subsection. $\square$

15.3 A coefficient audit independent of machine size

The height bound does not come from estimating the number of summands. Instead, each type of monomial can occur in only a bounded number of places. This distinction is essential because the machine may have many labels.

For distinct e, f , the possible coefficients are as follows:

Monomial	Expanded coefficient or bound
1	2
b_e, c_e	-2
b_e^2	2
c_e^2	$2 + \ \delta_e\ _2^2 \leq 3$
$b_e b_f$	$2 + 2\mathbf{1}_{\text{tgt}(e)=\text{tgt}(f)} \leq 4$
$c_e c_f$	$2 + 2\mathbf{1}_{\text{src}(e)=\text{src}(f)} + 2\delta_e \cdot \delta_f$
$b_e c_f$	$-2\mathbf{1}_{\text{tgt}(e)=\text{src}(f)}$
$x_i^2, y_i^2, x_i y_i$	1, 1, -2
$x_i c_e, y_i c_e$ from updates	$2\delta_{e,i}, -2\delta_{e,i}$

Since each effect is zero or a signed unit vector, $|\delta_e \cdot \delta_f| \leq 1$. The $c_e c_f$ coefficient therefore has absolute value at most six. A zero-edge guard contributes $c_e x_{i(e)}$ to P_2 , or $c_e^2 x_{i(e)}^2$ to P_4 , with coefficient one. Because a zero edge has zero effect, the quadratic guard does not overlap a nonzero update coefficient for that edge. All variables in the table belong to their indicated, distinct coordinate blocks. This accounts for every monomial.

Two parallel-source increment edges acting on the same counter attain the coefficient six. Thus six is sharp for this particular unweighted expansion; it is not a universal lower bound for all possible encodings.

The residual representation has linear size in $m + d + |\bar{Q}|$. Each b_e appears in one target residual and each c_e in one source residual; a nonzero effect appears in one update residual. Expanded polynomials can be quadratic in the number of edges, so the residual form is preferable for storage and proof checking.

15.4 A geometric observation

Homogenize P_2 with an additional variable h , replacing the constants in $U(b)$ and $U(c)$ by h . Its homogeneous quadratic form is a sum of squares of linear forms plus nonnegative coordinate products. Equivalently, its symmetric matrix is a positive-semidefinite matrix plus an entrywise nonnegative matrix. No difficult copositivity certification is being hidden: the decomposition is supplied by the compiler.

Proposition 15.2 (The one-step graph is semilinear). *The relation $P_2(X, Y) = 0$ over $\mathbb{N}$ is effectively a finite union of linear sets, and hence is Presburger-definable.*

Proof. Fix the finitely many possibilities for the selected incoming edge e and next edge f . Discard mismatched source/target pairs. The remaining case fixes b and c , requires $y = x + \delta_f$, and possibly sets a single counter to zero or imposes its positive lower bound. Every unrestricted counter can be chosen independently above a fixed lower bound; its increase adds the same unit vector to the old and new counter blocks. This is a linear set. Taking the finite union proves the assertion. $\square$

Thus the non-arithmetical complexity to come is not present in one local Diophantine relation. It arises from quantifying over coherent infinite paths through repeated copies of that relation.

16 Finite histories and what their certificates mean [\[source 11\]](#)

For a fixed initial state a , write

$$A(b) = \sum_{\text{tgt}(e) \in F} b_e.$$

On valid natural states, $A(b)$ is exactly the marked-state indicator. For $T \geq 1$, define

$$H_T(X_1, \dots, X_T) = \sum_{t=0}^{T-1} P_2(X_t, X_{t+1}) + (A(b_T) - 1)^2, \quad X_0 = a. \quad (45)$$

Corollary 16.1 (Exact finite-history certificates). *The natural zeros of H_T are in bijection with length- T paths from a ending at a marked state. The polynomial has degree at most two and exactly $T(d + m)$ available state coordinates. It uses no transition auxiliaries. There are at most 2^T zeros, and each counter on a zero is at most its initial value plus T .*

Proof. Every summand is nonnegative on natural coordinates. Vanishing is therefore equivalent to every local transition being valid and the terminal marker being one. Apply [theorem 15.1](#). The branching and counter bounds follow from the machine semantics. $\square$

For $T = 0$, endpoint acceptance is tested directly on a . Terminal deadlocks are allowed in a finite path; we have not padded halted computations with self-loops.

A deterministic machine has at most one zero for each fixed input and fixed T . This is *not* a single-fold MRDP representation of unbounded acceptance. The dimension in (45) grows with T , and existentially choosing T does not turn the family into a single fixed-arity polynomial while preserving its witness multiplicity. The coefficient height six is a one-step bound; summing time slices or substituting initial constants can increase coefficient heights.

[write] *The next two subsections are the second half of source 11's section "Compatible deadlines restore effective compactness"; its first half, the compactness theorem, is printed as the third route to theorem 5.1. The second subsection is a second route to the observation after equation (62).*

16.1 The explicit quadratic certificate family

Let $T = g(K)$ and put $X_0 = a$. Introduce K natural slacks $s_1, \dots, s_K$. Define

$$\begin{aligned} R_{g,K}(X_1, \dots, X_T, s_1, \dots, s_K) &= \sum_{t=0}^{T-1} P_2(X_t, X_{t+1}) \\ &\quad + \sum_{j=1}^K \left(\sum_{t=0}^{g(j)} A(b_t) - j - s_j \right)^2. \end{aligned} \quad (46)$$

Corollary 16.2 (Compatible quadratic deadline certificates). *$W_g(K)$ holds exactly when $R_{g,K}$ has a natural zero. This polynomial has degree at most two and $g(K)(d + m) + K$ available witness coordinates. Once the finite path is fixed, every slack is uniquely determined.*

Proof. Every transition summand is nonnegative, so a zero first gives a valid path. The remaining equations say that the number of marked positions up to $g(j)$ is at least j , with its excess recorded by s_j . This is exactly the simultaneous deadline requirement. The converse and slack uniqueness follow from the same equations. $\square$

For example 5.7, take $g(k) = 4k$. The first deadline is feasible: choose a budget of one and visit A at time four. The first two deadlines are not simultaneously feasible. A budget of two first visits A at time six, missing the deadline at four, whereas a budget of one has no second visit. This illustrates the extra compatibility missing in (20).

16.2 Arithmetical formulas and redundant witnesses

For fixed computable g , $W_g(K)$ is a recursive predicate. MRDP can express it by an existential polynomial certificate, giving a formula of the shape $\forall K \exists w B_g(K, w) = 0$. That display alone suggests a Π_2^0 upper bound, but it is not sharp. The complement of W_g is also recursive. Applying MRDP to that complement gives a polynomial D_g with

$$\forall K W_g(K) \iff \forall K \forall w D_g(K, w) \neq 0.$$

This is consistent with the Π_1^0 classification. Representation witnesses can obscure a simpler logical complexity; their presence does not prove that an additional quantifier alternation is necessary.

17 A fixed quadratic law with analytic-complete recurrence [\[source 11\]](#)

17.1 The classical source problem

A tree on $\mathbb{N}$ is a prefix-closed set $\mathcal{T} \subseteq \mathbb{N}^{<\mathbb{N}}$. Its path space is

$$[\mathcal{T}] = \{f \in \mathbb{N}^{\mathbb{N}} : \forall k f \upharpoonright k \in \mathcal{T}\}.$$

The standard effective tree normal form gives a uniformly decidable family of primitive-recursive trees $(\mathcal{T}_e)$ whose ill-foundedness index set

$$\text{IF} = \{e : [\mathcal{T}_e] \neq \emptyset\} \tag{47}$$

is Σ_1^1 -complete. This is the classical normal-form input to the high-undecidability argument; it is not a new theorem of this report [\[15\]](#); see also [\[21, Theorem 2.6\(g\)\]](#).

Here Σ_1^1 means existential quantification over one infinite natural sequence followed by an arithmetical condition. This is different from Σ_1^0 , which means computable enumerability. The superscripts are not interchangeable.

17.2 Replacing infinitely many choices by binary delays

Given e , a finite program performs the following stages:

```
s := empty sequence
repeat:
  n := 0
```

```

repeatedly choose either ( $n := n + 1$ ) or STOP-CHOOSING
append  $n$  to  $s$ 
decide whether  $s$  belongs to  $T_e$ 
if not, halt
visit the marked checkpoint exactly once

```

The choice of n is implemented by a binary branch. All computations between a completed choice and the next checkpoint are deterministic and terminate, because membership in $\mathcal{T}_e$ is decidable. There are no marked states in the unbounded choosing loop.

Lemma 17.1 (Binary recurrence realizes tree paths). *The program has a recurrent run if and only if $[\mathcal{T}_e] \neq \emptyset$.*

Proof. An infinite branch f determines finite choices $f(0), f(1), \dots$. Every test succeeds, so the induced run visits the checkpoint once at every stage. Conversely, infinitely many checkpoint visits imply infinitely many completed choices and successful tests. Their values form an infinite path through $\mathcal{T}_e$. A run which chooses forever at some stage has no further checkpoint visits and therefore is not recurrent. $\square$

This last observation is the mechanism that turns an infinitely branching tree problem into a finitely branching *recurrent* path problem. Without the recurrence requirement, the infinite choosing loop would be a spurious positive instance.

17.3 Uniformity and a fixed polynomial

A single universal interpreter can read e from its finite encoded input, perform the preceding program, and expose one designated checkpoint. A Turing-machine implementation can be converted to unit counters. For completeness, section 23 describes the two-stack route and the three-counter arithmetic routines underlying it. Every internal routine terminates at a simulated step boundary. Acceptance marks are attached only to the designated checkpoints. This gives one fixed finite counter machine, not one polynomial with infinitely many instructions.

Theorem 17.2 (Fixed-law analytic completeness). *There exist a fixed integer D , a fixed polynomial $P_2 \in \mathbb{Z}[X_1, \dots, X_D, Y_1, \dots, Y_D]$, a fixed linear marker A , and a total computable input map $a : \mathbb{N} \rightarrow \mathbb{N}^D$ such that:*

- (i) $\deg P_2 \leq 2$, $\text{ht}(P_2) \leq 6$, and $P_2 \geq 0$ on the nonnegative real orthant;
- (ii) every natural state has at most two successors under $P_2 = 0$;
- (iii) the set of n for which some run from $a(n)$ satisfies $P_2(X_t, X_{t+1}) = 0$ for every t and $A(X_t) = 1$ infinitely often is Σ_1^1 -complete.

There is also a fixed globally nonnegative sum-of-squares P_4 with degree at most four, height at most six, and exactly the same natural transition graph.

Proof. Compile the fixed universal checkpoint machine using theorem 15.1. The input map writes the code of e into its prescribed initial counter encoding and chooses the dummy incoming edge. By lemma 17.1, this reduces (47) to recurrent reachability. Membership in Σ_1^1 follows by existentially quantifying the stream $(X_t)_{t \in \mathbb{N}}$ and requiring the computable local equations together with $\forall k \exists t \geq k [A(X_t) = 1]$. The companion P_4 comes from the same compilation. $\square$

The fixed universal instruction table and its fully expanded polynomial are not supplied in this package. The theorem proves their existence by a specified finite compilation route; the implementation supplied here compiles any explicitly supplied finite counter table. This distinction matters when assessing how much of the construction is already executable.

18 Exact quartic certificates for two-stack computations

18.1 A canonical natural coding

A stack word is written top first, $w = d_1 \cdots d_l$ with $d_i \in \{1, 3\}$. Use the natural sentinel code

$$v(\epsilon) = 1, \quad v(dw) = 4v(w) + d. \quad (48)$$

For example,

$$v(1) = 5, \quad v(3) = 7, \quad v(13) = 29, \quad v(31) = 23.$$

The base-four representation has a leading sentinel 1; least significant digits are successive top symbols. This avoids a collision between the empty word and leading blank symbols. Push and pop are linear equations:

$$\text{push } d : n' = 4n + d, \quad \text{pop } d : n = 4n' + d, \quad n' \geq 1.$$

A top- d guard can be written, with a natural slack u , as

$$n = 4u + 4 + d, \quad \text{so the popped stack is } u + 1. \quad (49)$$

An empty guard is $n = 1$.

18.2 Rule format and variables

A finite machine has control labels in $\mathbb{N}$, two stack codes x, y , and a finite labelled rule set $\mathcal{R}$ of size R . A rule r has source s_r , target t_r , and one guard/update pair for each stack. Guards are unrestricted, empty, top-1, or top-3. Updates are stay, push-1, push-3, or pop; a pop requires an explicit top guard. The guards do not need to be mutually exclusive. Distinct rules remain distinct choices. Let G be the total number of top guards and E the total number of empty guards, counted across all rule/stack pairs.

Fix a valid initial configuration (q_0, x_0, y_0) and horizon H . Let $\vec{d} = (d_1, \dots, d_k)$ be strictly increasing deadlines with $1 \leq d_1 < \cdots < d_k \leq H$. The initial configuration is substituted as constants. Introduce the following natural variables:

Coordinates	Number
q_t, x_t, y_t , for $1 \leq t \leq H$	$3H$
Rule indicators $e_{t,r}$, for $0 \leq t < H, r \in \mathcal{R}$	HR
A top-guard slack $u_{t,r,a}$ for each top guard on stack a	HG
Deadline surplus w_j , for $1 \leq j \leq k$	k

The total is

$$N = H(3 + R + G) + k. \quad (50)$$

We now give all the polynomial equations; there is no hidden trace-coding or existential validity predicate.

18.3 The quadratic residuals

For each step t , impose Booleanity and one-hot selection:

$$e_{t,r}(e_{t,r} - 1) = 0 \quad (r \in \mathcal{R}), \quad (51)$$

$$\sum_{r \in \mathcal{R}} e_{t,r} - 1 = 0. \quad (52)$$

Impose source control and target control:

$$e_{t,r}(q_t - s_r) = 0 \quad (r \in \mathcal{R}), \quad (53)$$

$$q_{t+1} - \sum_{r \in \mathcal{R}} t_r e_{t,r} = 0. \quad (54)$$

For an empty guard on a stack value n_t , impose

$$e_{t,r}(n_t - 1) = 0. \quad (55)$$

For a top- d guard with its own slack $u = u_{t,r,a}$, impose

$$e_{t,r}(n_t - 4u - 4 - d) = 0, \quad (56)$$

$$(1 - e_{t,r})u = 0. \quad (57)$$

The second equation is not optional: it canonically sets the slack of an inactive rule to zero. Without it, each inactive top guard would contribute arbitrary spurious witnesses.

For every rule and every stack, impose one gated update residual, chosen from

update	residual
stay	$e_{t,r}(n_{t+1} - n_t)$
push d	$e_{t,r}(n_{t+1} - 4n_t - d)$
pop	$e_{t,r}(n_{t+1} - u_{t,r,a} - 1)$.

(58)

Let F be the accepting control labels. At the end of step t the acceptance indicator is

$$a_{t+1} = \sum_{r: t_r \in F} e_{t,r}.$$

It is automatically either zero or one on a solution. For each deadline d_j , impose

$$\sum_{t=0}^{d_j-1} \sum_{r: t_r \in F} e_{t,r} - j - w_j = 0. \quad (59)$$

Because $w_j \in \mathbb{N}$, this is exactly the assertion of at least j accepting visits by that deadline; the surplus is uniquely determined.

All residuals have degree at most two. The step-wise residual counts are

Constraint family	Residuals per step
Boolean indicators and one-hot sum	$R + 1$
Source guards and target equation	$R + 1$
Empty guards	E
Top guards and inactive-slack equations	$2G$
Two stack updates for each rule	$2R$

Thus there are exactly

$$M = H(4R + E + 2G + 2) + k \quad (60)$$

listed residuals, including any residual that simplifies to the zero polynomial for a special instance. Retaining those listed residuals is the counting convention implemented by the prototype.

18.4 The polynomial and the bijection theorem

Let $L_1, \dots, L_M$ denote the residuals above, and put

$$Q_{M,H,\vec{d}} = \sum_{i=1}^M L_i^2. \quad (61)$$

Here M denotes the machine; when no confusion is possible we simply write Q . It has integer coefficients and degree at most four, and is nonnegative on every real assignment.

Theorem 18.1 (Exact finite-trace compiler). *For every finite two-stack machine, canonical initial stacks, horizon H , and deadline vector $\vec{d}$, the natural zeros of Q are in bijection with rule-labelled legal runs of length H satisfying all the deadlines. The certificate has exactly the variable and listed-residual counts in equations (50) and (60). In particular, a fixed legal rule-labelled run has exactly one natural witness assignment when it satisfies the deadlines, and no assignment otherwise.*

Proof. Because all L_i take integer values, $Q = 0$ is equivalent to the vanishing of every residual. The Boolean equations force every $e_{t,r}$ to be zero or one, and the one-hot equation forces exactly one active rule per step. The source residual for that rule forces $q_t = s_r$, and the target equation forces $q_{t+1} = t_r$.

Induct on the step number, starting from the stipulated valid stack codes. For an empty guard of the active rule, equation (55) gives $n_t = 1$. For a top guard, equation (56) gives $n_t = 4(u + 1) + d$ with $u + 1 \geq 1$. As n_t is a canonical stack code, its least significant digit is precisely the tested top symbol, and its canonical tail has code $u + 1$. The appropriate update equation then produces exactly the stack resulting from stay, push, or pop. Thus the next stack code is canonical, completing the induction. No independent regular-language constraints on intermediate stack codes are needed.

The deadline residuals assert the required inequalities because their surpluses are natural. This constructs a legal deadline-respecting run from every natural zero.

Conversely, given a legal labelled run, insert its actual configurations, set the chosen indicator to one and all others to zero, set each active top slack to the tail code minus one, set every inactive top slack to zero, and set w_j to the acceptance count by d_j minus j . All variables are natural and all residuals vanish.

Finally, the labelled run fixes all configurations and indicators. Active top slacks are fixed by the quotient equation, inactive ones by equation (57), and deadline slacks by equation (59). This proves uniqueness and hence the bijection. The counts follow from the explicit inventory, and squaring quadratic residuals proves the degree bound. $\square$

Remark 18.2 (The boundary of the uniqueness claim). The dimension grows with H and the number of deadlines. This is a variable-size finite certificate, not a single fixed-arity single-fold representation of arbitrary c.e. sets. Replacing all finite traces by a fixed-arity MRDP polynomial can introduce additional witnesses; uniqueness is not asserted to survive that replacement. In particular, theorem 18.1 does not resolve the finite-fold or single-fold Diophantine representation questions.

18.5 Deadline compactness as a quartic schema

For a fixed computable deadline d , set $H = d(k)$ in the compiler, retaining all k deadlines. For the two-stack system we obtain the exact equivalence

$$D_d(e) \iff \forall k \geq 1 \exists \vec{z} \in \mathbb{N}^{N(k)} : Q_{e,k}(\vec{z}) = 0. \quad (62)$$

The varying polynomial and dimension are generated effectively from e, k . Equation (62) looks syntactically like a Π_2^0 assertion if one treats polynomial solvability as a black box. Its actual complexity is Π_1^0 : the zeros correspond to a finite decidable set of schedules, and that entire finite search can be carried out directly. Witness magnitude is not a reason to ignore known operational bounds.

This is another useful distinction. A Diophantine certificate interface can hide the fact that its particular instances are decidable. Complexity classification should use the strongest available semantics, not only the surface quantifier shape of one encoding.

[write] *Source 11 makes the same observation for its counter certificates, applying MRDP to the complement of W_g (section 16, subsection “Arithmetical formulas and redundant witnesses”).*

Part IV

Transport, fairness and exact geometric realizations

[write] *Part IV prints source 10’s clock-transfer theorem with source 03’s and source 11’s transport theorems, source 03’s fairness reduction, source 10’s compact three-dimensional realization and source 11’s planar realization.*

19 Which simulations preserve a computable clock?

A universality theorem can preserve finite halting while distorting temporal properties. To transfer the preceding results to an unconventional substrate, we therefore state a stronger, explicitly timed hypothesis.

Definition 19.1. *A computable signal-faithful block simulation maps each legal finite source history to a target history, compatibly with prefixes. Each source step expands to a nonempty, finite, effectively computable block. Every infinite target execution from an encoded initial state decodes into such blocks and projects to a source execution; there are no spurious infinite internal executions. Accepting visits occur precisely at the ends of blocks whose corresponding source step ends in an accepting state, with one target visit per source visit. The projection of finite blocks is effective.*

The condition quantifies over all target executions, not merely those chosen by a favorable simulator. It excludes, for example, an internal nondeterministic loop that can generate infinitely many fake accepting signals without simulating any further source step.

Theorem 19.2 (Effective clock transfer). *For effective finitely branching source systems, a computable signal-faithful block simulation preserves existential recurrence and the existence of a computable deadline in*

both directions. It preserves bounded-gap recurrence in both directions when block lengths have a uniform finite bound. Without that bound, forward preservation of bounded gaps can fail.

Proof. The one-to-one correspondence of accepting signals proves the recurrence assertion. For a fixed encoded input e and source length n , enumerate the finitely many legal histories of length at most n . Compute all of their target expansions and let $g_e(n)$ be the maximum expansion length, including zero for the empty history. This is a total computable, nondecreasing function; the effective finite-branching hypothesis is used exactly here.

If a source schedule meets a computable deadline d , its k th target pulse occurs no later than $g_e(d(k))$. The strictly increasing computable function $k + g_e(d(k))$ is therefore a target deadline. Conversely, each source step uses at least one target step. On any target run with deadline d , the k th source pulse occurs after at most $d(k)$ source steps, so the same d bounds the projected source run.

If each block has length at most L , a source gap bound b becomes a target gap bound Lb ; the initial delay obeys the same estimate. A target gap bound is also a source gap bound because blocks are nonempty. For failure without a uniform upper bound, take a source that pulses every step and expand its n th step into n nonaccepting steps followed by one pulse. This is a computable signal-faithful simulation, but the target gaps are unbounded although its progress clock is computable. $\square$

This theorem provides a practical contract for future translations to rewriting systems, cellular automata, or reaction-style models. It is not enough to cite their Turing completeness: one must prove finite effective branching, total expansion of each block, absence of rogue infinite executions, and the appropriate correspondence of acceptance signals. A step-exact simulation is especially valuable, since it preserves a specified deadline without modification.

[write] *Source 03's transport theorem is a second route to theorem 19.2 and extends it to computable runs (CP, source 10's Comp); source 11's terminating-macro lemma (lemma 23.2, in section 23) is a third. Source 03's section follows.*

20 Transport to other effective computational substrates [\[source 03\]](#)

A useful compiler should preserve more than halting. The next statement identifies a sufficient semantic interface that can be established for a particular rewriting language, circuit formalism, or other effective model. It avoids claiming that Turing completeness alone preserves an arbitrarily chosen notion of recurrence.

Definition 20.1 (Faithful finite-block simulation). A simulation between marked effective systems is a faithful finite-block simulation when each source edge is represented by a nonempty finite target block; the block is computable from the source edge; instruction boundaries are recognizable and effectively decodable; target paths from valid initial encodings have no extra nondeterministic behaviors or infinite unfinished blocks; and each source marked arrival produces exactly one target marked arrival, with no other target marks. For every finite source history its corresponding target history is therefore computable.

Theorem 20.2 (Preservation of the three recurrence notions). *A faithful finite-block simulation of an effectively finitely branching source system preserves and reflects Rec, CB, and CP. In particular, each such source system with computable finite successor lists and decidable marking admits an interpretation in the fixed quadratic substrate with these three predicates preserved.*

Proof. For unrestricted recurrence, concatenate the nonempty target blocks of an infinite source path. The result is an infinite target path with precisely the corresponding marked events. Conversely, a target recurrent path crosses infinitely many blocks and decodes to an infinite source path, because a single unfinished block cannot contain infinitely many marks or support an infinite valid execution. Mark reflection gives recurrence.

A computable source path yields a computable target path by computing and concatenating its finite blocks. To compute the target state at time t , generate blocks until their total positive lengths exceed t ; this terminates. A computable target recurrent path yields a computable source path by searching for successive recognizable boundaries and decoding them. Thus CP is preserved and reflected.

For the less immediate forward direction of CB, fix an initial source state c and a finite source horizon T . Its finite successor tree through depth T is computable and finite. Compute the target implementations of all edges in these finitely many histories, then take the largest total implemented duration of a history of length at most T . Including any initial loading time gives a total computable bound $F(c, T)$. If a source path has k th visit time at most $h(k)$, then the corresponding target path has k th marked time at most $F(c, h(k))$, after monotonicization if needed. This is a computable deadline. Conversely, each source step consumes at least one target step, so a target deadline also bounds the number of source steps preceding the corresponding marked event. Thus CB is reflected.

For the final assertion, an interpreter computes the complete finite list of source successors, then makes a bounded finite binary selection from that list and computes the selected next state. Invalid selections can be avoided by a finite binary decision tree. A zero-length list causes halting. The algorithm for the list terminates by hypothesis, and the selection procedure has a finite bound once the list is available; hence there is no spurious infinite guess within a source step. After a chosen edge is completed, the interpreter tests the decidable marker and emits exactly when required. Standard finite coding and the counter implementation of Section 13 realize this interface inside U . $\square$

The last assertion is under the promise that the supplied successor-list and marker algorithms are total on valid source states. The interpreter need not decide whether arbitrary program codes satisfy that promise. For a particular substrate this totality and faithful boundary decoding are the obligations to prove. A mere embedding of terminating computations would not justify any of the three infinite-path conclusions.

This theorem also explains why a change to a slower conventional machine model does not repair the noncomputable-deadline example. Effective finite branching permits a computable worst-case time distortion over every fixed source horizon. The absence of *any* computable bound survives such faithful changes of implementation.

21 One weak-fairness obligation already reaches the analytic boundary

[source 03]

Fairness concerns the choices made along a path, not just the existence of local transitions. We use the following precise convention. An action μ is weakly fair on an infinite run if, whenever it is continuously enabled from some point onward, it occurs infinitely often. Equivalently, each suffix on which it remains continuously enabled contains a subsequent occurrence. In the construction

below μ is enabled at every live state, so weak fairness simply requires infinitely many occurrences of μ . No fairness requirement is imposed on the other actions.

Theorem 21.1 (Explicit one-obligation fairness reduction). *Given a marked counter network with at most two outgoing edges per control, one can effectively construct a counter network with at most three outgoing edges per control and one designated action μ , enabled at every live state, such that*

$$\begin{array}{c} \text{the original has a recurrent infinite run} \\ \iff \\ \text{the new network has a weakly } \mu\text{-fair infinite run.} \end{array}$$

The new network again has an exact degree-two, height-at-most-eight polynomial transition relation. For a fixed universal instance, existence of such a fair run is Σ_1^1 -complete.

Proof. Replace every original control q by two controls $(q, 0)$ and $(q, 1)$, called unlocked and locked, and add a dead control $\perp$. Both copies retain the original outgoing edges and counter updates, but every such edge goes to the unlocked copy of its original target. Add one μ no-operation edge at every live control as follows:

$$\begin{array}{ll} (q, 0) \xrightarrow{\mu} (q, 1), & q \text{ marked,} \\ (q, 0) \xrightarrow{\mu} \perp, & q \text{ not marked,} \\ (q, 1) \xrightarrow{\mu} \perp, & \text{all } q. \end{array}$$

There are no outgoing edges from $\perp$. Thus μ is enabled at every live configuration, and each live control has at most two old edges plus one new edge.

An infinite fair run cannot take an edge to $\perp$, so its infinitely many μ occurrences must all be successful unlocked-to-locked transitions at marked original controls. After such a transition, another μ would reach $\perp$. At least one original edge must therefore occur before the next successful μ . Delete the μ steps and forget the lock bit. The remaining sequence contains infinitely many original edges, is a valid original run, and visits marked original controls infinitely often.

Conversely, given an original recurrent run, start unlocked and insert one successful μ step at each of its marked visits before taking its next original edge. That original edge unlocks the target. The resulting run is infinite and takes μ infinitely often, hence is weakly fair. The lock is what prevents a single accepting state from producing an artificial fair self-loop without source progress.

The quadratic compiler applies unchanged to the augmented edge table. Membership in Σ_1^1 follows by existentially quantifying an infinite path and checking transitions and fairness arithmetically. Apply the construction to U and use [Theorem 6.4](#) for hardness. $\square$

This is a specific elementary realization of the classical link between recurrence, trees, and fairness [\[15\]](#). It does not claim hardness for simultaneous fairness of every original edge, nor does it retain the binary outdegree bound: the stated bound is three.

22 A compact rational piecewise-affine substrate

22.1 Finite stacks as rational points

For a top-first finite word $w = d_1 \cdots d_l$, set

$$\rho(w) = \sum_{i=1}^l \frac{d_i}{4^i}, \quad \rho(\epsilon) = 0. \quad (63)$$

Every such point is rational and belongs to $[0, 1)$. The stack operations become rational affine maps:

$$\rho(dw) = \frac{\rho(w) + d}{4}, \quad (64)$$

$$\rho(w) = 4\rho(dw) - d. \quad (65)$$

For a valid nonempty stack, the top- d guard is exactly

$$\rho(w) \in [d/4, (d+1)/4), \quad d \in \{1, 3\}. \quad (66)$$

The empty stack is characterized by $\rho(w) = 0$. Strict upper endpoints are intentional: no finite valid tail has coordinate 1.

Choose distinct rational control levels $z_q \in [0, 1]$, for example $q/(s+1)$ for s finite control labels numbered $0, \dots, s-1$. A configuration is represented by

$$(\rho(w_L), \rho(w_R), z_q) \in [0, 1]^3. \quad (67)$$

Each rule has rational linear guards and an affine update on these three coordinates. A finite union of these guarded graphs is a finitely branching piecewise-affine transition relation. For a binary-controlled machine, there are two guarded piecewise-affine functions F_0, F_1 , one per action, whose pieces are disjoint on valid configurations. The finite guard partition can be refined and completed by a rejecting sink on its unused regions. This does not assert a finite-guard test for the entire countable set of finite-stack encodings; noncanonical points inside the chosen affine pieces may follow the same affine formulas.

Theorem 22.1 (Step-exact compact geometric realization). *There is a fixed binary-controlled rational guarded piecewise-affine system on $[0, 1]^3$, a fixed accepting set given by rational control levels, and a computable family of rational initial points, for which all the completeness statements in theorem 6.1 hold. The c.e.-degree spectrum realizations of theorem 8.3 hold as well. For every finite horizon, the natural quartic certificates of theorem 18.1 certify its encoded trajectories with exactly the same deadline and witness counts.*

Proof. Apply equations (63) and (67) to the fixed two-stack system constructed in section 6.6. Equations (64)–(66) show, by induction on the run, that each rule and its affine piece have the same enabledness, resulting configuration, and accepting label. The initial point is rational and computable from the finite input, and every reachable encoded point is a finite-stack point. Thus the simulation preserves every microstep, acceptance count, and deadline. The hierarchy and the spectrum results transfer without overhead. The quartic witnesses use $v(w)$, not $\rho(w)$, to represent exactly the same finite words. $\square$

The compactness of the ambient cube does not turn it into a finite-state system. Stack denominators and numerators in their alternative natural coding have no uniform bound. Nor is the guarded relation asserted to be closed or the functions continuous. Rational interval guards and exact real arithmetic are part of the mathematical semantics. None of these statements asserts noise tolerance, experimentally realizable infinite precision, or a physical solution of the halting problem.

22.2 One deterministic map with the scheduler in the initial point

The external control stream can instead be encoded in one more real coordinate. For $f \in \{0, 1\}^{\mathbb{N}}$, define

$$\eta(f) = \sum_{t=0}^{\infty} \frac{1 + 2f(t)}{4^{t+1}}, \quad C = \eta(\{0, 1\}^{\mathbb{N}}) \subseteq [1/3, 1]. \quad (68)$$

This is a Cantor set with disjoint first-digit regions

$$f(0) = 0 : \quad \eta(f) \in [1/3, 1/2], \quad f(0) = 1 : \quad \eta(f) \in [5/6, 1].$$

The digit can therefore be read by rational interval guards. With S the left shift on schedules,

$$\eta(Sf) = 4\eta(f) - 1 - 2f(0). \quad (69)$$

Define a deterministic guarded rational piecewise-affine map on $[0, 1]^4$ by

$$\mathcal{F}(X, z) = (F_b(X), 4z - 1 - 2b) \quad \text{on the first-digit region for } b, \quad (70)$$

with an arbitrary rejecting completion elsewhere. For a standard encoded initial point $(X_e, \eta(f))$, its orbit is exactly the controlled run on schedule f , with one bit consumed at every microstep, including steps where the action is ignored.

The map η preserves Turing degree under the usual effective representation of reals: f computes arbitrarily accurate rational approximations to $\eta(f)$ by truncation, while the separated digit regions allow recovery of each bit from increasingly accurate approximations to the real. Also, $\eta(f)$ is rational exactly when f is eventually periodic. Indeed, rational base-four expansions are eventually periodic, and conversely any eventual period yields a rational geometric series. The endpoint 1 is represented by the all-3 digit stream and causes no ambiguity within this digit set.

Theorem 22.2 (Initial-parameter hierarchy). *For the fixed deterministic map $\mathcal{F}$ above, with standard finite input e determining X_e , the following classifications are sharp:*

Question about a recurrent orbit from (X_e, z)	Complexity
The scheduler coordinate $z \in C \cap \mathbb{Q}$ is specified as finite input	Π_2^0
Some rational scheduler coordinate $z \in C \cap \mathbb{Q}$ works	Σ_3^0
Some computable scheduler coordinate $z \in C$ works	Σ_3^0
Some scheduler coordinate admits a computable progress deadline	Σ_3^0
Some scheduler coordinate of HIF degree works	Σ_3^0
Some unrestricted scheduler coordinate $z \in C$ works	Σ_1^1

The existential computable-parameter condition is strictly stronger than the existential computable-deadline condition, despite their equal index complexity.

Proof. A specified rational scheduler coordinate has an effectively computable orbit, giving the Π_2^0 upper bound. A deterministic mode ignoring the scheduler realizes the Π_2^0 lower bound from theorem 9.1, even with the fixed value $z = 1/3$.

Enumerate rational points of C by finite descriptions of eventually periodic digit streams. For a chosen description, recurrence is Π_2^0 ; existentially choosing the description gives Σ_3^0 . The Σ_3^0 reduction of section 6 uses only one completed finite unary choice of m . All later simulation steps

are deterministic and may receive the constant zero action. The finitely many nonzero action bits before completion of the guess, followed by zeros forever, form an eventually periodic schedule. Thus every positive instance has a rational scheduler coordinate, whereas a negative instance has no accepting coordinate at all. This proves Σ_3^0 hardness for rational coordinates.

Computable coordinates correspond exactly to computable schedules, so their classification is that of Comp. Computable deadlines correspond to CD, and the HIF condition is equivalent by theorem 7.3 and the degree equivalence under η . Unrestricted coordinates correspond to arbitrary schedules, giving Σ_1^1 completeness. The DNR construction in proposition 8.1 separates computable schedules from computable deadlines, and the step-exact coding preserves that separation. $\square$

The quantified domain of the initial coordinate is indispensable. “A deterministic dynamical system” alone does not fix the problem’s complexity. A specified rational initial point, an existentially chosen rational point, and an existentially chosen arbitrary real point encode three different questions. The analytical result is obtained by allowing an entire infinite schedule in the real initial condition, not by deriving noncomputable information from a finite rational input under a deterministic computable evolution.

[write] Source 11’s planar realization follows. It packs the finite control into the integer part of one coordinate, so its controlled relation lives in a planar strip where source 10’s lives in $[0, 1]^3$; it proves the Σ_1^1 row for a fixed system and the Σ_3^0 rows and separations across systems, where source 10’s theorem 22.1 carries every row, the c.e. spectra and a deterministic map with the scheduler in a fourth coordinate. Its stack code η is source 10’s ρ after the digit relabelling $d = 1 + 2a$, not source 10’s scheduler code η (section 2.3).

23 An unconventional realization: planar rational stack dynamics [source 11]

23.1 Finite stacks as rational points

The counter presentation is convenient for Diophantine equations, but the same phenomena occur in a bounded planar phase space. Let a finite binary stack be $w = a_1 \cdots a_\ell$, with a_1 at the top. Encode it by

$$\eta(w) = \sum_{j=1}^{\ell} \frac{1 + 2a_j}{4^j}, \quad \eta(\varepsilon) = 0. \quad (71)$$

Every code is rational and lies in $[0, 1)$. The encoding is injective: after clearing the larger power-of-four denominator, distinct finite words with digits one and three cannot differ only by trailing zero digits.

The empty stack is exactly zero. A nonempty stack with top zero has code in $[1/4, 1/2)$, while top one has code in $[3/4, 1)$. On these cells, push and pop are the affine operations

$$\text{push}_a(u) = \frac{1 + 2a + u}{4}, \quad (72)$$

$$\text{pop}_a(u) = 4u - (1 + 2a). \quad (73)$$

For valid finite codes these formulas are exact inverses. There is no numerical rounding and no test for the equality of unspecified computable real numbers.

23.2 Packing finite control into planar strips

Number the control labels $q = 0, \dots, s - 1$. Encode a two-stack configuration (q, L, R) by

$$(z, v) = (q + \eta(L), \eta(R)) \in [0, s) \times [0, 1). \quad (74)$$

Writing $u = z - q$, the left-stack instructions become

$$\begin{aligned} \text{push}_a(L) : (z, v) &\mapsto \left(q' + \frac{1 + 2a + z - q}{4}, v \right), \\ \text{pop}_a(L) : (z, v) &\mapsto (q' + 4(z - q) - (1 + 2a), v). \end{aligned}$$

Right-stack instructions use (72) or (73) on v and replace z by $q' + z - q$. Empty tests use $u = 0$ or $v = 0$. Top-symbol tests use the two half-open rational intervals above. Pure binary choices give two possible target strips without changing the stack fractions.

These are finitely many rational affine maps on rational rectangular cells, including degenerate cells for empty-stack tests. Pop tests have mutually disjoint domains, so they do not increase the number of enabled choices. The whole region can be rescaled into the unit square. Marking a control label means marking its corresponding strip.

Proposition 23.1 (Exact planar simulation). *A finite binary-choice two-stack machine has a rational piecewise-affine planar realization whose runs from encoded finite-stack inputs are in bijection with its machine runs. Every iterate of an encoded input is again an encoded finite-stack configuration. Marked visits are preserved exactly.*

Proof. The top-symbol cells separate all valid nonempty codes, and zero separates the empty code. Equations (72)–(73) implement the stack updates exactly. The integer strip records the unique next control label. Induction proves invariance of valid finite codes and one-step correspondence; iteration gives the path bijection. $\square$

Invalid real or rational points can have additional behaviors. They are not quantified over in the input problem. Universality is obtained from a computable sequence of exact finite rational input codes, not from arbitrary real inputs containing uncomputable information.

23.3 Why this is Turing-complete, and how it reaches counters

Two stacks represent a Turing tape: one contains the symbols left of the head, nearest first, and the other contains the current symbol and those to its right. A finite control holds a popped symbol while a move or rewrite is performed. Empty stacks supply the blank symbol. Explicit blank symbols can be stored when needed, so no infinite tape needs to be preloaded. Encoding a fixed finite tape alphabet into binary blocks reduces the alphabet to two symbols. Each Turing transition becomes a finite terminating stack routine.

For the counter implementation, one may instead encode each finite stack by an integer in base three with digits one and two, keeping the top at the least significant end:

$$N(a_1 \cdots a_\ell) = \sum_{j=1}^{\ell} (1 + a_j) 3^{j-1}.$$

Push replaces N by $3N + (1 + a)$, and pop divides by three, with remainder one or two. The two stack integers and one scratch counter suffice for the following routines. To multiply by three, decrement the source to zero, adding three units to the scratch counter on each pass, then transfer the scratch back and add the fixed digit. To divide by three, decrement the source while cycling a three-state remainder register and incrementing the scratch on each complete group of three; transfer the quotient back and retain the remainder in finite control. The other stack counter is untouched. Scratch starts and ends at zero. All loops terminate on natural inputs.

This supplies the elementary finite compilation route used in section 17. The alphabet and control size may grow, and no small universal state-count record is claimed. The underlying geometric perspective is related to generalized shifts [22]; the formulas above are given explicitly so that their boundary conventions can be checked independently.

23.4 Deadlines survive terminating simulations

Lemma 23.2 (Computable-deadline preservation under finite macros). *Let the source and target have computable complete finite successor lists. Suppose an exact effective simulation maps each source transition, after its choice has been made, to a positive, finite, deterministic target routine. Assume that macro boundaries are effectively recognizable, source configurations are effectively recoverable there, and there is no additional target behavior from encoded inputs. Mark only the boundary corresponding to each marked source configuration, once per visit. If the routines terminate for all reachable source configurations, then existence of recurrent runs, computable recurrent runs, and recurrent runs with computable deadlines is preserved in both directions.*

Proof. Run correspondence and checkpoint preservation give recurrence. A computable source run can be expanded by executing the routines. Conversely, from a computable recurrent target run, successive completed checkpoints and their source configurations can be found effectively; intermediate source steps can also be recovered from the simulation boundaries. This gives a computable source run. Infinite internal divergence cannot create an accepting target run, since it produces no further marked checkpoints.

For a source deadline g , the source tree up to depth $g(k)$ is finite and can be fully generated. Simulate all routines along all its paths and take the maximum total target length. Every simulation in this finite collection terminates, so the maximum is a total computable function of k . It bounds the corresponding target checkpoint time. In the reverse direction, every source transition uses at least one target transition, so source visit times are bounded by target visit times, up to a fixed initialization convention which can be absorbed into the bound. $\square$

The lemma is existential rather than an efficiency theorem. It does not assert a polynomial simulation-time bound.

Corollary 23.3 (Planar recurrence and the witness hierarchy). *A fixed finite rational piecewise-affine relation in a bounded planar region, with at most two successors on valid encoded states, has Σ_1^1 -complete recurrent reachability on an effectively encoded family of finite rational inputs. Across such effectively presented systems, the computable-run and computable-deadline problems are Σ_3^0 -complete and are strictly distinct. The separating examples of section 8.7 are realizable in this substrate.*

Proof. Use a fixed universal two-stack interpreter for the tree program and proposition 23.1 for the fixed-system assertion. For upper bounds, exact rational states and their finite successor lists are

effectively computable, so the arguments of theorem 6.8 apply. Hardness and the strict separations transfer through the terminating stack and counter routines using lemma 23.2. $\square$

These maps are not asserted to be continuous across their cell boundaries, noise-robust, volume-preserving, or injective. The result concerns exact rational dynamics. Removing those qualifications would be a different research problem.

Part V

Fixed-arity signatures, no-go theorems, implementation, formalization and questions

[write] *Part V prints source 10's fixed-arity quantifier signatures and no-go theorem with source 03's and source 11's finite-certificate obstructions, the three implementation and verification sections, the three formalization plans, the merged research questions and the three conclusions.*

24 Fixed-arity Diophantine normal forms and impossibility results

24.1 Parameter tuples and degree reduction

The repository interface equation (1) is stated with one free natural input. It yields the usual finite-tuple form by a polynomial injection. For example,

$$\langle u, v \rangle = (u + v)^2 + u$$

is injective on $\mathbb{N}^2$: for a fixed sum s , its values lie in $[s^2, s^2 + s]$, and these intervals are disjoint for different s . A c.e. relation on pairs can be enumerated under this coding; substitute the displayed polynomial into its MRDP representation. Iterate for larger tuples. The projection of any natural polynomial zero set is c.e. in the opposite direction, simply by enumerating natural witness tuples and checking exact integer evaluation.

Whenever a fixed polynomial has degree higher than four, it can be replaced by an existential quartic certificate. Split it as $P = P^+ - P^-$ with nonnegative integer coefficients. Compute the two parts by straight-line circuits using natural additions and multiplications, introduce one natural coordinate for each intermediate value, and impose residuals $z - x - y$, $z - xy$, and the equality of the final outputs. Squaring and adding the residuals produces a nonnegative quartic Q with

$$P(\vec{x}) = 0 \iff \exists \vec{v} \in \mathbb{N}^r : Q(\vec{x}, \vec{v}) = 0. \quad (75)$$

This is a degree normalization, not a small-variable claim. All circuit intermediates are natural because positive and negative parts were separated before evaluation. The extra coordinates belong to the same existential block, or to a universal block after negating the zero-existence condition.

24.2 The precise arithmetic signatures

Theorem 24.1 (Fixed-polynomial liveness signatures). *For every fixed computable binary-controlled system and every fixed computable deadline d , there exist fixed finite integer polynomials $P_d, P_{\text{gap}}, P_{\text{clk}}$ such*

that

$$D_d(e) \iff \forall \vec{y} \in \mathbb{N}^{r_d} : P_d(e, \vec{y}) \neq 0, \quad (76)$$

$$BG(e) \iff \exists b \in \mathbb{N} \forall \vec{y} \in \mathbb{N}^{r_g} : P_{\text{gap}}(e, b, \vec{y}) \neq 0, \quad (77)$$

$$CD(e) \iff \exists m \in \mathbb{N} \forall k \in \mathbb{N} \exists \vec{z} \in \mathbb{N}^{r_c} : P_{\text{clk}}(e, m, k, \vec{z}) = 0. \quad (78)$$

The polynomials can be chosen nonnegative of degree at most four, with additional fixed witness coordinates. No numerical fixed-arity bound is claimed. For a deterministic system, recurrence also has a fixed polynomial form $\forall k \exists \vec{z} P_{\text{det}}(e, k, \vec{z}) = 0$.

Proof. For the first formula, the failure relation

$$\text{Bad}_d(e) \iff \exists k : \neg \text{Fin}(e, d \upharpoonright k)$$

is c.e. Apply MRDP and negate its existential zero condition. This gives equation (76) by theorem 5.1.

For the second, let $\text{BadGap}(e, b)$ mean that some finite horizon has no $(b + 1)$ -admissible prefix. This is again c.e., uniformly in e, b . Its MRDP polynomial has no natural zero precisely when a run meets the gap bound $b + 1$. Existentially quantify b .

For the third, the relation

$$V^*(e, m, k) \iff \exists s V(e, m, k, s)$$

from equation (21) is c.e. uniformly in all three parameters. Its MRDP representation gives equation (78). Notice that the stage bound s and all computation witnesses can be included in the last existential block; there is no need to assert that the program index m is total by an extra outside formula.

In a deterministic system, “the first k accepting visits occur” is c.e. as a relation on e, k , which proves the last assertion. Finally apply equation (75). In a negated zero-existence condition, $\neg \exists \vec{y} P = 0$ becomes $\forall \vec{y} \forall \vec{v} Q \neq 0$, so the quantifier-block patterns are preserved. $\square$

These are theorem-level uses of MRDP. The accompanying program implements only the explicit finite-horizon compiler, not the general extraction of P_d , P_{gap} , or P_{clk} from arbitrary program syntax. The repository guide makes the same distinction for its existence interface [2].

24.3 Why a stronger universal polynomial compiler cannot exist

Theorem 24.2 (Quantifier-sensitive no-go theorem). *For the fixed universal controlled system of theorem 6.1, none of D_d , BG , CD , or B has an ordinary existential Diophantine representation as a set of finite inputs. Nor can there be a total computable procedure mapping each input e to a finite integer polynomial whose natural solvability is equivalent to any one of these properties. The unrestricted recurrence set B is not definable by any single first-order arithmetical formula, with any fixed finite number of number-quantifier alternations.*

Proof. Ordinary Diophantine sets are c.e.: enumerate the finite natural witness tuples and evaluate the polynomial. The same argument applies to a computably input-dependent polynomial with input-dependent finite arity: compute that instance’s finite syntax, then enumerate its witnesses. But a Π_1^0 -complete set, a Σ_2^0 -complete set, and a Σ_3^0 -complete set cannot be c.e., by strictness of the arithmetical hierarchy. A Σ_1^1 -complete set cannot be c.e. either.

If a single first-order arithmetic formula defined the analytic-complete set B , it would place that set at some finite level of the arithmetical hierarchy. Every analytic set many-one reduces to B , contradicting the existence of nonarithmetical analytic sets. This proves the final assertion. The finite-horizon schema equation (62) and the alternating formulas equations (76) to (78) do not contradict these conclusions: they keep the quantifiers that ordinary Diophantine representation would remove. $\square$

The theorem is not a prohibition on all useful arithmetic certificates. A supplied successful finite prefix has a quartic certificate; failure of a fixed deadline has an existential polynomial certificate; bounded-gap existence has a two-block description; computable-clock existence has a three-block description. The obstruction concerns the stronger assertion that one existential finite integer witness can characterize every temporal property of a Turing-complete substrate.

[write] Source 03's obstruction is a second route to theorem 24.2 for its fixed quadratic graph (and extends it to computable runs); source 11's (theorem 26.1) is a third, for recurrence. Source 03's section follows.

25 What finite Diophantine certificates can and cannot certify [source 03]

Theorem 25.1 (Finite-certificate obstruction). *For the fixed universal quadratic system, no ordinary Diophantine representation defines Rec, CB, or CP. In the case of Rec, no first-order arithmetic formula with any fixed finite arrangement of number quantifiers defines the set. More generally, there is no sound and complete computably checkable finite positive certificate system for any of these three predicates.*

Proof. An ordinary Diophantine set is computably enumerable. A sound and complete computably checkable finite certificate system also makes the accepted inputs computably enumerable: enumerate all input–certificate pairs and run the checker. Theorems 6.5 and 6.6 give Σ_3^0 -completeness, and Theorem 6.4 gives Σ_1^1 -completeness. Hierarchy strictness shows none of these sets is computably enumerable. The same strictness says that a Σ_1^1 -complete set is not arithmetic, proving the stronger assertion for unrestricted recurrence. $\square$

The obstruction is one of *uniform completeness*. An individual run can have an excellent finite proof, a particular program can have a simple invariant, and a restricted class can admit complete certificates. None of these facts conflicts with the theorem. For CB and CP, arithmetic formulas with three alternating number-quantifier blocks do exist; it is only finite existential certification that fails in general.

25.1 A useful positive fragment: finite recurrent regions

Proposition 25.2 (Finite-region lasso certificate). *Suppose a finite set F of N states contains an initial state and is closed under successors. A recurrent infinite path exists if and only if there is a path of length at most $N - 1$ from the initial state to a marked state, followed by a nonempty closed walk of length at most N back to that marked state. The finite path and closed walk form a checkable lasso certificate.*

Proof. If a recurrent path exists, some marked state $a \in F$ occurs infinitely often, since only finitely many states exist. There is a path from the initial state to a ; deleting loops gives one of length at most $N - 1$. There is also a positive return walk from a to itself. A shortest such return walk has no

repeated internal vertices and has length at most N . Conversely, follow the finite prefix and repeat the closed walk forever. The marked state is visited infinitely often. $\square$

For a known counter box, the state count includes controls and tags as well as counter values; the starting zero tag should also be included if it is not already in the selected set. The closure assumption is a separate obligation, not something implied by observing a few bounded trajectories. Each proposed finite lasso is checked using the local quadratic polynomial, and endpoint equality can be added by affine squares. A variable-length lasso predicate is computably enumerable and hence Diophantine via MRDP.

Lassos are not complete for unbounded counter systems. A single marked control with the transition $x \mapsto x + 1$ has a computable recurrent run with deadline $h(k) = k$, but no state ever repeats. Thus even the existence of a computable recurrent path need not have a periodic witness.

25.2 No hidden conclusion about finite-fold MRDP

The target tag makes the witness of an individual *labelled local edge* canonical, and a fixed finite path is checked exactly. This does not imply that a fixed-variable Diophantine representation of arbitrary halting has only finitely many witnesses, or only one witness. Passing from arbitrarily long histories to a fixed finite tuple is a substantive additional arithmetization, with its own witness behavior. No finite-fold or single-fold MRDP conjecture is resolved here.

26 Consequences for Diophantine representation [\[source 11\]](#)

Theorem 26.1 (No complete finite existential compiler for recurrence). *There is no total computable procedure which takes an arbitrary instance of the recurrent reachability problem in theorem 17.2 and produces a finite integer polynomial system whose natural solvability is equivalent to a positive recurrence answer. This remains impossible when degree, coefficient size, and the finite number of variables may depend arbitrarily on the input.*

Proof. The natural solvability of any effectively produced finite integer polynomial system is computably enumerable: enumerate tuples of the required finite arity and evaluate the equations exactly. A total computable reduction of recurrence to that property would make recurrence computably enumerable. But the recurrence set is Σ_1^1 -complete and not arithmetical, hence not computably enumerable. $\square$

In particular no one fixed MRDP-style existential equation represents that recurrence set. The problem is not merely a shortage of available variables. Adding a fixed finite number of alternating natural quantifier blocks also cannot represent it, because that would give an arithmetical definition. This is a sharp contrast with the c.e. finite-reachability relation in the same quadratic graph.

The same reasoning shows that there is no sound, complete, effectively enumerable finite-certificate system for all true recurrent instances. A sound proof assistant can verify particular positive recurrence proofs, and complete proof systems may exist for restricted classes. What fails is the claim that all positive instances of the unrestricted problem have certificates in one effectively enumerable complete scheme.

26.1 A useful contract for future substrate compilers

A compilation claim should specify the observation being preserved. In this setting, the following are different outputs:

Observation	Appropriate mathematical object
One step	One witness-free low-degree polynomial relation
A fixed finite horizon	A finite quadratic history certificate
Some finite accepting horizon	An ordinary c.e./MRDP representation
An infinite safe run	A compatible family of finite-prefix tests
Recurrence with a supplied computable clock	Deadline-compatible certificates
Some computable-clock recurrent run	A Σ_3^0 condition
Arbitrary recurrent behavior	A genuine infinite-path quantifier

A compiler that proves only finite acceptance cannot be reused for recurrence without an additional semantic theorem. Similarly, a finite collection of successful horizon tests is evidence about those horizons, not a proof of infinite recurrence.

27 The executable certificate package

27.1 Implementation surface

The archive includes a dependency-free Python module `code/clock_compiler.py`. Its sparse polynomial representation uses integer coefficients and explicitly stored monomials; it provides exact addition, multiplication, evaluation, and degree computation. Rational stack tests use Python's `exact fractions.Fraction`, never a floating-point approximation.

A machine is a finite tuple of Rule values with source and target labels and two `StackOp` guard/update pairs. The module checks the initial stack codes and rejects malformed guards, incompatible pop operations, and invalid deadline vectors. Its public construction is

```
certificate = compile_trace(machine, horizon, deadlines)
witness = certificate.witness(rule_sequence)
assert witness is not None
assert certificate.evaluate(witness) == 0
```

The certificate stores the residual polynomials and their variable names. An optional method expands their sum of squares into an ordinary quartic. Keeping the sum-of-squares representation normally avoids unnecessary expression growth. Witness generation follows the canonical assignment in the proof; it returns `None` for an illegal trace or for a missed deadline.

This interface supplies and verifies witnesses for finite traces. It does not contain a solver for arbitrary quartic equations and does not decide any infinite recurrence property.

27.2 A completely explicit example

The sample machine has initial state $(q, x, y) = (0, 1, 1)$, accepting state 2, and five rules:

Rule	Source	Target	Left stack	Right stack
0	0	1	push 1	stay
1	0	1	push 3	stay
2	1	2	top-1, pop	stay
3	1	2	top-3, pop	stay
4	2	0	empty, stay	stay

For $H = 5$ and deadlines $(2, 5)$, it has $R = 5$, $G = 2$, $E = 1$, and $k = 2$. The compiler therefore gives

$$N = 5(3 + 5 + 2) + 2 = 52, \quad M = 5(20 + 1 + 4 + 2) + 2 = 137.$$

The rule sequence $(0, 2, 4, 1, 3)$ produces

Time	q	x	y	Accepting
0	0	1	1	no
1	1	5	1	no
2	2	1	1	yes
3	0	1	1	no
4	1	7	1	no
5	2	1	1	yes

Both deadline surpluses are zero, and every active top slack is zero because the popped tail is empty with code one. The exported `artifacts/example_certificate.json` contains all 52 variable names, all 137 residuals in sparse form and readable text, the rule sequence, and the complete natural witness. Direct evaluation gives $Q = 0$, and expansion gives degree four.

27.3 Executed tests and their limits

The test script `code/test_clock_compiler.py` was executed using Python 3.13.5. The resulting report is stored in `artifacts/test_report.json`. All tests passed. The principal checked families are:

Finite check family	Count
Stack words of length at most eight: exact natural-code round trips	511
Exact affine push identities	1,022
Exact affine pop identities	510
Rule sequences for the sample machine, horizons zero through five	3,906
Candidate assignments exhaustively evaluated in a local auxiliary test	3,888
Guard/update cases exercising both stacks	3,150
Single-coordinate increments of valid witnesses, all rejected	496
Individual burst-prefix feasibility checks	12
Coherent burst-prefix obstructions	11

The report also records residual/variable count assertions, direct and expanded polynomial evaluation, degree checks, natural-domain rejection, and invalid-input cases. Its total of 17,197 counts combines assertions and enumerated candidate assignments; it should not be read as 17,197 independent infinite-system tests.

The exhaustive local test uses two legal, observationally identical stay rules, one with a top guard and one without. It finds exactly two zeros, one per rule label, with inactive slack zero. This tests a subtle part of the claimed bijection. All full correctness and infinite-behavior statements rest on the mathematical proofs, not on finite enumeration. The package contains no executable universal interpreter table and no machine-checked proof of the completeness reductions.

28 Executed verification and proof audit [\[source 03\]](#)

The accompanying Python implementation uses only the standard library. It represents expanded polynomials exactly as integer coefficients indexed by degree-at-most-two monomials. Its factored energy evaluator, sparse polynomial evaluator, and direct operational successor function are separate implementations. The verification program compares them and exercises malformed controls, malformed selectors, illegal decrements, zero tests, parallel labelled edges, and mutated valid target states.

The following results were obtained by running `python code/verify.py` with Python 3.13.5 and random seed 20260930:

Exact finite check category	Number
Local polynomial/transition equivalence checks	302,498
Sparse-versus-factored polynomial evaluations	27,652
Valid transitions explicitly checked	5,152
Coefficient tables checked	169
Finite-visit example paths checked	41
Affine-fiber growth examples checked	576

All checks passed. The first count includes exhaustive tests of the sixteen two-edge instruction-kind combinations on the documented small coordinate ranges, plus random networks and targeted mutations. The examples include the sharp coefficient-eight pair and the polynomial (18).

These tests do not establish a universal quantified theorem by themselves. They provide independent finite checks against errors in the explicit construction. In particular, no finite test proves Σ_1^1 -completeness, nonexistence of a computable bound, or absence of every computable branch. Those assertions rely on the mathematical proofs above. No Lean or Rocq execution was performed for this manuscript.

28.1 The highest-risk proof points

The local energy argument uses *natural* coordinates, not arbitrary integers. The global branching bound counts successor states, not witnesses of an existential transition formula. The affine lower bound depends on a *uniform* finite bound, rather than mere finite branching. The deadline compactness proof checks all elapsed deadlines, not just the final visit count. The strict-separation examples test bounded computations at finite stages, so their trees are decidable despite using undecidable halting facts in the proofs of existence or nonexistence. Finally, the fairness reduction requires its lock bit to rule out acceptance by stuttering.

Each of these distinctions appears explicitly in the statements, in the code where applicable, and in the separate `PROOF_AUDIT.md`. The package also states the non-instantiated universal-interpreter boundary.

29 Executable artifacts and verification results [\[source 11\]](#)

29.1 What is implemented

The package contains a standard-library-only Python implementation of finite unit-counter tables, a direct successor semantics, and an independent sparse integer-polynomial compiler for (43) and (44). It also contains exact rational stack operations, finite-prefix exploration, simultaneous deadline checking, and fully expanded JSON polynomials for the budget counterexample.

The sparse representation stores an integer coefficient and a sorted tuple of variable indices for every monomial. All integer and rational evaluations are exact. There is no numerical solver, floating-point zero test, or hidden computer-algebra oracle in these checks.

29.2 The performed checks

The reproducible run in `data/verification_results.json` reports:

Check	Number
All pairs in a complete natural-coordinate box for a guarded fixture	59,049
Independent residual/expanded-polynomial comparisons on rational inputs	1,000
Global quartic nonnegativity checks on signed integer inputs	1,000
Generated finite machines checked for degree and coefficient bounds	300
Exact finite-stack push/pop tests	2,046

Every state coordinate of the exhaustive fixture ranges over $\{0, 1, 2\}$, including malformed selectors. Thus the test does not assume one-hot encodings when checking that non-encodings cannot create zero-energy transitions. Seven state pairs in that box are genuine edges. The generated machines use a fixed seed, and their largest observed expanded coefficient height is six. A separate explicit two-increment fixture attains six.

For the budget example, the exploration checks horizons through 100. At horizon $4k$ it finds a path with at least k marks for $1 \leq k \leq 25$. The simultaneous deadline tests for $g(k) = 4k$ are true for $K = 1$ and false for $2 \leq K \leq 8$, exactly as the proof predicts.

These are finite regression tests of the compiler and examples. They do not establish the infinite-run theorems, the tree normal form, or literature priority. Those assertions rest on the mathematical arguments and the explicitly cited classical inputs.

29.3 Reproduction

From the extracted package directory, run

```
python code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

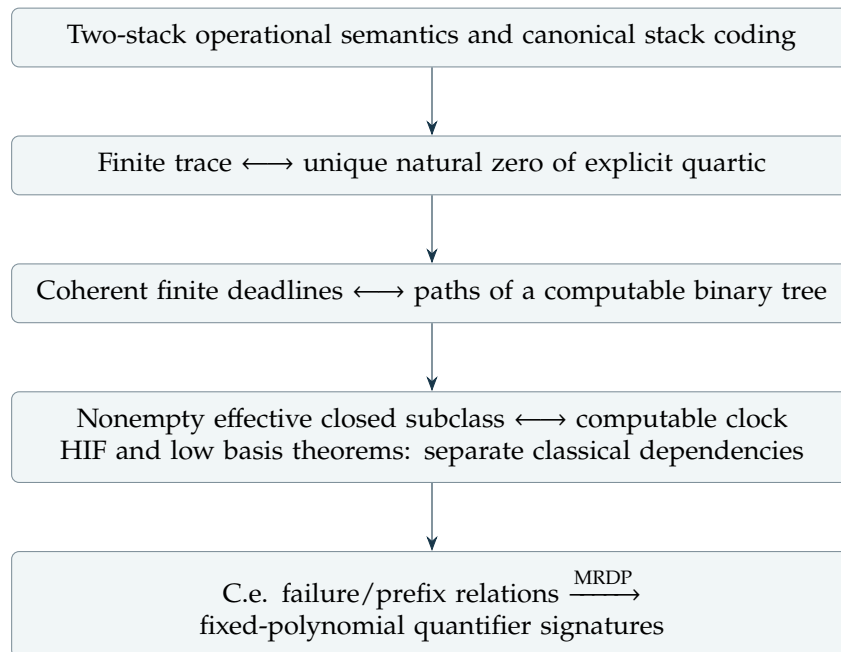
The Python scripts require Python 3.10 or later and no third-party packages. The LaTeX source uses standard packages and an embedded bibliography. The first command regenerates the JSON

results, the two example polynomials, and the finite-prefix CSV. The runtime field in the result JSON is naturally machine-dependent.

30 Formalization and integration with ProveIt

30.1 A small trusted interface rather than an all-at-once port

The natural integration point is a new liveness layer between finite trace semantics and the repository's MRDP theorem. The following separation keeps classical existence arguments away from executable certificate generation.



The arrows show a sensible implementation order, not that the MRDP theorem logically needs the basis theorems. In fact, the arithmetic signatures can be proved after the clock compactness theorem without any HIF construction.

30.2 Suggested module boundaries

A practical Lean development could use the following conceptual modules; these are proposed names, not assertions that the files already exist.

Liveness/Operational.lean. Define an effective binary transition system, finite execution by recursion, accepting counts, and the k th visit relation. Prove that a finite schedule fixes every configuration and count. Keep totalized rejecting behavior explicit rather than using an implicit convention for halted paths.

Liveness/Deadlines.lean. Define finite coherent deadline feasibility and its recursive binary tree. Prove prefix closure and the exact path characterization. Establish [theorem 5.1](#) through a clearly identified compactness or König lemma interface, rather than introducing an axiom named after the desired liveness theorem.

Liveness/TwoStackQuartic.lean. Define finite rule syntax, canonical stack coding, the variable-index sum type, and a polynomial generator. The critical theorem is an equivalence between a labelled trace and the generated constraints, followed by uniqueness of guard and deadline slacks. The sum-of-squares implication should be proved over integers, and only then used for natural valuations. The variable and residual counts become cardinality calculations on the index types. Python can generate examples, but the mathematical compiler must itself have a correctness proof.

Liveness/ClockDegrees.lean. Define computable domination and the deadline oracle spectrum on genuine oracle-recursion semantics. The easy implication from an HIF accepting schedule to a computable clock uses only the oracle computation of the k th accepting time. The reverse direction should depend on a separately formalized and audited HIF basis theorem. The c.e.-degree realization can be formalized independently from that basis theorem: its proof uses the explicit enumeration modulus and unary-cost bound.

Liveness/Diophantine.lean. Apply `Diophantine.mrdp` to the c.e. relations Bad_d , BadGap , and V^* . Do not try to prove these temporal properties themselves c.e.; that is exactly what the no-go theorem excludes. Keep the compiler for a concrete finite trace distinct from a general MRDP witness extractor.

Liveness/AffineRealization.lean. Prove rational push/pop identities and rule-by-rule preservation on the subset of encoded finite stacks. Only afterward define the arbitrary completion of the map on the rest of the cube. The proof need not decide whether every real point is a valid finite stack code. The fourth-coordinate construction additionally requires the convergent digit series and its computable inverse on the separated Cantor set.

30.3 Proof obligations most likely to conceal an error

The important boundary checks are short enough to list explicitly.

- (1) Prefix coherence: the length- $d(k)$ certificate must include every $j \leq k$ deadline, not merely the final acceptance count.
- (2) Finite branching: computable reachability of arbitrary long finite paths is not itself a substitute for a computably bounded finite successor list when extracting the effective compact tree.
- (3) Oracle scope: totality of a chosen schedule program is not equivalent to existence of an arbitrary schedule; a computable clock does not imply that its witnessing branch is computable.
- (4) Slack uniqueness: all inactive top slacks must be forced to zero, and active quotients must represent a positive sentinel tail.
- (5) Timing preservation: acceptance flags must have the stated microstep semantics, especially in the bounded-gap reductions.

- (6) Domains and arities: natural witnesses, real state coordinates, finite-horizon variable counts, and fixed MRDP witness dimensions must not be conflated.

These checks identify the actual mathematical proof boundary. Merely successfully type-checking an abstract theorem about an incorrectly defined “trace” or “scheduler” would not establish the intended result.

31 Connection to ProveIt and a formalization plan [\[source 03\]](#)

31.1 The inspected source boundary

The repository was inspected at commit

4e128356d0ef75308be8ed405d89aea2ffdb8a57

from 30 September 2026. The most directly relevant source file is

Computability/HilbertTenthProblem/Lean/
Diophantine/Common/DiophantineTrace.lean.

It declares `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph` [12]. The first is a bounded universal closure theorem, the second handles an exact finite iteration length, and the third existentially quantifies the finite length. The repository’s Hilbert-tenth README reports MRDP in both directions and documents the supporting formalization [11].

These are source-inspection observations. This session did not build the repository, rerun its axiom audits, or independently validate all claims in its README. Nothing in the present infinite-path proofs is claimed to be an existing ProveIt theorem merely because its finite-trace prerequisites are available there.

31.2 Suggested module separation

A sound implementation should keep at least three semantic layers distinct. The first layer is the finite edge-table language and its complete successor function. The second is the quadratic polynomial, with a theorem identifying its natural zero relation with the successor relation. The third is the space of infinite paths and the different quantifiers used in recurrence, computable deadlines, and computable paths. Naming the third layer “Diophantine” without the qualification “infinite-path semantics” would obscure the main obstruction of this paper.

A proposed dependency order is:

1. Define edges, one-hot controls, tagged states, and enabled successors. Prove that the returned successor list is complete, duplicate free by tags, and bounded by the source control’s syntactic outdegree.
2. Define the residual polynomials and the energy. Prove nonnegativity on natural assignments; prove that a finite sum of natural products and integer squares vanishes only when each component vanishes; then prove [Theorem 11.1](#) and the coefficient bound.
3. Connect the encoded finite-dimensional relation to the existing finite-iteration interfaces. Keep state coding, input zero, and natural versus positive domains explicit.

4. Formalize finite path trees and deadline-respecting subtrees. Establish Theorem 5.1 with an explicitly chosen König-lemma principle, recording any classical choice used for an infinite branch.
5. Formalize effective simulations, then the arithmetic upper bounds, recursive-tree reductions, explicit separating trees, and the affine lower obstruction. A full universal interpreter edge table is a further concrete implementation task, not a prerequisite for the generic compiler theorem.

The finite compiler theorem should be independently useful even before the analytical-hierarchy layer is available. Conversely, the compiler must not be advertised as a formalization of all recurrence results when only local polynomial identities have been checked.

32 Relationship to the inspected Provelt development [\[source 11\]](#)

32.1 Snapshot and mathematical dependencies

The repository documentation was inspected at immutable commit

e18718e837d43e162252f9a314e8cb797fbd1a1f.

The relevant entry points are the Hilbert-tenth-problem `README.md` and its Lean MRDP `.md` guide [\[18, 19\]](#); the bibliography provides their complete snapshot paths. The latter states the fixed-finite-polynomial MRDP interface, its converse, bounded universal quantification, primitive-recursive graphs, and the availability of separate trace interfaces. It also explicitly distinguishes the public existence theorem from a practical numerical polynomial-generation algorithm.

This report uses that distinction rather than obscuring it. The low-degree local compiler is explicit and executable for finite counter tables. MRDP is used to situate finite reachability and to identify the obstruction to existentially representing unrestricted recurrence. The infinite-path and deadline theorems are independent mathematical developments and have not been transplanted into the repository's theorem prover.

The inspected documentation records no added axioms for its MRDP endpoint and describes its audit files. That is the repository's documented status; this session did not rebuild or independently audit that Lean development. Only the relevant source documentation was inspected, not every research article in the repository.

32.2 A feasible formalization boundary

A first integration should formalize the finite algebra before attempting effective descriptive set theory. The natural dependency order is:

- (1) Define the finite edge table, enabling relation, incoming-edge states, and dummy initialization, using signed polynomial evaluation but natural counter coordinates.
- (2) Prove that a natural tuple summing to one is one-hot. Prove the zero-of-sum lemma for the squares and nonnegative products.
- (3) Prove the one-step equivalence and path bijection. Establish the degree and coefficient bounds by the monomial-case audit, not by testing examples.

- (4) Formalize the finite history and deadline polynomials, including the unique-slack statement. Connect finite reachability to the existing MRDP API.
- (5) Separately formalize streams, effective finite branching, and deadline compactness. Then add the computability-index arguments and the two diagonal separations.

The first four steps have a small and concrete mathematical interface. The last step needs a careful representation of partial computations, program indices, and second-order quantification. Merely introducing a predicate called “analytic complete” without the relevant reduction definitions would not constitute its formalization.

32.3 Theorem-status ledger

Component	Status in this report	External dependency
Quadratic/quartic local compiler	Full proof; implemented and tested	Elementary arithmetic
Height-six bound	Full monomial audit; regression tests	None
Fixed-law analytic completeness	Full transfer proof	Classical tree normal form and universality
Deadline compactness	Full proof; finite checker implemented	König’s lemma
Σ_3^0 classifications	Full upper/lower-bound proofs	Standard computation coding
Strict witness separations	Explicit constructions and proofs	Halting undecidability
Planar realization	Explicit formulas and simulation proof	Standard tape/stack simulation
New proof-assistant verification	Not performed	Future integration task
Literature priority	Not established	Further targeted review

33 Further research questions and concrete next targets

The questions below are proposed research directions. They are not all asserted to be previously published open problems. Each is a specific extension or optimization left unresolved by this article, rather than an invitation to remove a quantifier that the proved hierarchy forbids.

[write] The research questions of the three sources follow, source 10’s first. A question of source 03 or 11 that overlaps an earlier one is printed after it, unchanged, as a paragraph naming its source and title (“Overlapping question”); the others are printed as questions of their own, with their sources in their titles. Sources 03 and 11 wrote their questions as subsections; they are printed here as question environments with unchanged text. Cross-report notes name questions of Reports A (canonical-diophantine-certificates) and B (probabilistic-quantum-and-continuous-computation) of this category by their labels.

Research question 33.1 (Deadline spectra beyond c.e. cones). Which upward-closed classes of Turing degrees occur as $\text{Spec}_{\text{clk}}(e)$ for a single input of a fixed two-stack system? In particular, which degrees can be a least element?

Theorem 8.3 realizes every principal cone above a c.e. degree. Its proof uses a computable increasing enumeration and a modulus whose upper bounds decide the enumerated set. A route beyond the c.e. case is to study recursive trees whose paths admit domination-based recovery of a non-c.e. set. One must separately establish that a matching deadline is computable from that set; an arbitrary limit approximation need not have a convergence modulus computable from its limit. Nonprincipal spectra would distinguish “some sufficiently strong oracle” from a unique least clock resource.

Research question 33.2 (A verified effective-closed-subclass compiler). Can a repository implementation accept a certified computable tree T with $\emptyset \neq [T] \subseteq \mathcal{B}_c$ and extract the program for a valid deadline with a proved totality theorem?

Proposition 5.5 supplies the algorithm: search for a tree level where every node has the required number of accepting visits. The hard part of a proof assistant implementation is not polynomial arithmetic; it is packaging the compactness argument that proves termination of every search. An initial target can avoid HIF degrees entirely and establish a usable bridge from safety invariants to explicit progress programs.

Overlapping question of source 03, “Effective selection inside the deadline fragment”: Theorem 8.6 shows that computable deadlines alone do not permit computable path selection. Which additional conditions on branching, uniqueness, or extendibility do? For example, a decidable extendibility predicate would permit recursive choice of a next surviving successor. The substantive problem is to obtain such a predicate from a natural syntactic or algebraic condition rather than assume it as an oracle.

Research question 33.3 (Primitive-recursive and resource-bounded clocks). What are the sharp index classifications and degree-theoretic analogues when deadlines are required to be primitive recursive, elementary, or polynomially bounded in k ?

A computable deadline permits arbitrarily expensive total computation. Uniform syntactic families of total clocks change the quantifier accounting: for an effectively listed total family (d_m) , the existence condition has an immediate $\exists m \forall k$ upper bound, unlike the general Σ_3^0 condition that must verify totality. Establishing sharp lower bounds requires simulations whose pulse costs fit that family. The HIF basis argument by itself supplies no elementary or polynomial bound.

Overlapping question of source 11, “Witness spectra beyond computable deadlines”: **Question.** Classify recurrent runs whose visit-time functions are bounded by primitive-recursive, elementary, polynomial, or oracle-computable functions. A fixed effective family of total clocks gives one kind of index set; arbitrary program indices with a totality condition give another. The proof of theorem 6.8 suggests a systematic treatment, but the exact hardness can depend on which clock descriptions are allowed.

Research question 33.4 (Clock synthesis under regular reachability abstractions). Which effective regular abstractions certify a nonempty closed subclass of accepting schedules, and when do they compute useful deadlines?

To–Libkin-type recurrent reachability procedures and their recent extensions exploit effective regularity assumptions [10]. A promising bridge is to turn a regular recurrent witness into a recursive schedule tree with a quantified progress guarantee. The key task is to prove that an abstract infinite path corresponds to a concrete path; a regular overapproximation may contain

recurrent behavior absent from the concrete system. The deadline extraction should be conditional on a proved simulation or refinement relation, not just apparent recurrence in an abstraction.

Overlapping question of source 11, “Decidable fragments and certificate extraction”: **Target.** Identify structural conditions ensuring that a recurrent run has a finite lasso or an effectively bounded return schedule, and turn those conditions into a complete certificate procedure for that class. Existing regular-model-checking work supplies important positive contrasts to the unrestricted setting [23]. A careful bridge should state exactly which regularity or acceleration hypothesis rules out the diagonal examples.

Research question 33.5 (Probabilistic and adversarial progress). How do computable deadlines interact with almost-sure recurrence, positive-probability recurrence, and strategies that work against every adversary?

These quantifiers are different from existential selection of one schedule. Recent regular-model-checking work treats both sure and almost-sure properties under explicit hypotheses [10]. For the present framework, one concrete target is a measure-theoretic substitute for the closed-subclass condition: does a positive-measure computable-deadline class exist whenever recurrence has a specified effective probability bound? Another target is to formalize the precise order of strategy, opponent, time-bound, and probability quantifiers before attempting any Diophantine compilation.

[write] Compare Report B’s question $pqc:q:continuous$ (continuous-time and robust physical semantics). Report B’s Part I classifies almost-sure termination and probability thresholds; the deterministic Π_2^0 row of theorem 9.1 is their non-probabilistic analogue.

Research question 33.6 (Minimal geometric dimension and regularity). Can the clock and c.e.-degree-spectrum realizations be achieved with fewer coordinates or stronger regularity, while retaining a finite rational input encoding and an explicitly stated scheduler domain?

The construction here uses three coordinates for a controlled relation and four for an autonomous map with a scheduler parameter. It does not claim optimality. Reducing dimension by interleaving codes may destroy piecewise-affineness or the finite guard partition. Requiring continuity, invertibility, or a smooth flow creates separate obligations. Any candidate should first establish a step-exact or computable signal-faithful simulation, then determine which fixed deadlines or gap bounds survive.

[write] Partly answered for the controlled relation by source 11: its planar realization (proposition 23.1 and corollary 23.3) packs the finite control into the integer part of one coordinate, so the rows it proves (the fixed-system Σ_1^1 row, and the Σ_3^0 rows and separations across systems) hold for a guarded rational piecewise-affine relation on a planar region instead of $[0, 1]^3$. Whether the same packing applied to source 10’s fixed two-stack system carries every row of theorem 22.1, the c.e. spectra included, on a planar region (and the autonomous map on three coordinates) is not proved here; the rules of that system act affinely on each rectangle, which suggests it. Report B’s question “Reduce the three-dimensional realizer” asks for the continuous version, which this does not touch.

Research question 33.7 (Robust encodings and noise). What survives under bounded perturbations, finite precision, or a noise-tolerant decoding rule?

The current rational codes represent unbounded information at ever finer scales. Noise can erase low-order digits, so compact exact universality alone says nothing about robustness. An appropriate problem is to specify an error model and prove either a robust simulation contract

or an impossibility theorem for a chosen deadline class. This should be treated as a change of mathematical model, not as an implementation detail of the exact construction.

Overlapping question of source 11, “Robust planar computation”: **Question.** Which of the recurrence distinctions survive under continuous maps, closed guard conventions, finite perturbation tolerance, or uniform gaps from decision boundaries? The current exact affine coding does not establish robustness. An interesting positive theorem would provide explicit guard margins and a noise model while retaining the intended observation semantics; an interesting negative theorem would isolate the compactness principle that makes analytic completeness impossible.

[write] Compare Report B’s question `pqc:cb:q:noise` (different noise semantics), and Report B’s Part III, whose continuous realizations carry noise radii that the discontinuous exact maps of theorem 22.1 and proposition 23.1 do not.

Research question 33.8 (Other unconventional substrates with timing certificates). Can one build clock-faithful translations to interaction nets, contextual rewriting, tiling-based computation, or reaction systems with an explicit effective zero-test mechanism?

The useful deliverable would be a simulation theorem satisfying theorem 19.2, plus a local polynomial compiler for the target’s own finite steps. Mere finite-halting universality is insufficient. The first obligations are termination of administrative phases, absence of accepting internal divergence, and effective bounds on all finite simulation trees. Restricting to a model with no zero tests may radically change its computational power, so the operational language must be fixed before importing universality claims.

[write] Batch 78, 2 October 2026: partly answered, it appears, for reaction systems whose explicit zero test is an inhibitor, by Part I of the neighbouring report `quadratic-orthant-certificates` of this category (written in commit `c51b9880d`; its source 08, batch-78 manuscript 08, which does not draw the connection itself). Its guarded register-machine embedding (in its section `qoc:mp:sec:variants`) simulates a register machine by maximal-parallel multiset rewriting with one control object: an increment is one rule, and a decrement-or-branch is a consuming rule together with a rule inhibited by the presence of the register’s object. Checked at this write against the definition before theorem 19.2: the source machine is deterministic, hence effectively finitely branching; each instruction is exactly one nonempty round, so the blocks have length one, are computable and are uniformly bounded (bounded gaps transfer as well); with one control object each round is forced, so no target execution from an encoded state has internal steps or alternative branches; and accepting signals are read off the control object, one per source visit. One convention is needed: in that Part’s semantics the zero extent is the unique maximal extent at a deadlocked state, so a halted run continues by empty rounds (its section `qoc:mp:sec:semantics`). These must count as termination, as its first-halting certificate (`qoc:mp:cor:firsthalt`) counts them; otherwise an accepting halt label would emit infinitely many spurious accepting signals. The local polynomial compiler for the target’s own finite steps is its guarded-round certificate, of degree two once the guard bits have degree-two certificates (`qoc:mp:prop:guards`). So the embedding appears to be a computable signal-faithful block simulation with block length one, and the question is answered only partly: for register-machine sources and this one class of reaction systems. A clock-faithful simulation of this report’s own fixed universal systems by register machines is a separate step, not checked here; the embedding is elementary (its source calls it no improvement on unguarded universality), and guarded histories are not implemented in that report’s shipped code.

Overlapping question of source 11, “Verified front ends for unconventional substrates”: **Target.** Develop marker-preserving front ends from exact graph rewriting, queue systems, symbolic dynamical maps, and reversible interpreters to the finite edge table. Each front end should provide a terminating-macro theorem of the kind used in lemma 23.2. This prevents a common failure mode: a correct finite-step simulation that introduces an infinite internal run and thereby changes a liveness property.

[write] Compare Report A's question $cdc:q:ski$ (bridges between combinatory or graph evaluation and certificates).

Research question 33.9 (Smaller exact local certificates). Can the counts $H(3 + R + G) + k$ and $H(4R + E + 2G + 2) + k$ be improved without losing the labelled-trace bijection or raising degree above four?

Possible savings include eliminating deterministic rule selectors, sharing guards when their active conditions are provably disjoint, and using a different control encoding. Each change should include a new witness-uniqueness proof; eliminating inactive-slack equations without an alternative canonicalization is not a valid optimization. Lower bounds for natural quadratic encodings of guarded affine transitions would also be informative. The present counts are exact for this compiler, not optimal across all compilers.

[write] Compare Report A's question $cdc:q:degree$ and Report B's question $pqc:q:smaller$. For counter machines, source 11's deadline certificate (corollary 16.2) already stays at degree two.

Research question 33.10 (Concrete fixed-arity MRDP extraction). Can the three c.e. relations used in theorem 24.1 be compiled to explicit fixed polynomials with audited witness counts, operation counts, and a verified correspondence to the programs?

The repository's MRDP existence theorem is already enough for the logical normal forms. A practical extractor would require a compiler for the particular program semantics and an explicit accounting of all arithmetic encodings. This project should not begin by promising preservation of single-fold witnesses: remark 18.2 marks that as a separate and substantially stronger issue.

[write] Compare Report B's questions $pqc:q:direct$ and $pqc:cb:q:extract$.

Research question 33.11 (Quantitative certificates for the degree realizations). Can one export finite prefixes of the c.e.-degree spectrum construction as exact quartic certificates together with a formal proof that every successful deadline computes the intended c.e. set?

This is a focused near-term project combining the most concrete parts of the article. It needs a standard bounded enumeration machine, the tree predicate equation (28), a verified unary-cost lower bound, and the finite compiler. Finite examples alone cannot certify the lower-degree claim, but they can exercise every arithmetic component used by its proof.

Research question 33.12 (The mass problem of finding a recurrent schedule). How do the solution sets $\mathcal{B}_e$, their nonempty effectively closed subclasses, and their computable-deadline unions compare under uniform and nonuniform reductions between mass problems?

Theorem 7.3 characterizes nonemptiness of the computably clocked part through HIF solutions, but it does not provide a uniform algorithm selecting a schedule from a clock. The DNR example rules out the most naive selector. A finer classification could separate the difficulty of finding a clock, finding a closed subclass, and selecting a path inside that subclass. This is especially relevant to proof-producing synthesis systems, where “there exists a strategy” and “a strategy can be extracted uniformly” are operationally very different outputs.

Overlapping question of source 03, “Structural classes with complete computable deadlines”: Identify useful classes in which every recurrent path existence instance has a computably bounded recurrent path. Finite-state regions satisfy a much stronger lasso property, while Theorem 8.8 excludes a general result. Intermediate possibilities include well-structured resource restrictions, limited zero tests, and recurrence supported by computably bounded return regions. Any proposed class needs a proved closure property that excludes the halting-time tree mechanism.

Research question 33.13 (A numerically explicit universal quadratic relation; source 03). Instantiate the event-reflecting interpreter as a complete edge table, compile its sparse polynomial, and publish the exact dimension, monomial count, circuit size, and coefficient distribution. The first target should be a reproducible and formally checked instance, not a premature claim of minimality. A successful artifact would turn the current constructive existence theorem into a directly executable fixed-polynomial benchmark.

Overlapping question of source 11, “A completely expanded fixed universal law”: **Target.** Supply a particular finite universal checkpoint machine, run the compiler, publish its exact D , edge count, monomial count, and input encoding, and prove a simulation theorem for that exact table. The current report gives a fixed-law existence theorem and a generic finite-table compiler, but not this final universal artifact. Optimizing D while keeping height six and binary branching is a meaningful quantitative problem.

[write] Compare Report A’s question “Quantitative universal compilation” and Report B’s question $pqc:q:direct$.

Research question 33.14 (Height reduction versus dimension; source 03). The bound eight is sharp for this compiler because two parallel identical increments contribute four overlapping quadratic residual terms. Can a modified encoding attain coefficient height two, or even one, while retaining one equality, bounded branching, degree two, and a manageable fixed state dimension? Variable splitting can redistribute coefficients, but must not create unconstrained target coordinates or additional successors. The appropriate result is a proved quantitative tradeoff, not merely a smaller unexpanded expression.

[write] Partly answered, for a different encoding, by source 11: its incoming-edge law (theorem 15.1) has height at most 6 and drops the q control coordinates. It does not reach height two or one, and it changes the encoding, so the question for source 03’s encoding stays open (section 10).

Research question 33.15 (When are edge tags dispensable?; source 03). Tags guarantee that labelled choices remain distinct. For edge tables whose enabled labelled edges always have different control-and-counter targets, a tag-free encoding may be possible. Characterize the exact semantic condition and the minimal extra coordinates needed when only some edges collide. Preserve recurrence and the computable-deadline distinctions, not only reachability.

[write] Source 11 answers the dual question: given incoming-edge tags, the control coordinates are dispensable (theorem 15.1). Whether tags can be dropped stays open.

Research question 33.16 (Binary fairness compilation; source 03). The explicit fairness construction raises the outdegree from two to three. Can the same one-always-enabled-action specification be retained in a binary graph without introducing fair infinite runs that only traverse selection gadgets? A correct binary construction must account for continuous enabling during every intermediate state and must preserve the lock-based progress argument.

Overlapping question of source 11, “Fairness as a compiler parameter”: **Question.** Extend the incoming-edge method to multiple progress obligations and distinguish weak fairness, strong fairness, and generalized Büchi acceptance. A round-robin finite-control monitor handles finitely many Büchi sets qualitatively, but deadline bounds must be transformed explicitly. Scheduler fairness should not be conflated with the existence of an arbitrary recurrent branch; Harel’s classical fairness results are the appropriate starting point [15].

Research question 33.17 (One-dimensional and low-dimensional boundaries; source 03). The affine obstruction is dimension sensitive: for $D \geq 2$ it forces a finite target set, while $D = 1$ is functional. Determine stronger quantitative restrictions for quadratic graphs in very small dimension, under the same uniform branching and marker assumptions. This is different from reducing the number of registers in a counter model, because the present encoding also spends coordinates on controls and edge tags.

Research question 33.18 (Integer rather than natural coordinates; source 03). The compiler exploits nonnegative products. A replacement over unrestricted integers needs an explicit nonnegativity encoding or a different local construction. Standard changes of domain can alter degree, dimension, and branching multiplicities simultaneously. A useful target is a carefully stated integer-coordinate analogue that preserves recurrence and has a proved global successor bound, rather than a domain substitution that silently introduces many representations of one state.

Overlapping question of source 11, “Optimal degree under stronger geometric restrictions”: **Question.** What is the minimum degree and dimension when the local polynomial must be globally nonnegative on all real inputs, not merely on the nonnegative orthant, and must remain witness-free and binary-branching? The quartic construction is an upper bound. The degree-one obstruction does not settle the globally nonnegative quadratic case, whose zero set has a much more rigid affine geometry. Any claim of degree optimality there needs a separate theorem.

[write] *The two formulations ask the same beyond-the-orthant question from two sides: integer coordinates (source 03) and global nonnegativity on all real inputs (source 11). Compare Report A’s question `cdc:q:degree`, whose integer-domain part is related.*

Research question 33.19 (Formalizing the infinite semantic boundary; source 03). Prove the compiler in Lean first, then the deadline compactness equivalence and both explicit separating trees. These examples make useful regression tests for the intended theorem statements: an incorrect compactness lemma would derive recurrence for (18), while an incorrect effective choice lemma would produce a computable branch through T_D . A full formalization should record separately finite trace arithmetization, classical infinite existence, and computability of witnesses.

[write] *Compare Report A’s question `cdc:q:verified`, and the formalization plans of section 30, section 31 and section 32.*

Research question 33.20 (Optimal incomplete certificate families; source 03). A complete finite certificate family is impossible for the universal sets, but useful incomplete families can still be compared. Study which recurrent programs are certified by periodicity, monotone recurrence, finite-state abstractions, or computable return maps, and prove strict inclusions between these classes. The relevant optimization criterion is coverage of meaningful subclasses together with a small trusted checker, not an unattainable complete positive checker for every recurrent input.

Research question 33.21 (Sparse and low-interaction versions; source 11). **Question.** Can the same exact path semantics be realized by a quadratic polynomial in which every coordinate interacts with only a bounded number of other coordinates, with a controlled state-dimension increase? The global one-hot sums create dense selector interactions. Replacing them with local flow gadgets must preserve complete successor lists and must not introduce unbounded auxiliary choices. The relevant invariant is path correspondence, not just existential satisfiability.

34 Conclusion

The decisive boundary in Diophantine compilation is not the aesthetic form of the computational substrate. It is the information content and quantifier structure of the requested behavior. Finite successful computations are c.e. and hence Diophantine. Recurrent behavior requires coherent infinite choices, and finite branching alone does not remove that requirement.

A shared computable progress deadline supplies precisely the compactness needed to pass from coherent finite certificates to an infinite execution. It is equivalent to a nonempty effectively closed family of accepting schedules and, through the classical basis theorem, to the existence of an HIF accepting schedule. The degree-spectrum construction gives a separate quantitative answer: every c.e. degree can be exactly the minimum oracle strength needed to compute a successful clock. The halting-degree instance has accepting runs but no computable deadline at all.

These results coexist with a completely explicit finite arithmetization. The two-stack compiler uses natural witnesses and quadratic residuals, producing a nonnegative quartic with a proved labelled-trace bijection. The same computations occur in a compact rational guarded affine system; adding a real scheduler coordinate turns it into a deterministic map while making the initial-parameter quantifier visible. Fixed-arity MRDP then provides the correct alternating polynomial signatures, rather than an invalid universal existential representation of liveness.

The immediate research value is a precise architecture for extending ProveIt: exact finite compilers, coherent deadline trees, separately audited basis and degree arguments, and explicit arithmetic quantifier boundaries. This architecture makes both positive representations and impossibility results testable against the intended operational semantics.

35 Conclusion [\[source 03\]](#)

A single quadratic equation over natural source and target coordinates can express a universal local computational substrate without existential transition witnesses and without losing a uniform binary branching bound. The degree is optimal under these precise constraints. Nevertheless, small-degree local arithmetic does not collapse infinite behavior to finite arithmetic evidence.

The exact distinction is

$$\text{computable recurrent run} \implies \text{computably bounded recurrent run} \implies \text{recurrent run},$$

with both implications strict. Prescribed computable deadlines restore a closed finite-prefix condition and yield a Π_1^0 description; choosing some computable deadline raises the complexity to Σ_3^0 ; removing the effectivity restriction on the infinite witness reaches Σ_1^1 . These distinctions survive faithful effective implementations and remain visible in an explicit low-degree polynomial normal form.

For ProveIt, the practical lesson is architectural. Reuse the existing finite-trace Diophantine theorems, add an exact local compiler, and keep the infinite-path quantifiers explicit. The boundary itself is a useful theorem: it identifies where more clever finite arithmetization can help, and where uniform finite positive certification is mathematically impossible.

36 Conclusion [\[source 11\]](#)

A single local quadratic law can have an elementary one-step relation, a computable finite successor list, and at most two successors, while the existence of an infinitely recurrent run is analytic-complete. Low polynomial degree is therefore not a measure of the logical complexity of all temporal observations of the resulting system.

The decisive additional structure is not merely “more finite witnesses.” It is compatibility. Prescribed computable deadlines turn recurrence into a closed finite-prefix condition and admit exact quadratic certificates at each horizon. Existentially supplying an algorithm for the deadline has the sharp Σ_3^0 cost proved here. Supplying a computable run has that same index-set complexity but is a strictly stronger demand on an individual instance.

These distinctions are preserved by exact rational planar stack dynamics. They give the Diophantine research program a concrete next step beyond another existential representation of finite halting: build compilers whose contracts include the kind of infinite witness they preserve, and make the quantifiers visible in the theorem statement.

37 Provenance

[write] This report was built from three manuscripts in the batch-62 write phase of Provel’s incoming reports.

Source	Batch, manuscript	Archive (arrival commit)	Pin	Placed
10 (base)	62, 10	Beyond_Halting_Diophantine_Liveness.zip (b57c0b5ff); main file beyond_halting.tex, 34-page PDF	b6bf6406a, b998f70c6	adeddcecc
03	62, 03	Quadratic_Diophantine_Dynamics.zip (b57c0b5ff); 26-page PDF	4e128356d	adeddcecc
11	63, 01	quadratic_infinite_computation.zip (a4268e78e); 27-page PDF	e18718e83	62f1ad07c

Source 10 inspected the MRDP source at b6bf6406a and the MRDP guide and the Turing-degree README at b998f70c6, an ancestor of b6bf6406a; both pins are kept. Computability/ is unchanged from every pin to the write, so every repository statement of the three sources was checked at the write and is true there.

What each contributed. Source 10: the progress-deadline semantics, coherent compactification, the clock-closed-class-hyperimmune-free equivalence with relativization, the full hierarchy with the bounded-gap row, the DNR and halting-oracle separations, exact c.e. deadline spectra, the deterministic table, the two-stack quartic compiler with exact counts, the clock-transfer theorem, the compact rational piecewise-affine realization and the initial-parameter hierarchy, the fixed-arity quantifier signatures and the no-go theorem, a formalization plan, twelve questions, and its program code/10-progress-deadlines-clock_compiler.py with tests. Source 03: the auxiliary-free degree-two control-vector compiler with the sharp height 8 and its circuit count, the affine bounded-branching obstruction and degree minimality, a fixed universal quadratic graph with its Hit, Inf, Rec,

CB, CP classifications and strict chain, the nine-coordinate counterexample, the Π_1^0 sharpness of prescribed deadlines, the transport theorem for three recurrence notions, the one-obligation weak-fairness reduction, the finite-certificate obstruction and the lasso fragment, a formalization plan, ten questions, and its compiler and verification programs. Source 11: the incoming-edge degree-two law with height 6 and its sum-of-squares quartic companion, the positive-semidefinite-plus-nonnegative observation, semilinearity of the one-step graph, pinned finite-history certificates, a fixed law with Σ_1^1 -complete recurrence, the nonnegative degree-one obstruction, the budget counterexample, the temporal rows EFG, AGF and AFG, degree-two deadline certificates, the Σ_3^0 classifications across machines, both separations, the $0'$ -computable run, the planar rational realization with base-three counter routines and the terminating-macro lemma, the no-finite-compiler theorem with its compilation contract, a reuse checklist, a formalization plan, eight questions, and its programs.

Where the merge had to choose.

- *Base.* Source 10, because it is the more general: arbitrary computable finitely branching (binary-controlled) systems, the hyperimmune-free equivalence and the degree spectra.
- *What is printed once* (section 2.2). Five statements that hold for all systems of a class are printed once with the other sources' proofs as second and third proofs; completeness statements, separations, transport theorems and obstructions for different fixed systems are each printed for their own system in sections marked as routes, and one table (section 6.7) collects them.
- *Order.* Source 10's sections were regrouped into five Parts; its scheduler-degree section follows the hierarchy instead of preceding it. Sources 03 and 11 are distributed over the Parts by subject.
- *Questions.* Source 10's twelve questions, source 03's ten and source 11's eight are all printed. Two of source 03's and four of source 11's overlap a question of source 10, and three more of source 11's overlap a question of source 03; each of these nine is printed after the question it overlaps, as a paragraph naming its source and title. That leaves twenty-one question environments: source 10's twelve, eight of source 03's and one of source 11's. The questions of sources 03 and 11 were subsections and are printed as question environments with unchanged text.
- *Notation.* No mathematical symbol was renamed; section 2.3 matches the three vocabularies and warns about the letters that change meaning. Heights 8 and 6 are printed as bounds for different encodings.
- *Additions.* Three merge notes correct or complete source text: the pinned initial tag for a single-fold version of proposition 14.1; a textbook reference beside source 03's citation of a preprint for the non-arithmeticity of Σ_1^1 -complete sets; and the verification of source 11's theorem numbers in Cenzer, Marek and Remmel (Theorems 2.6(g) and 2.10(e) of arXiv:1303.6555, checked at the write; the dossier had left them unverified).
- *Front matter.* The delivered author lines and PDF metadata named the assistant used or the addressee; they are replaced by the neutral description "AI-assisted research manuscript". The shipped source notes of the three sources (verbatim) describe how each read the repository; that description is not repeated here.

Shipped files and delivered names. Source 10's `code/clock_compiler.py`, `code/test_clock_compiler.py`, `artifacts/example_certificate.json`, `artifacts/test_report.json` and `research/source_ledger.md` are shipped as `code/10-progress-deadlines-clock_compiler.py`, `code/10-progress-deadlines-test_clock_compiler.py`, `data/10-progress-deadlines-example_certificate.json`, `data/10-progress-deadlines-test_report.json` and `10-progress-deadlines-source_ledger.md`. Source 03's `code/quadratic_compiler.py`, `code/verify.py`, `Makefile`, the six files of `examples/` and `verification/`, `PROOF_AUDIT.md` and `SOURCES.md` are shipped with the prefix `03-quadratic-recurrence-` in `code/`, `data/` and the report root. Source 11's `code/quadratic_dynamics.py`, `code/verify.py`, `code/__init__.py`, `Makefile`, the three files of `data/` and `SOURCE_NOTES.md` are shipped with the prefix `11-quadratic-laws-` likewise. The three manuscripts, their delivered READMEs and PDFs, and the checksum manifests are not shipped; this text replaces the manuscripts. The README lists every shipped file.

A Reproduction and artifact manifest

From the archive root, run:

```
python3 code/test_clock_compiler.py
python3 code/clock_compiler.py
latexmk -pdf -interaction=nonstopmode -halt-on-error beyond_halting.tex
```

The Python programs use the standard library only. The LaTeX source uses standard TeX Live packages, including New PX fonts, AMS mathematics, Microtype, TikZ, and `tcolorbox`. Alternatively, run pdfLaTeX three times to resolve the table of contents and cross-references. No external bibliography processor is required.

File	Purpose
<code>beyond_halting.tex</code>	Complete editable article source.
<code>beyond_halting.pdf</code>	Rendered and visually checked article.
<code>code/clock_compiler.py</code>	Exact finite-horizon polynomial compiler and sample export.
<code>code/test_clock_compiler.py</code>	Reproducible finite arithmetic tests.
<code>artifacts/example_certificate.json</code>	Full 52-variable, 137-residual sample and witness.
<code>artifacts/test_report.json</code>	Results from the executed tests.
<code>README.md</code>	Build commands, scope, and proof-status summary.
<code>research/source_ledger.md</code>	Inspected source references and provenance boundaries.

[write] *The listing and the table above keep source 10's delivered layout. Of the files it names, the article source and PDF, the README and the `research/` ledger are not shipped under those names: this report's `article.tex` and `article.pdf` replace the manuscript, this report's README replaces the delivery README, and the ledger is shipped as `10-progress-deadlines-source_ledger.md`; the programs and artifacts are shipped under the names of section 37. Rerun the programs on a copy with the delivered layout, as the README describes; in this repository's shell `py` replaces `python3` and `python`.*

B Reproduction guide [\[source 03\]](#)

From the package directory, run:

```
python code/verify.py
python code/quadratic_compiler.py examples/finite_visits.json \
  --output examples/finite_visits_polynomial.json
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The Python implementation requires Python 3.10 or later and no third-party packages. The PDF uses standard pdfLaTeX packages listed in its preamble. The JSON polynomial format lists variables in source-state order followed by target-state order. A monomial is a sorted tuple of variable indices; the empty tuple denotes a constant. Repeated indices denote a square.

The verification script regenerates the polynomial examples, sample finite runs, and its report. It intentionally does not implement an oracle for the diagonal trees or attempt to test an infinite-path theorem by bounded search. Its complete source states every finite range and sampling choice.

[write] *The listing keeps source 03's delivered layout; its programs are shipped as `code/03-quadratic-recurrence-quadratic_compiler.py` and `code/03-quadratic-recurrence-verify.py`, its examples and reports under `data/03-quadratic-recurrence-*`. Rerun on a copy with the delivered layout (README); in this repository's shell `py` replaces `python`.*

C The explicit eight-dimensional budget example [\[source 11\]](#)

Order the seven edges as follows:

Index	Source	Target	Effect or guard
0	*	G	dummy no-op
1	G	I	no-op
2	G	D	no-op
3	I	G	increment x
4	D	A	decrement x
5	D	H	test $x = 0$
6	A	D	no-op

The current state is $(x, b_0, \dots, b_6)$ and the next state is $(y, c_0, \dots, c_6)$. The exact quadratic polynomial, in residual form, is

$$\begin{aligned}
 P_2 = & \left(\sum_{j=0}^6 b_j - 1 \right)^2 + \left(\sum_{j=0}^6 c_j - 1 \right)^2 + c_0^2 \\
 & + (b_0 + b_3 - c_1 - c_2)^2 + (b_1 - c_3)^2 \\
 & + (b_2 + b_6 - c_4 - c_5)^2 + (b_4 - c_6)^2 + b_5^2 \\
 & + (y - x - c_3 + c_4)^2 + c_5x.
 \end{aligned}$$

Replace the final c_5x by $c_5^2x^2$ for P_4 . The marker is $A(b) = b_4$, and the initial state is $(0, 1, 0, 0, 0, 0, 0)$. This complete example can be checked directly without trusting any program generator.

The j th marked visit after choosing budget $n \geq 1$ occurs at

$$\tau_j = 2n + 2j, \quad 1 \leq j \leq n.$$

Indeed choosing the budget takes $2n$ transitions, entering D takes one, and its first decrement takes one more to reach A . Subsequent marked visits are separated by two transitions. Hence the last visit is at time $4n$. This proves the finite-prefix assertion for all n , beyond the tested range.

D A minimal proof checklist for reuse [\[source 11\]](#)

A future application of this report's normal form should discharge all of the following obligations in its own semantics:

- (1) Its state encoding is injective on the intended finite configurations.
- (2) Every emitted counter edge has one of the permitted effects and guards.
- (3) Dummy initialization cannot occur as a later transition.
- (4) Incoming-edge identity is retained when different choices would otherwise lead to the same machine configuration.
- (5) Marked events correspond to the intended observations, rather than to unbounded internal work.
- (6) A claimed computable deadline is total, and its time unit is specified.
- (7) Finite-prefix certificates enforce all earlier deadlines simultaneously.
- (8) Any macro simulation used for an infinite property has terminating internal routines or a proof that internal divergence cannot satisfy the observation.

Omitting one of these conditions can preserve ordinary finite reachability while invalidating a recurrence or witness-spectrum conclusion.

[write] *Source 11's reproduction commands are in section 29; its programs and data are shipped with the prefix 11-quadratic-laws- (README).*

E A compact theorem-dependency ledger

Endpoint	Principal ingredients	Verification status
Coherent compactification	Finite run semantics; computable binary tree; König compactness	Conventional proof.
Clock/closed/HIF equivalence	Compactification; uniform bounds on a compact class; classical HIF basis	Conventional proof; basis attributed.
C.e.-degree spectrum	Bounded enumeration tree; unary cost; enumeration modulus	Conventional explicit construction.
Hierarchy upper bounds	Finite decidability; common program index; analytic schedule quantifier	Conventional proof.
Hierarchy lower bounds	Unary witness selection; decidable search; recursive-tree normal form	Conventional reductions; recurrence endpoint classical.
Finite quartic bijection	One-hot indicators; canonical sentinel stacks; inactive slack equations	Conventional proof and finite exact tests.
Compact affine realization	Rational push/pop identities; finite linear guards	Conventional proof and exact identity tests.
Initial-parameter hierarchy	Cantor digit coding; rational periodicity; schedule-degree equivalence	Conventional proof.
Fixed-polynomial forms	MRDP; polynomial parameter pairing; quadratic circuit lifting	Classical backend; no general extractor implemented.
No-go theorem	Diophantine sets are c.e.; strict hierarchy and analytic hardness	Conventional consequence.

References

- [1] Vladimir Reshetnikov, *ProveIt: Diophantine/MRDP.lean*, repository source at commit b6bf6406a4c49017fedc6fba068b36c634987150, inspected September 30, 2026. [Pinned theorem source](#).
- [2] Vladimir Reshetnikov, *MRDP: recursively enumerable if and only if Diophantine*, ProveIt repository guide, revision b998f70c6886e6a00339a6f4a02ed8625324d357, inspected September 30, 2026. [Pinned MRDP guide](#).
- [3] Vladimir Reshetnikov, *Turing degrees in Lean and Rocq*, ProveIt repository README, revision b998f70c6886e6a00339a6f4a02ed8625324d357, inspected September 30, 2026. [Pinned Turing-degree project guide](#).
- [4] Yuri V. Matiyasevich, “Enumerable sets are Diophantine,” *Soviet Mathematics Doklady* **11** (1970), 354–358. English translation of *Doklady Akademii Nauk SSSR* **191** (1970), 279–282. Reprinted in *Mathematical Logic in the 20th Century* (2003), DOI: [10.1142/9789812564894_0013](#).
- [5] Carl G. Jockusch, Jr. and Robert I. Soare, “ Π_1^0 classes and degrees of theories,” *Transactions of the American Mathematical Society* **173** (1972), 33–56. DOI: [10.1090/S0002-9947-1972-0316227-0](#).
- [6] Keng Meng Ng, Frank Stephan, Yue Yang, and Liang Yu, “Computational aspects of the hyperimmune-free degrees,” in *Proceedings of the 12th Asian Logic Conference*, World Scientific, 2013, 271–284. DOI: [10.1142/9789814449274_0015](#). Author manuscript: https://personal.ntu.edu.sg/kmng/Files/Papers/hif_ALC.pdf.
- [7] David Harel, Amir Pnueli, and Jonathan Stavi, “Propositional dynamic logic of nonregular programs,” *Journal of Computer and System Sciences* **26**(2) (1983), 222–243. DOI: [10.1016/0022-0000\(83\)90014-4](#). The recurrent-machine completeness endpoint is Proposition 5.1, also explicitly cited and stated in [10].
- [8] Krishnendu Chatterjee and Nathanaël Fijalkow, “Finitary languages,” 2011. <https://arxiv.org/abs/1101.1727>.
- [9] Cristopher Moore, “Unpredictability and undecidability in dynamical systems,” *Physical Review Letters* **64** (1990), 2354–2357. DOI: [10.1103/PhysRevLett.64.2354](#). Author summary: <https://sites.santafe.edu/~moore/pubs/gs.html>.
- [10] Javier Esparza and Valentin Krasotin, “Regular model checking for systems with effectively regular reachability relation,” in *50th International Symposium on Mathematical Foundations of Computer Science (MFCS 2025)*, *Leibniz International Proceedings in Informatics* **345**, article 45, 2025. DOI: [10.4230/LIPIcs.MFCS.2025.45](#). Full version: <https://arxiv.org/abs/2506.18833>.
- [11] Vladimir Reshetnikov and contributors. *ProveIt: Hilbert’s tenth problem*, repository documentation. Snapshot 4e128356d0ef75308be8ed405d89aea2ffdb8a57, inspected 30 September 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/README.md>. Source inspection only; the repository was not rebuilt for this manuscript.
- [12] Vladimir Reshetnikov and contributors. *Diophantine bounded universals and finite traces*, *DiophantineTrace.lean*, same pinned ProveIt snapshot. <https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean>.
- [13] J. C. Shepherdson and H. E. Sturgis. Computability of recursive functions. *Journal of the ACM* **10**(2) (1963), 217–255. doi:[10.1145/321160.321170](#). Classical register-computation background; the operational stack construction needed here is included in Section 13.

- [14] David Harel. Recurring dominoes: Making the highly undecidable highly understandable. *North-Holland Mathematics Studies* **102** (1985), 51–71. doi:10.1016/S0304-0208(08)73075-5.
- [15] David Harel. Effective transformations on infinite trees, with applications to high undecidability, dominoes, and fairness. *Journal of the ACM* **33**(1) (1986), 224–248. doi:10.1145/4904.4993. Author-institution publication record and abstract inspected.
- [16] Olivier Finkel. Highly undecidable problems about recognizability by tiling systems. Preprint, 2008, arXiv:0811.3704. The inspected preprint supplies related analytical-hierarchy background and infinite-tiling results. Its language-level problems are not identified with the finite-input predicates studied here.
- [17] [write] Hartley Rogers, Jr., *Theory of Recursive Functions and Effective Computability*, McGraw-Hill, 1967. Textbook reference for the analytical hierarchy, added in the merge.
- [18] Vladimir Reshetnikov and contributors. *ProveIt: Hilbert’s tenth problem documentation*. Repository snapshot e18718e837d43e162252f9a314e8cb797fbd1a1f, inspected September 30, 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/e18718e837d43e162252f9a314e8cb797fbd1a1f/Computability/HilbertTenthProblem/README.md>.
- [19] Vladimir Reshetnikov and contributors. *MRDP: recursively enumerable if and only if Diophantine*. Same repository snapshot. <https://github.com/VladimirReshetnikov/ProveIt/blob/e18718e837d43e162252f9a314e8cb797fbd1a1f/Computability/HilbertTenthProblem/Lean/MRDP.md>.
- [20] Miroslav Chodil and Antonín Kučera. The satisfiability and validity problems for probabilistic computational tree logic are highly undecidable. *ICALP 2025, LIPIcs* **334**, 151:1–151:20, 2025. Full version: arXiv:2504.19207. <https://doi.org/10.4230/LIPIcs.ICALP.2025.151>; <https://arxiv.org/abs/2504.19207>.
- [21] Douglas Cenzer, Victor W. Marek, and Jeffrey B. Remmel. *Index sets for finite normal predicate logic programs*. arXiv:1303.6555, 2013. <https://arxiv.org/abs/1303.6555>.
- [22] Christopher Moore. Generalized shifts: unpredictability and undecidability in dynamical systems. *Nonlinearity* **4**(2), 199–230, 1991. <https://doi.org/10.1088/0951-7715/4/2/002>. Author’s publication list: <https://sites.santafe.edu/~moore/pubs.html>.
- [23] Anthony Widjaja To and Leonid Libkin. Recurrent reachability analysis in regular model checking. *LPAR 2008, LNCS* **5330**, 198–213, 2008. https://doi.org/10.1007/978-3-540-89439-1_15. Author-hosted text: <https://homepages.inf.ed.ac.uk/libkin/papers/lpar08b.pdf>.